



数字逻辑与处理器基础

Fundamentals of Digital Logic and Processor



第七讲 流水线技术

第2部分 冒险、包含冒险的数据通路

李学清

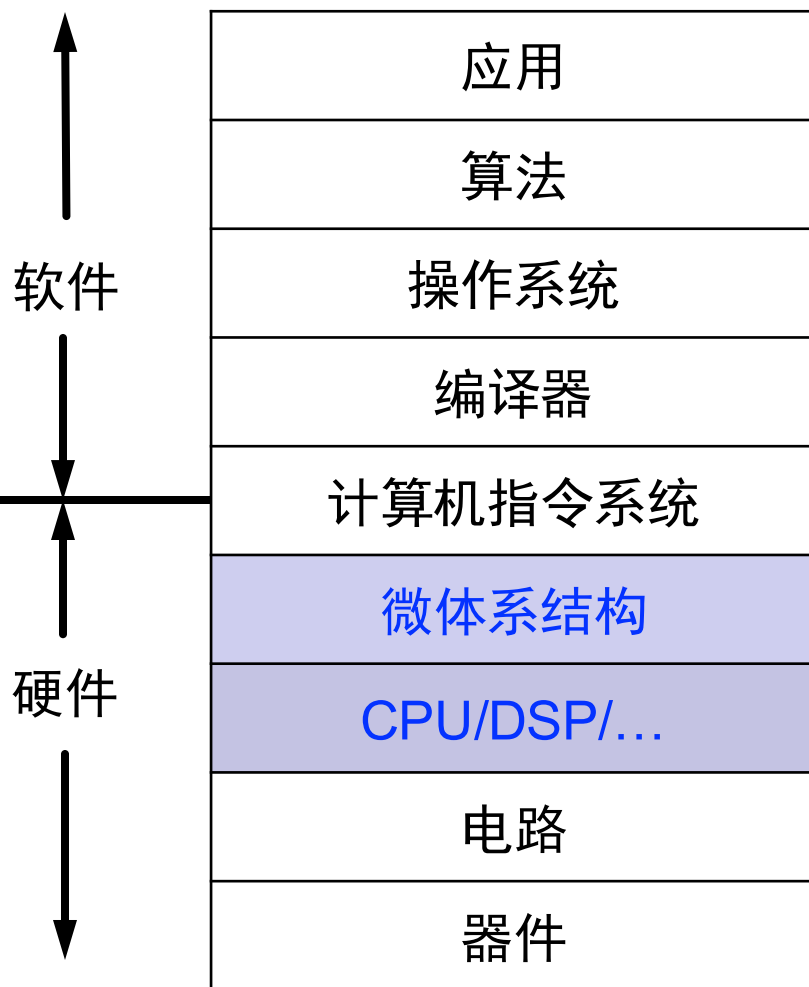
#腾讯会议： 453-1488-6548

助教： 唐文骏*/陈仲豪/曾令安

*本节内容负责答疑助教

2024年12月2日

微体系结构



计算机系统

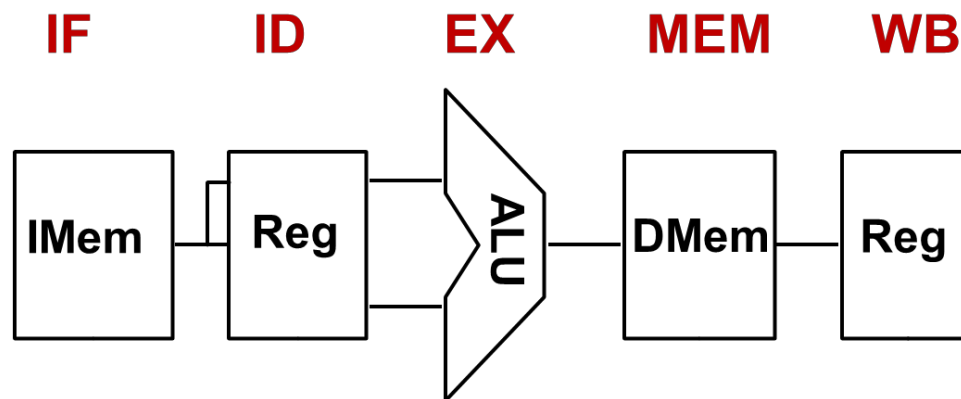
周	日期	暂定主要内容
1	9/9	绪论
2	9/21	布尔代数 (中秋节调休)
3	9/23	组合逻辑
4	9/30	时序逻辑
5	10/7	国庆节放假
6	10/14	时序逻辑
7	10/21	计算机指令系统
8	11/3	期中考试
9	11/4	计算机指令系统
10	11/11	微处理器
11	11/18	微处理器
12	11/25	流水线
13	12/2	流水线
13	12/9	流水线/存储器
15	12/16	存储器
16	12/23	存储器/IO与总线/总结

本节课目录

- **复习：流水线基本概念 P4**
- **复习：流水线数据通路 P6**
- **流水线的冒险**
 - **结构冒险 P10**
 - **数据冒险 P15**
 - **控制冒险 P38**
- **数据通路的实现（考虑冒险） P57**

回顾：流水线的基本概念

- 每条指令的执行过程，可以划分为若干步骤
- 通常，一条MIPS指令包含5个处理步骤：
 - **Instruction Fetch (IF)**: 取指，从存储器读取指令
 - **Instruction Decode / Register File (ID/RF)**: 指令译码并读取寄存器
 - **ALU Execution (EX)**: 执行运算或计算存储器地址
 - **Memory Data Access (MEM)**: 访问存储器
 - **Register Write-Back (WB)**: 将结果写回寄存器



回顾：流水线的性能

- 吞吐率

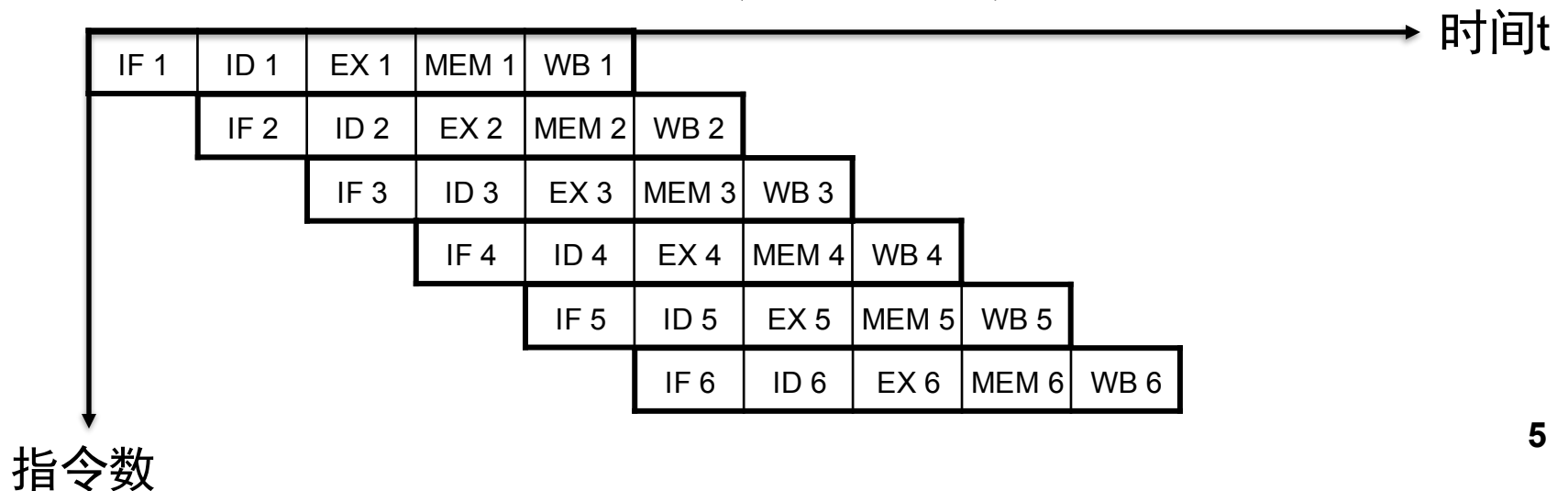
n 条指令， k 级流水线，级间延时（周期） Δt
 T_k 为 k 级流水线处理 n 条指令所用的时间

- 实际吞吐率：

$$TP = \frac{n}{T_k} = \frac{n}{(k + n - 1)\Delta t}$$

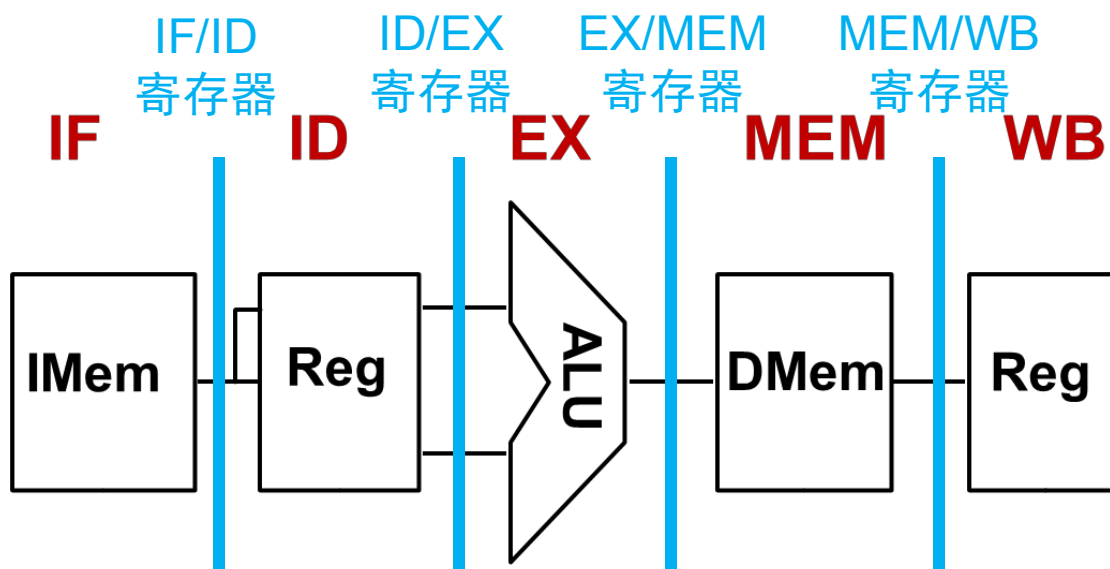
- 最大吞吐率：

$$TP_{max} = \lim_{n \rightarrow \infty} \frac{n}{(k + n - 1)\Delta t} = \frac{1}{\Delta t}$$

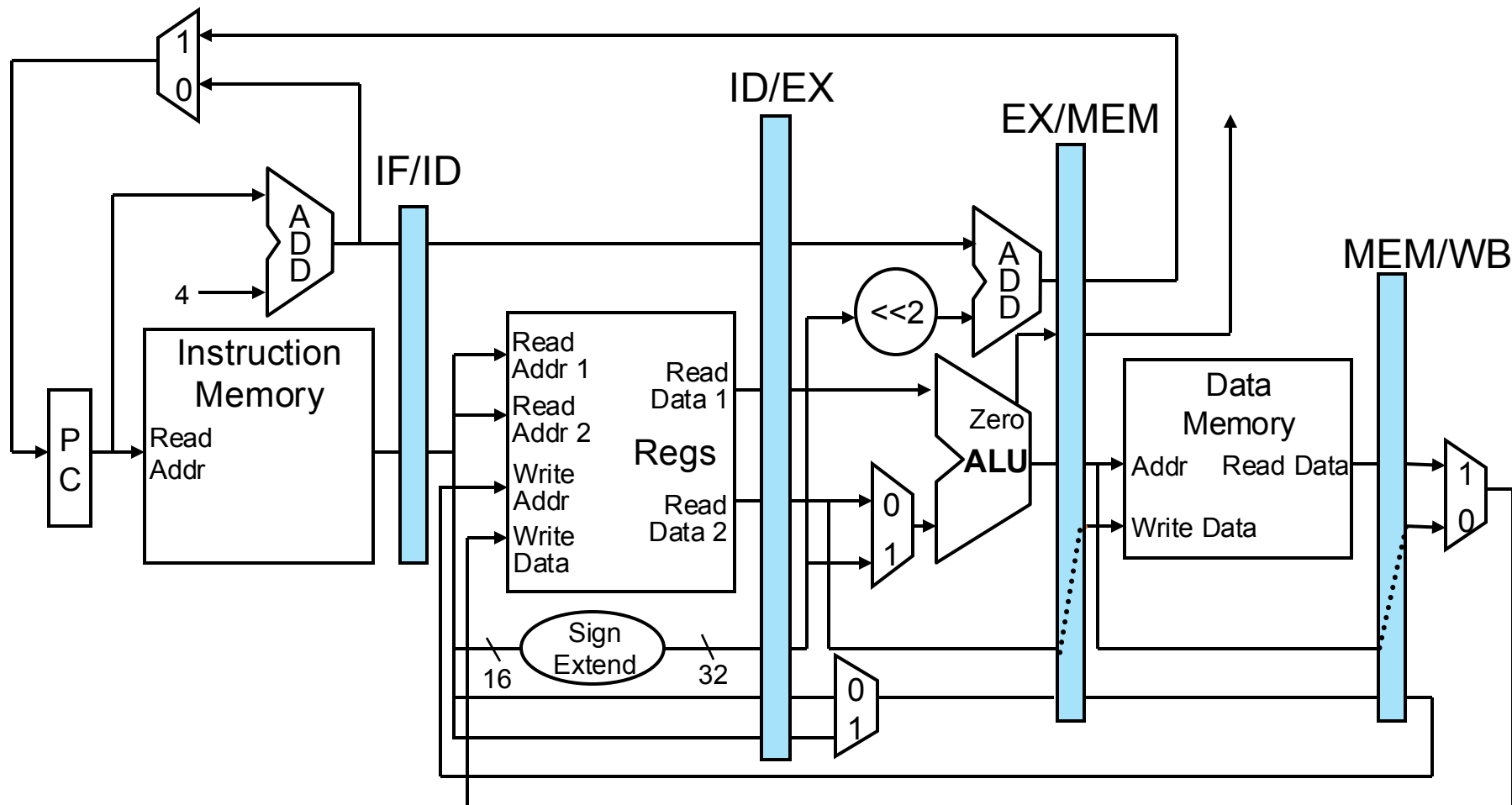


回顾：流水线的数据通路

- 可以分为5个步骤
 - 输入：来自于上一步骤上一周期
 - 输出：被下一步骤在下一周期使用
- 数据在不同周期间传递
 - 需要寄存器保持中间数据



回顾：流水线的数据通路



IF, ID, EX, MEM, WB流水级间插入寄存器
IF/ID, ID/EX, EX/MEM, MEM/WB

回顾：流水线的控制信号

- 控制信号可在IF之后的 ID 阶段产生

- 由控制逻辑电路译码生成

- 几个基本的控制信号

- 针对EX阶段的控制信号

- ALUSrc等——在第3个周期用到

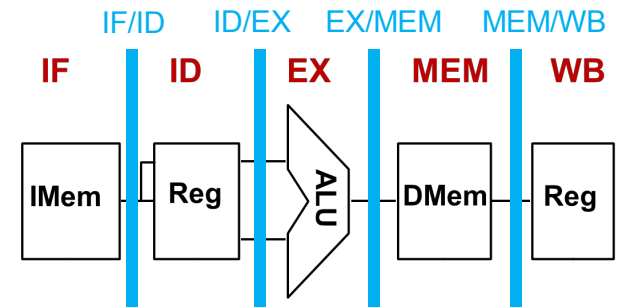
- 针对Memory Access的控制信号

- MemWr, Branch等——在第4个周期用到

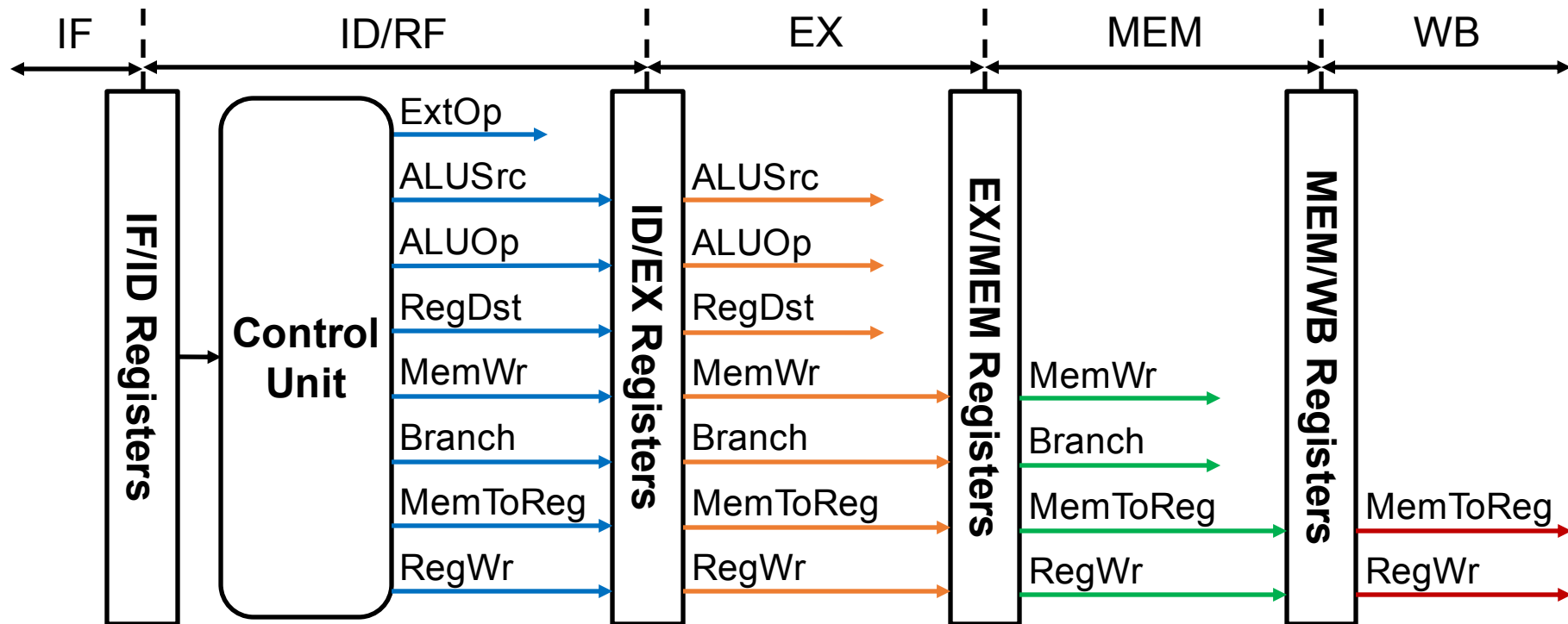
- 针对Write Back的控制信号

- MemToReg, RegWr等——在第5个周期用到

- 数据在流动，控制信号也一同流水线流动



回顾：流水线的控制信号



控制信号寄存器

ID/EX 寄存器

EX/MEM 寄存器

MEM/WB 寄存器

MEM/WB.RegWr:

MEM/WB寄存器中的RegWr控制信号

注：一些控制信号可在不同阶段解码

回顾：结构冒险

- 产生的原因

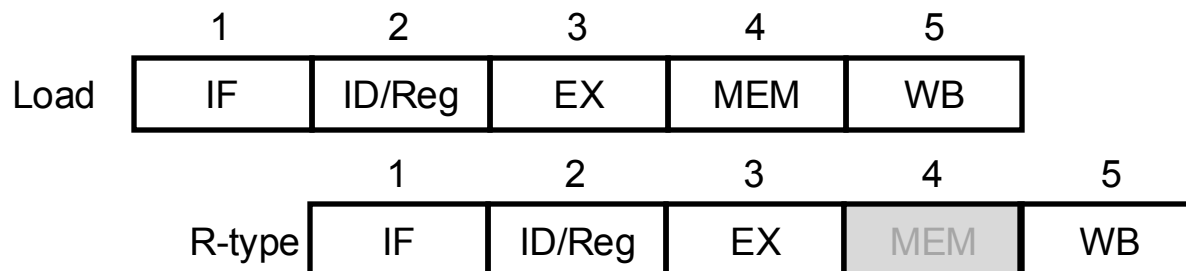
- 流水执行的多条指令在同一个周期使用同一个硬件资源

- 解决方法：

- 增加资源

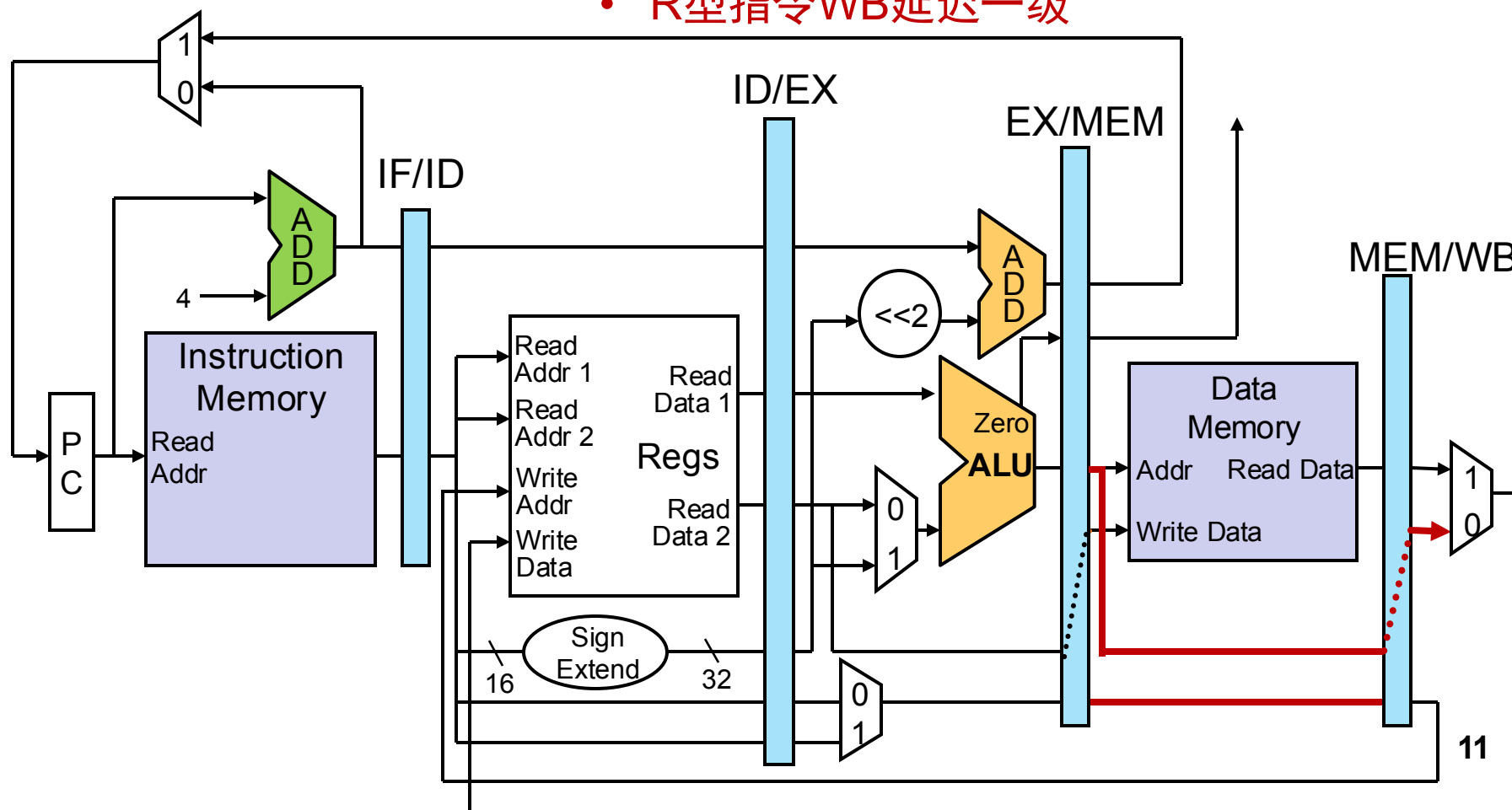
- 增加运算单元：例如PC+4和ALU运算分离
 - 采用指令存储器和数据存储器分离的哈佛体系结构

- 调整指令对资源占用的周期



回顾：结构冒险

- 增加硬件资源：
 - ALUOut计算分解为ALU和PCAdd
 - 额外的加法器用于PC+4
 - 分离的指令/数据存储器
 - R型指令WB延迟一级



从结构冒险到数据冒险

- **结构冒险（Structural Hazard or Resource Conflict）：资源矛盾！**
 - 两条指令需要使用同样的硬件
 - 解决方法：增加硬件资源；调整指令占用资源的时间
- **数据关联冒险（Data Hazard）：操作数是否已经更新？**
 - 指令依赖于流水线中运行的前序指令结果
- **控制冒险（Control Hazard）：下一条指令的PC是多少？**
 - 取指令依赖于流水线中前序指令的结果

从结构冒险到数据冒险、控制冒险

- 数据冒险

- 计算是否使用的是正确的数据？数据是否已经准备好？

- 控制冒险

- 是否执行了正确的指令？PC寄存器是否正确？

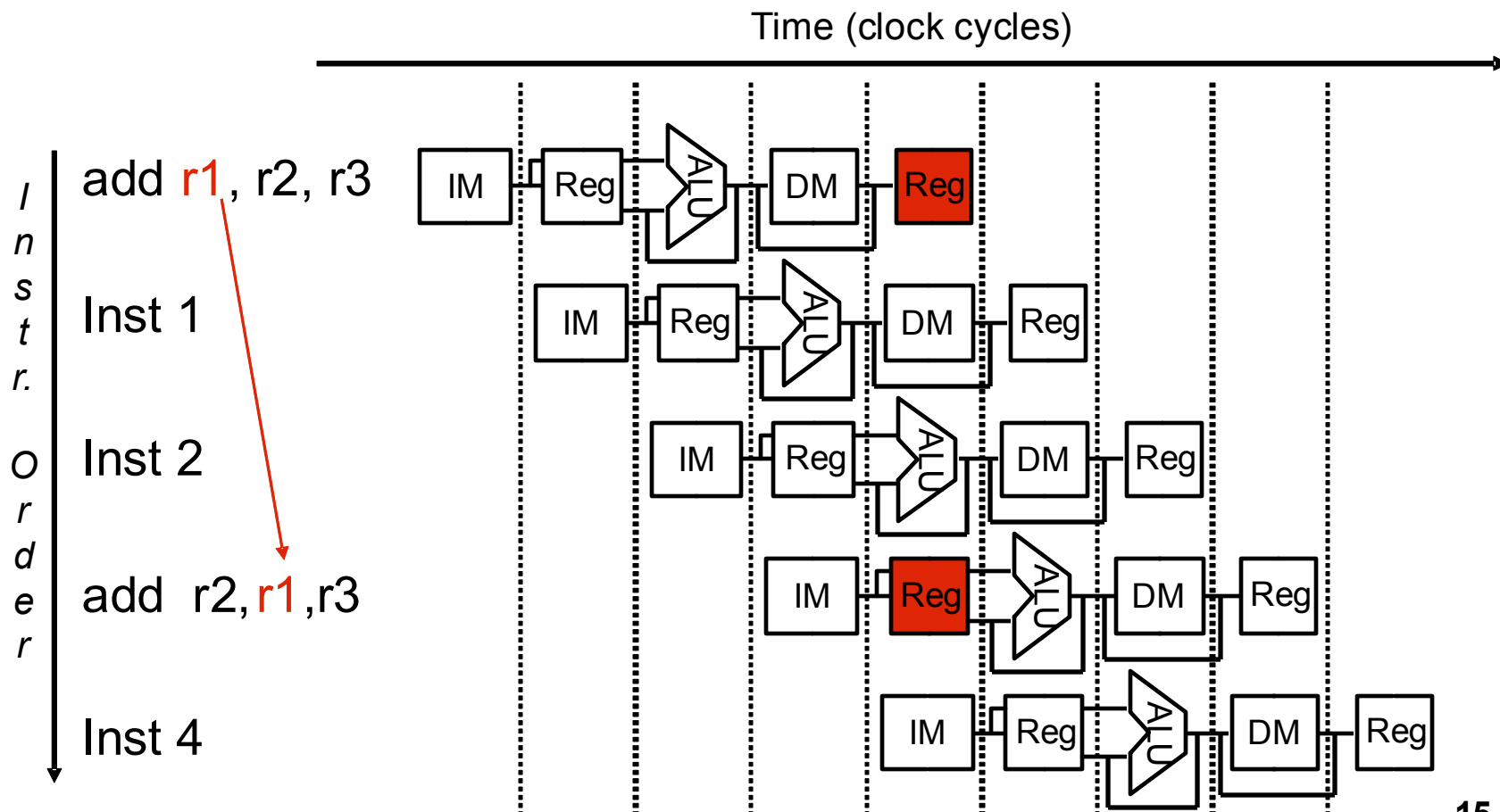
流水线	R类型	读存储器	写存储器	分支
IF	IR <= MemInst[PC]; PC <= PC + 4			
ID	op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]];			
EX	ALUOut <= A op B	ALUOut <= A + signext(IR[15:0])		ALUOut <= A - B PCAdd <= PC + signext(IR[15:0]) << 2
MEM		MDR <= MemData[ALUOut];	MemData[ALUOut] <= B	if (Zero) PC <= PCAdd
WB	Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		13

本节课目录

- 复习：流水线基本概念 P4
- 复习：流水线数据通路 P6
- 流水线的冒险
 - 结构冒险 P10
 - **数据冒险 P15**
 - 控制冒险 P38
- 数据通路的实现（考虑冒险） P57

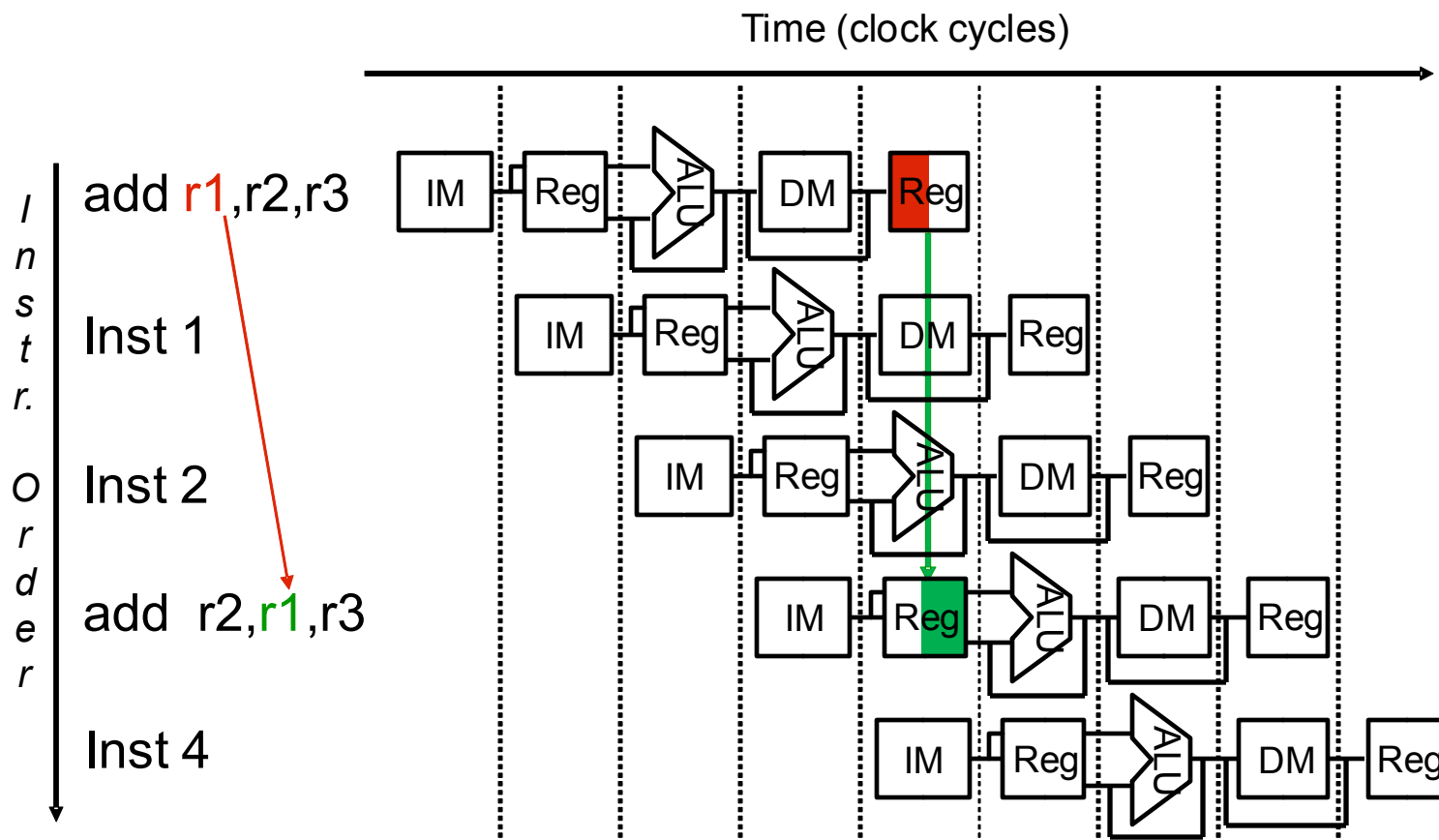
寄存器堆的访问

- 同时对寄存器堆进行读取和写入：依靠硬件来解决冲突



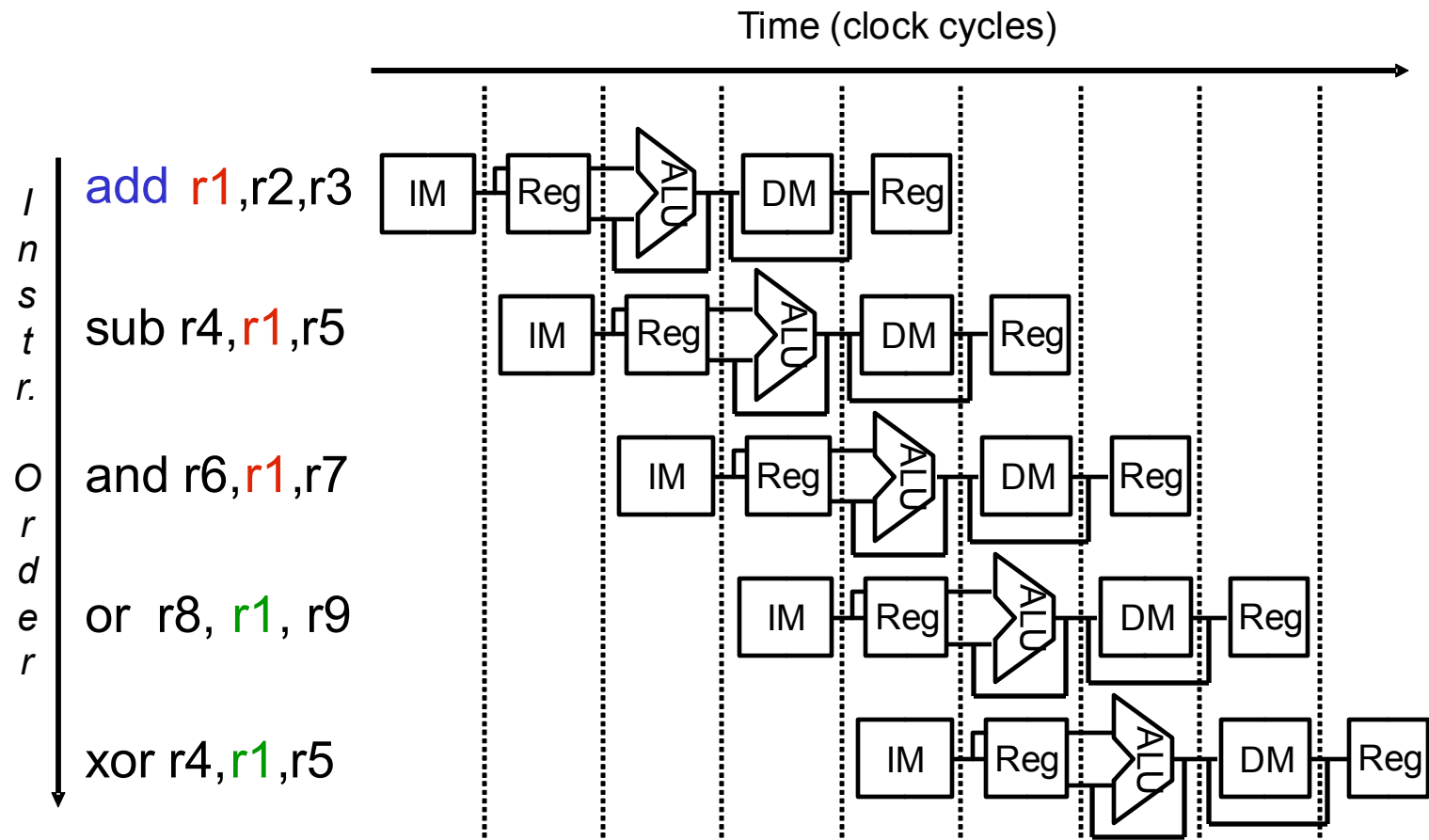
寄存器堆的访问

- 提供端口同时支持读&写：一个周期内先写后读
 - 依靠硬件实现解决
- 硬件实现保证能先写后读



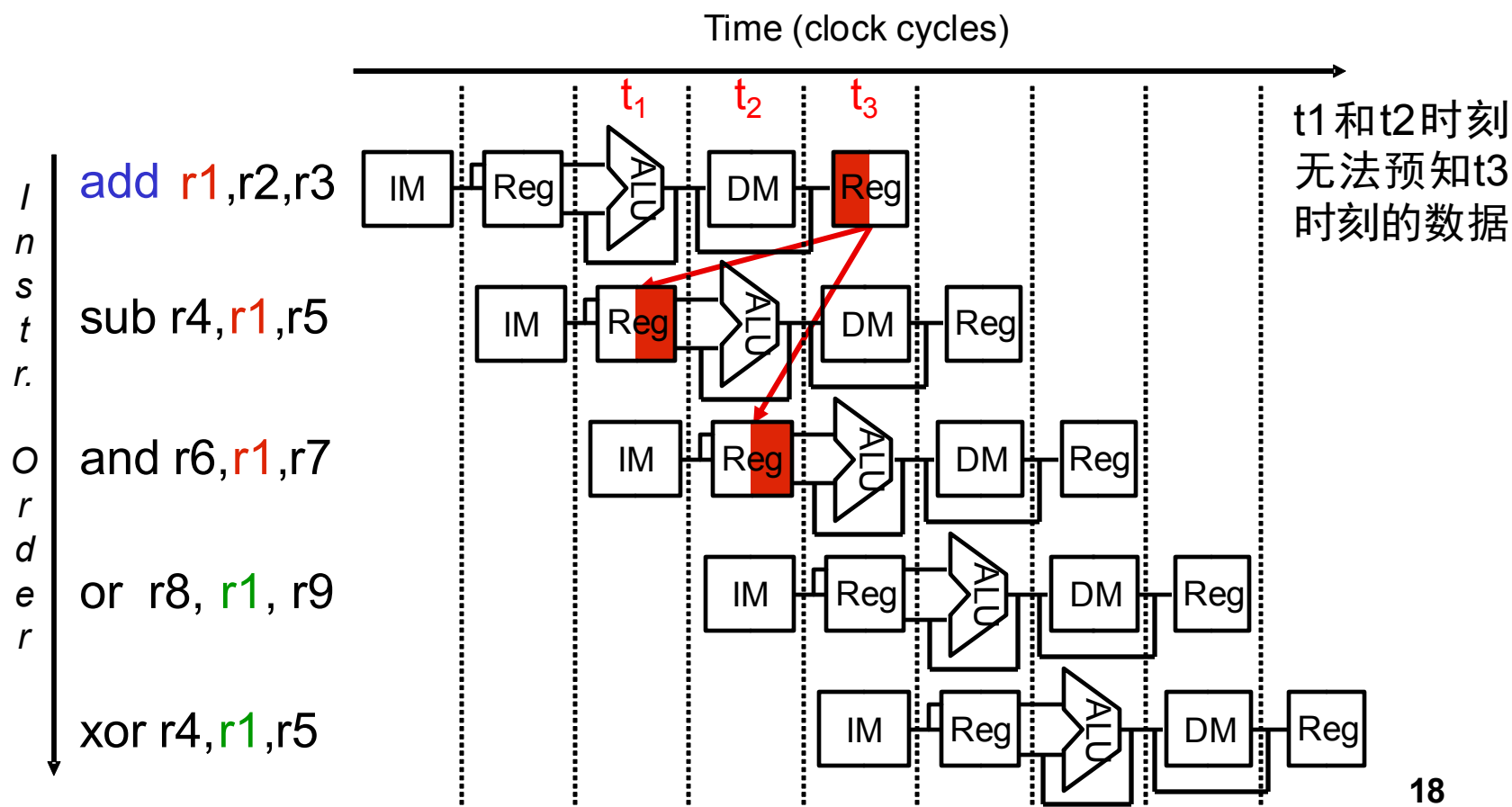
寄存器使用引起的冒险

- 是否存在冒险？
 - r1 is used right in the next instruction



寄存器使用引起的冒险

- 指令在寄存器使用上的时间顺序关联引起了数据冒险
- Read after write (RAW) data hazards



寄存器使用引起的冒险

- 指令在寄存器使用上的时间顺序关联引起了数据冒险
- Read inst. After Write inst. (RAW)** data hazards

指令序列

I
n
s
t.
O
r
d
e
r

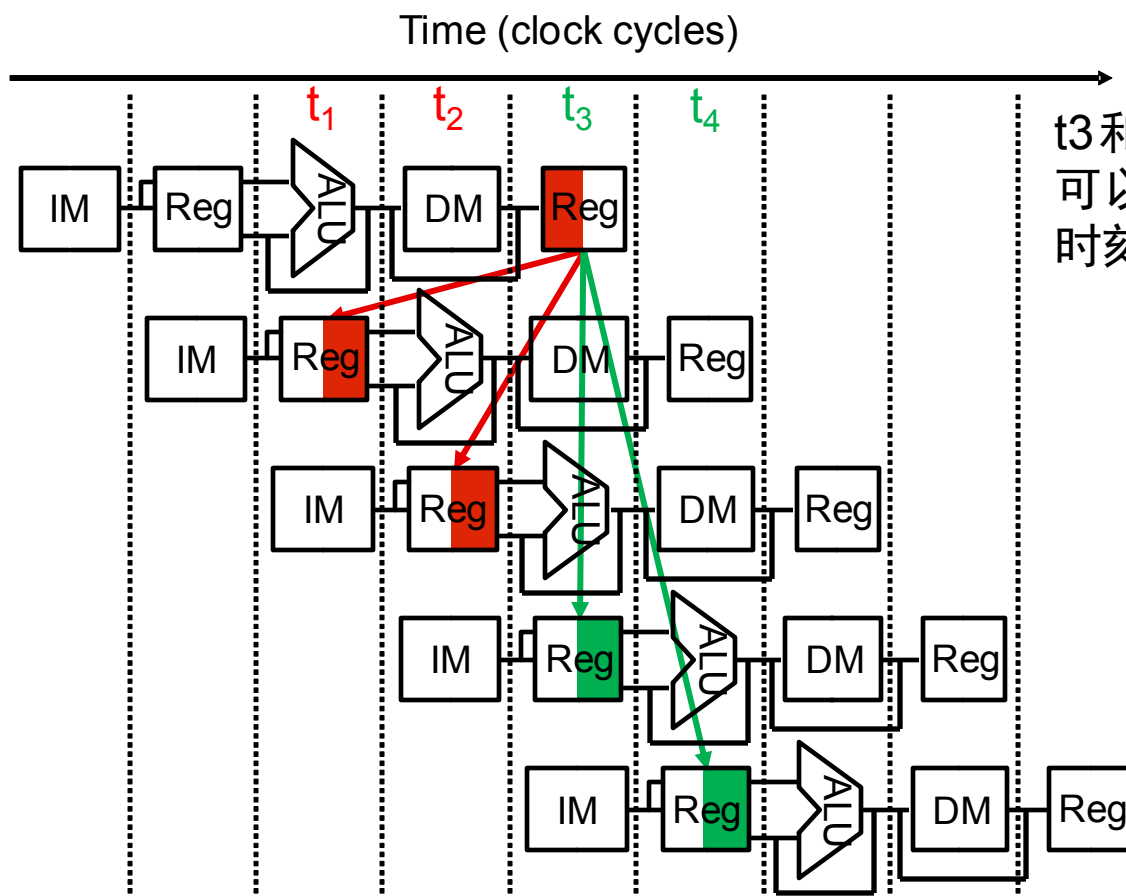
I1 **add** r1,r2,r3

I2 sub r4,r1,r5

I3 and r6,r1,r7

I4 or r8, **r1**, r9

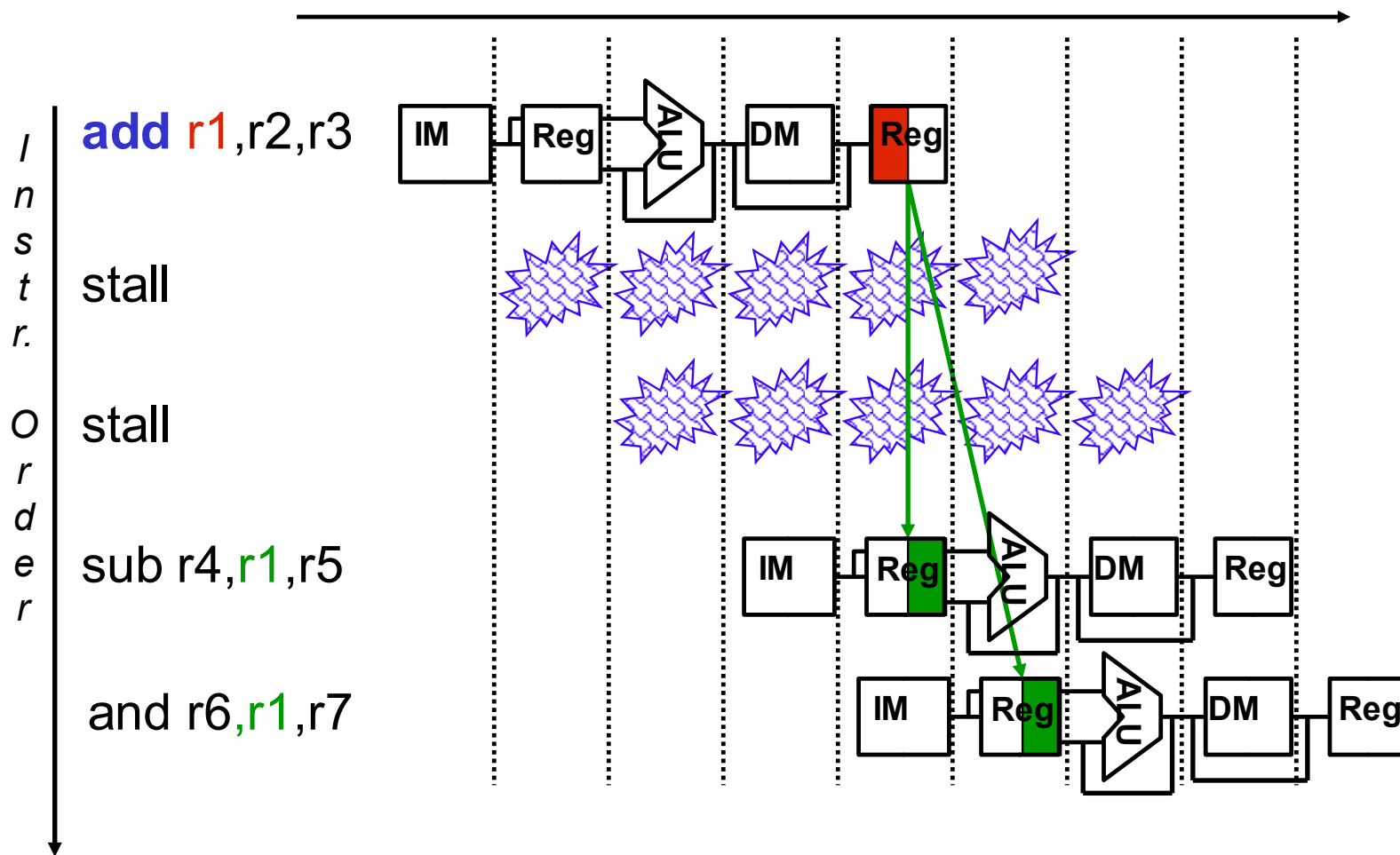
I5 xor r4,r**1**,r5



t₃和t₄时刻
可以获得t₃
时刻的数据

寄存器数据冒险-解决方案-1

- 流水线阻塞 (stall) : 插入stall等待 (空白指令)
 - 对吞吐率的影响?

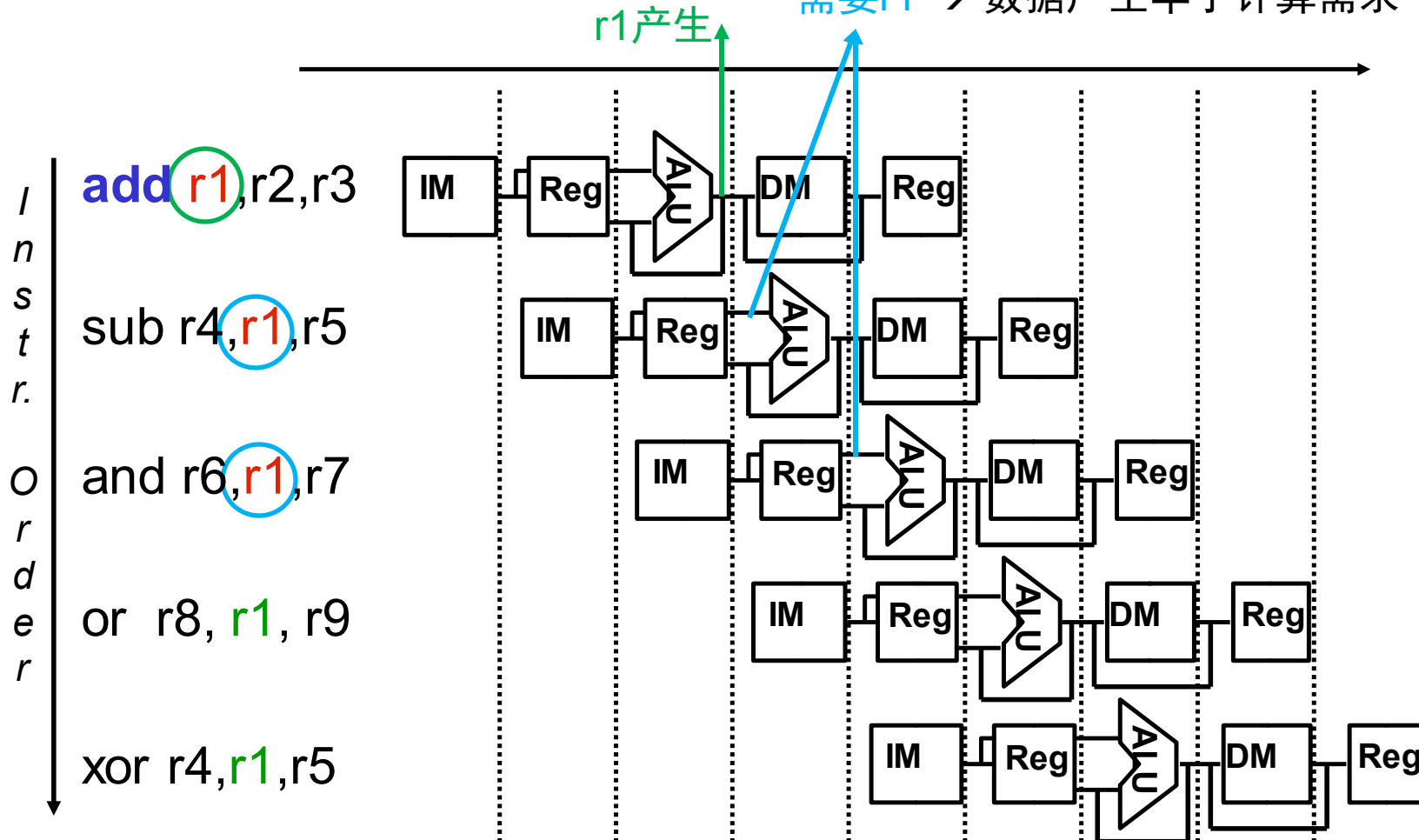


寄存器数据冒险-解决方案-1

- 是否可以减少stall的周期？

- r1寄存器更新前已经产生！

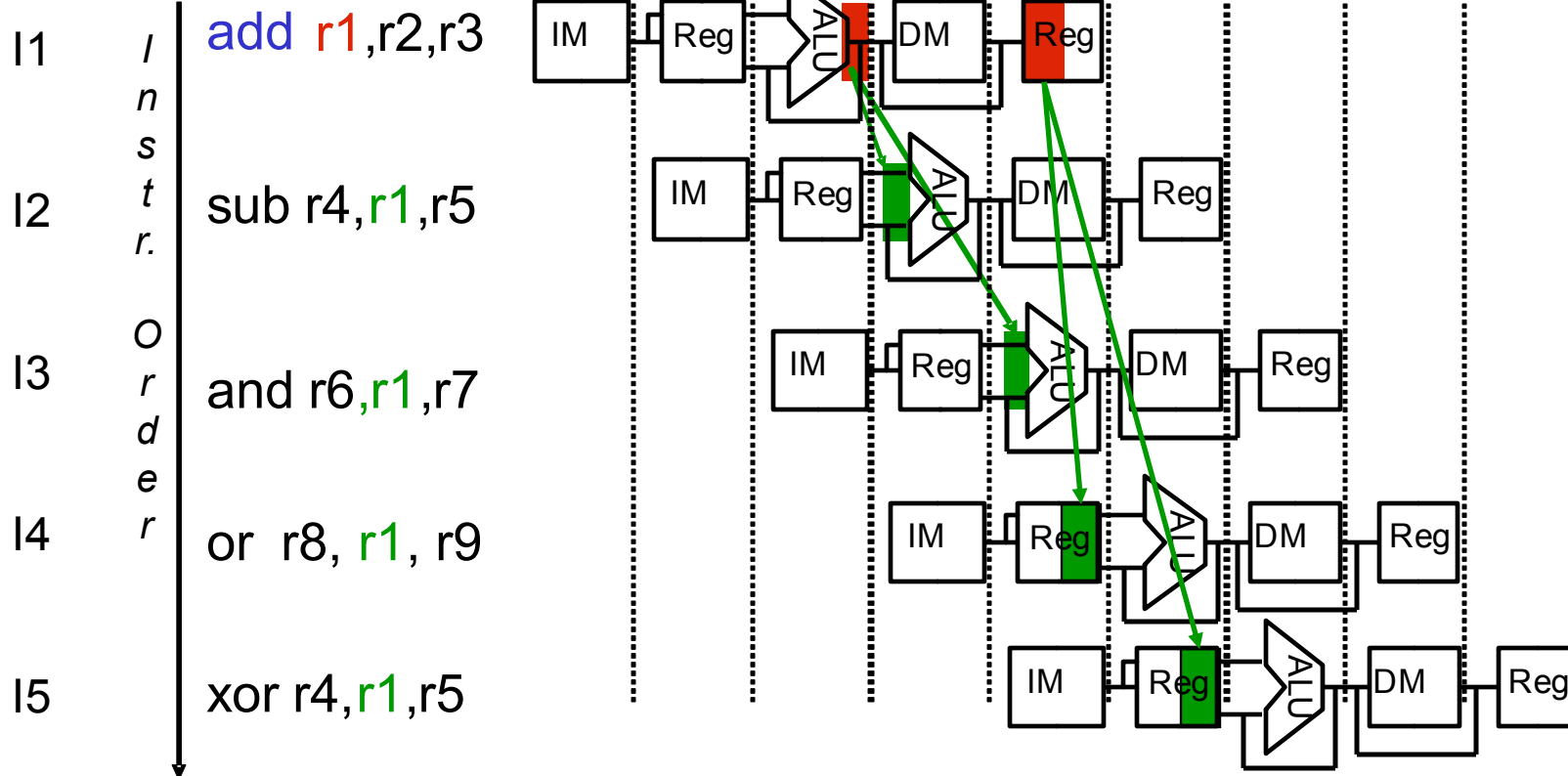
需要r1 → 数据产生早于计算需求



寄存器数据冒险-解决方案-2

- 流水线旁路 (Forward or Bypass)
 - 数据计算完成后, 马上提供给后续指令使用
 - 提前提供数据, 需要额外的硬件电路支持

指令序列



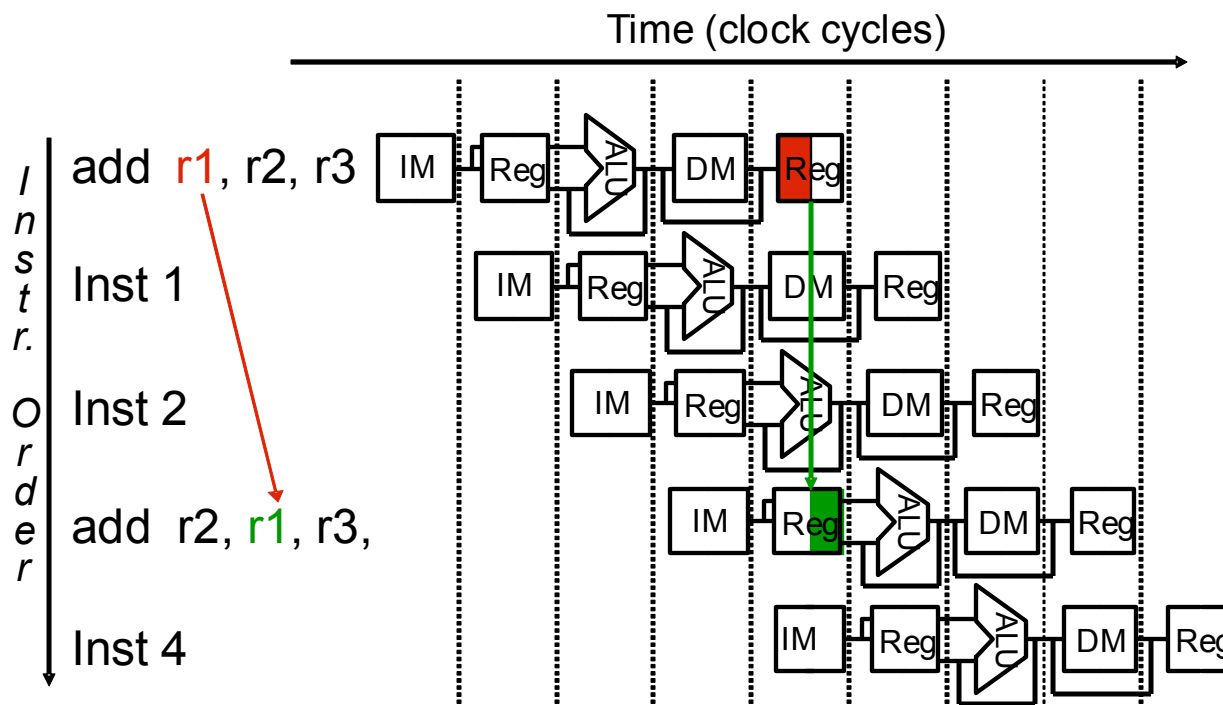
寄存器数据冒险-解决方案-2

- ALU操作数可能来自EX/MEM.ALUOut或MEM/WB.ALUOut
 - EX/MEM.ALUOut来自上一条指令
 - MEM/WB.ALUOut来自上上条指令

流水线	R类型	读存储器	写存储器	分支
IF	IR <= MemInst[PC]; PC <= PC+4			
ID	op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]];			
EX	ALUOut <= A op B EX/MEM.ALUOut MEM/WB.ALUOut	ALUOut <= A + signext(IR[15:0]) EX/MEM.ALUOut MEM/WB.ALUOut		ALUOut <= A - B PCAdd <= PC + signext(IR[15:0]) << 2
MEM		MDR <= MemData[ALUOut];	MemData[ALUOut] <= B	if (Zero) PC <= PCAdd
WB	Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		

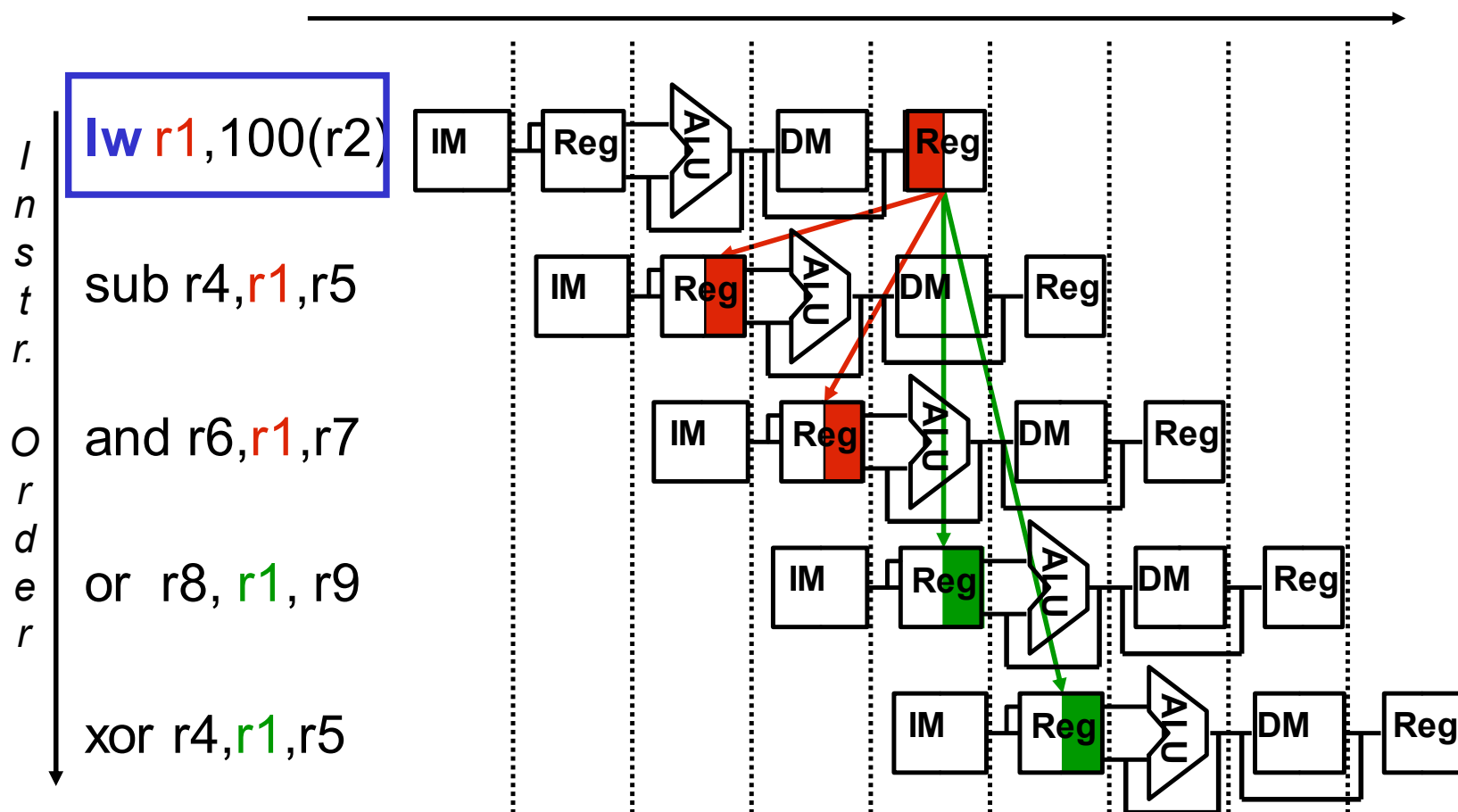
解决数据冒险的方法-2

- 流水线旁路（Forward or Bypass）
 - 需要额外的硬件电路支持
 - 在寄存器堆内，“先写后读”也可以看做一种转发



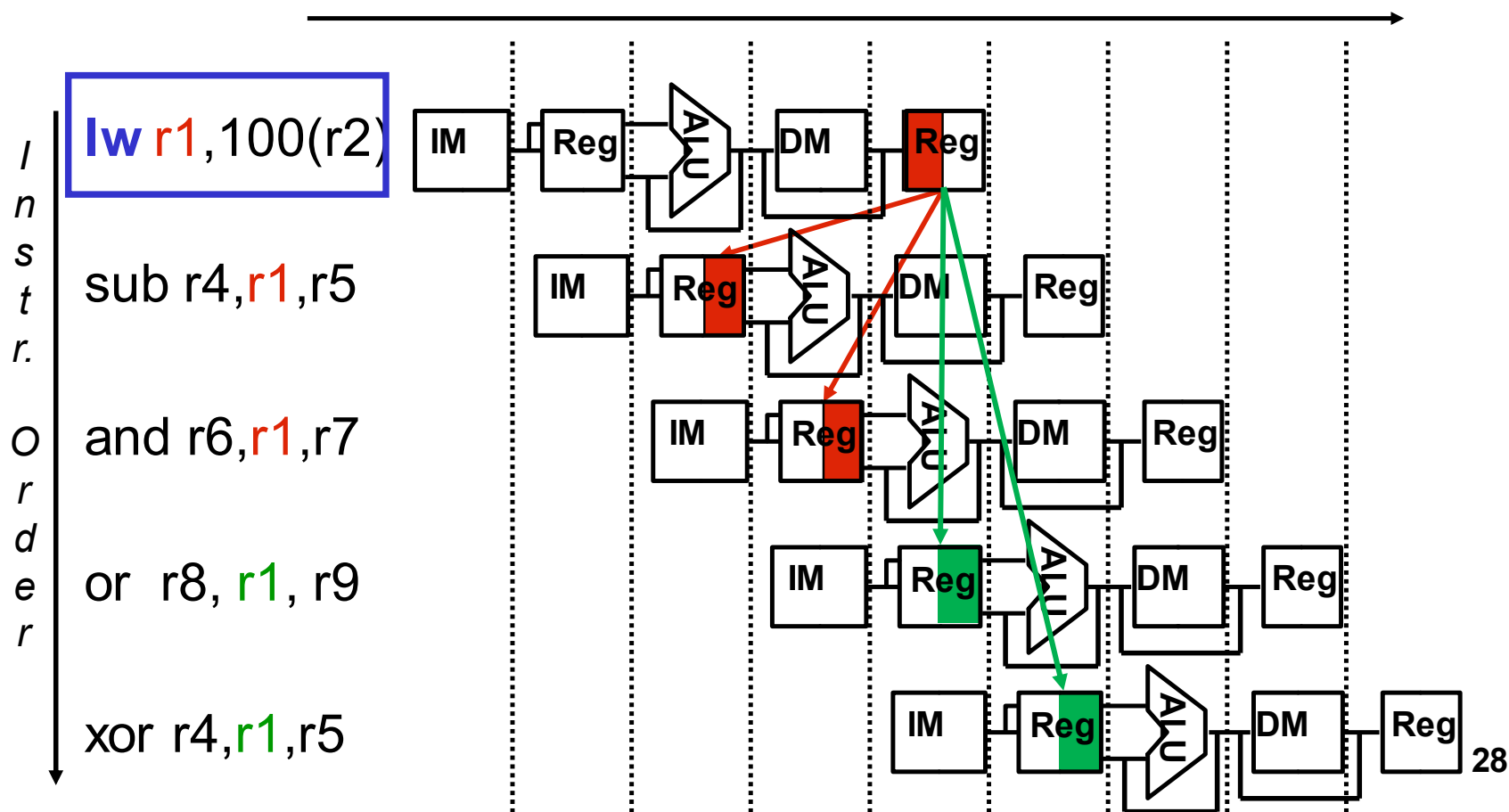
Load指令引起的冒险

- 指令在寄存器使用上的时间顺序关联引起了数据冒险
- Load-use** data hazard



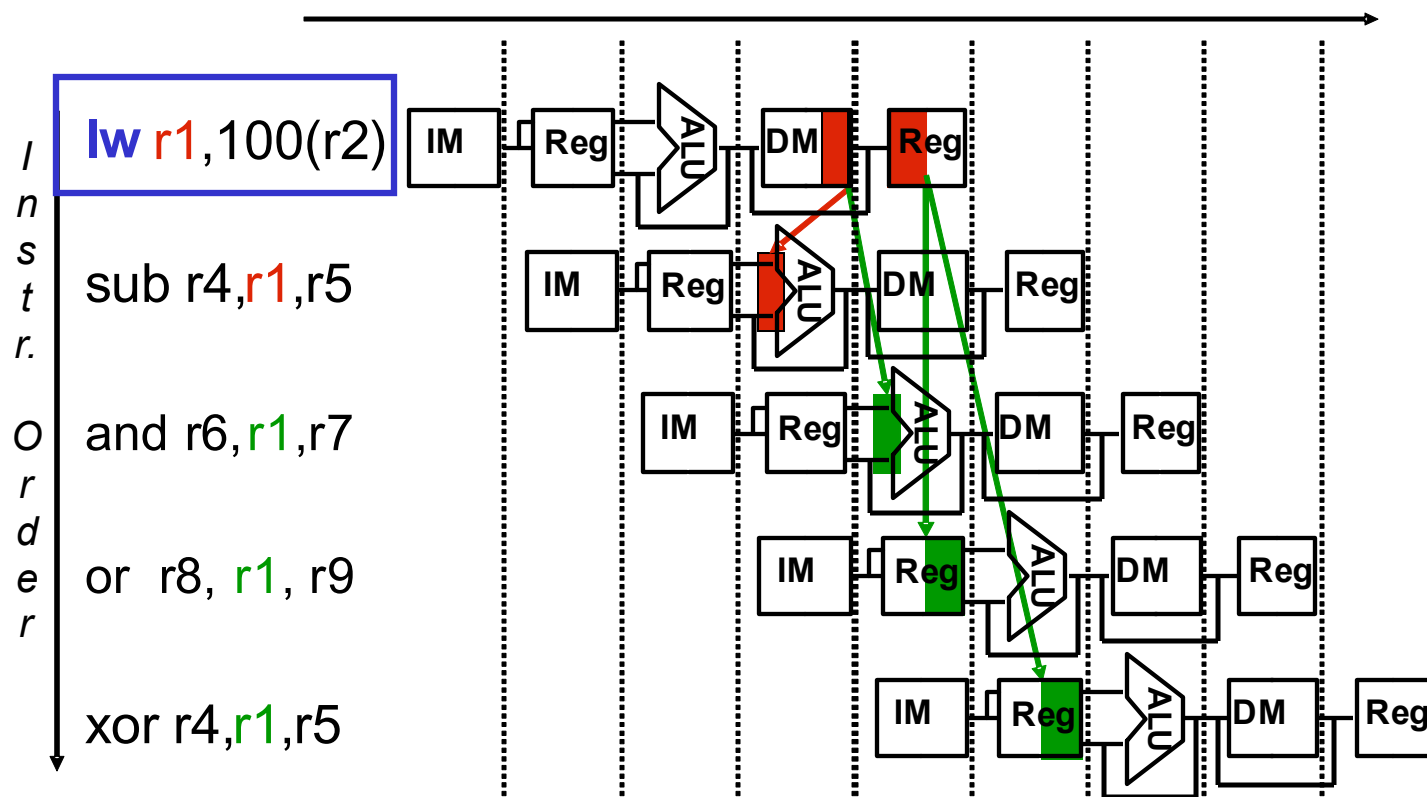
解决Load-use冒险的方法

- Load-use冒险会影响几条后续的指令？
 - 后续的第三、第四条指令受影响吗？Instructions or, xor?



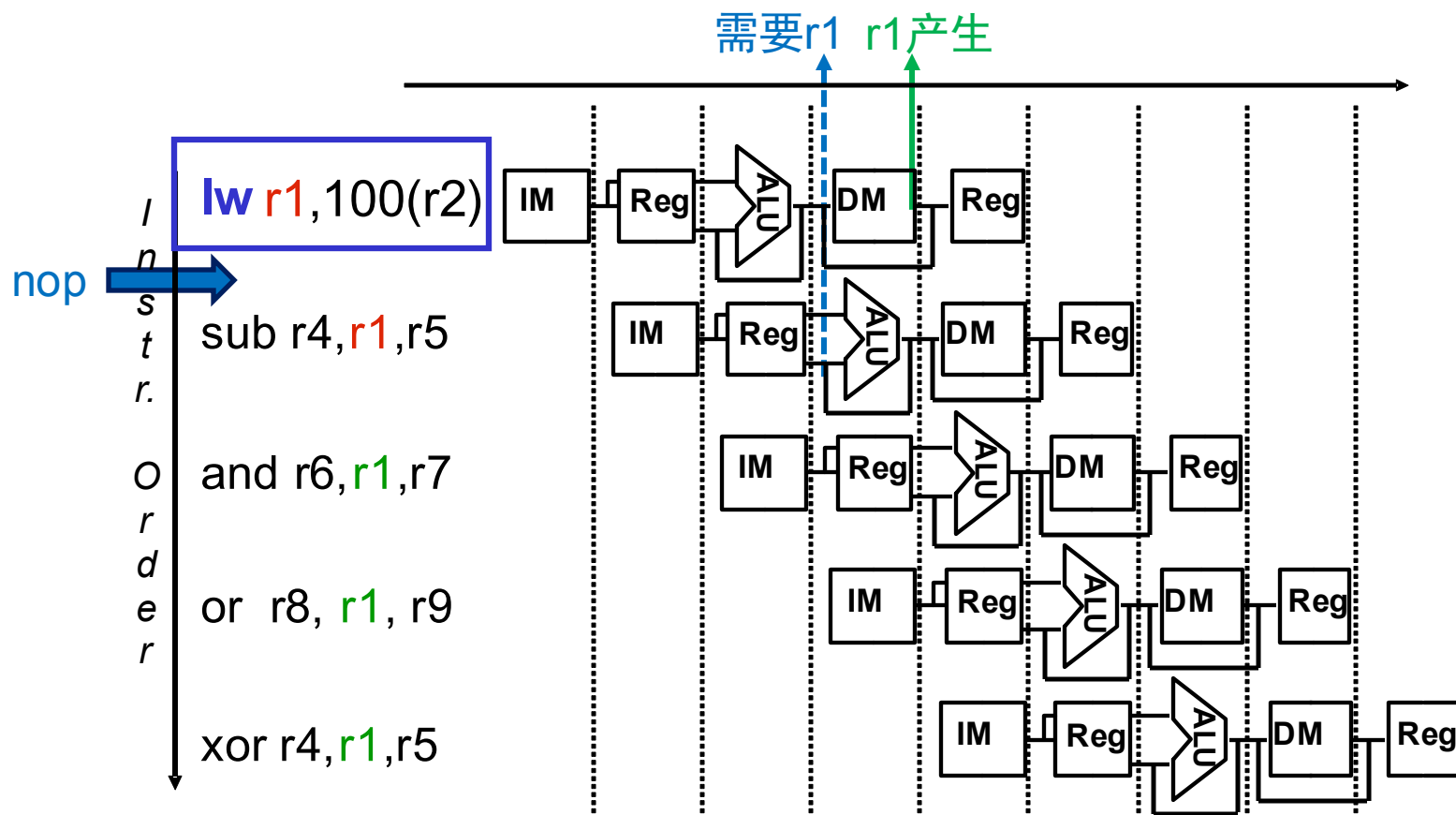
解决Load-use冒险的方法

- Load-use冒险会影响几条后续的指令？
 - 后续的第二条指令受影响吗？Instruction “and”
 - 可采用forwarding策略，不会造成一个周期的延误
 - 存储器访问后数据在MDR寄存器中，可直接转发



解决Load-use冒险的方法

- 采用forwarding后，紧接的第一条指令依赖性消除了吗？
 - 注意数据产生晚于计算需求产生时刻点
 - 所以，仍然需要stall一个周期



流水线的RTL描述

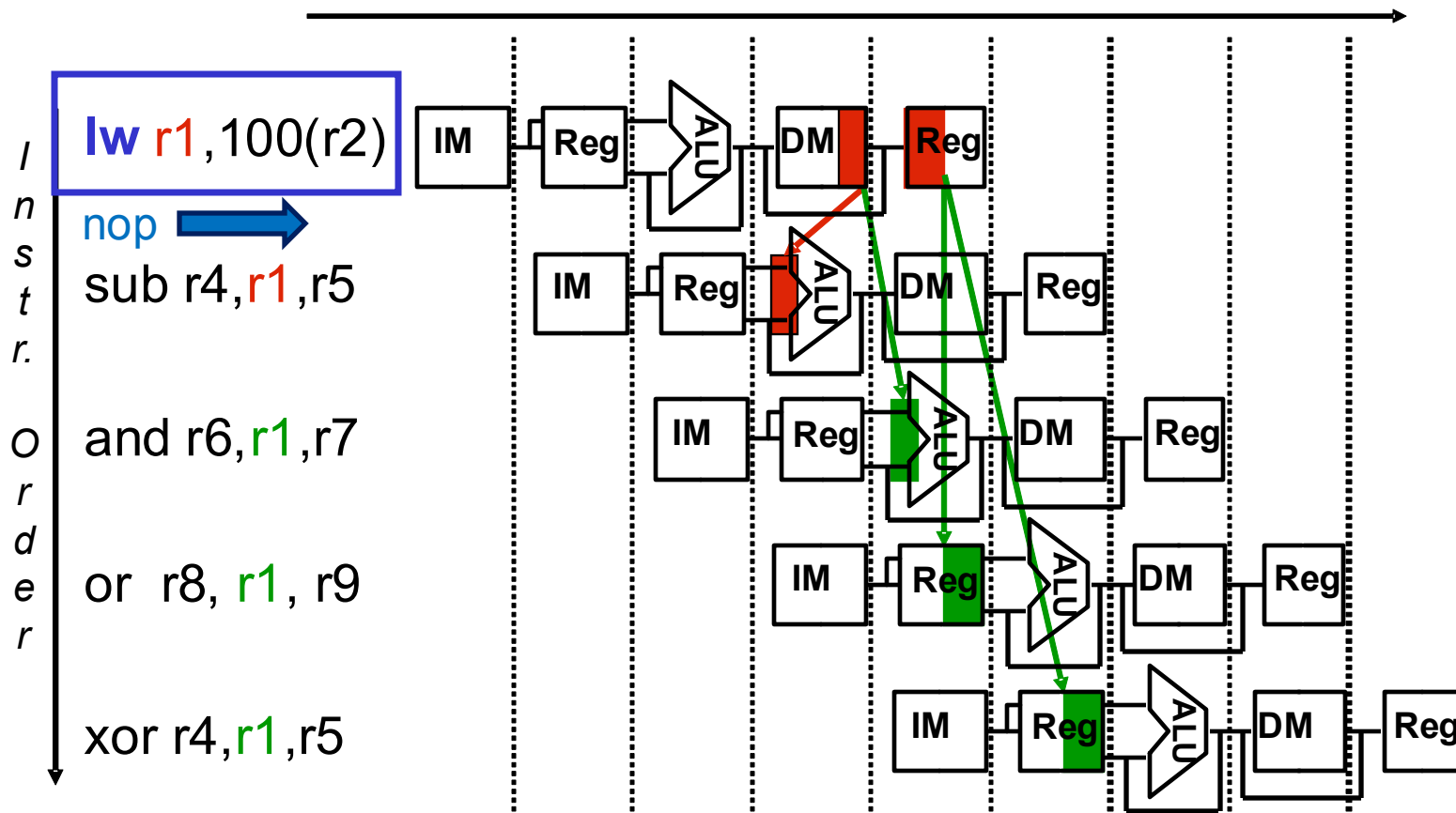
- ALU操作数可能的来源
 - RF, EX/MEM.ALUOut, MEM/WB.ALUOut, MDR等

流水线	R类型	读存储器	写存储器	分支
IF	IR <= MemInst[PC]; PC <= PC + 4			
ID	op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]];			
EX	ALUOut <= A op B EX/MEM.ALUOut MEM/WB.ALUOut MDR	ALUOut <= A + signext(IR[15:0]) EX/MEM.ALUOut MEM/WB.ALUOut MDR	ALUOut <= A - B PCAdd <= PC + signext(IR[15:0]) << 2	
MEM		MDR <= MemData[ALUOut];	MemData[ALUOut] <= B	if (Zero) PC <= PCAdd
WB	Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		31

解决Load-use冒险的方法

- 对LOAD采用forward?

- 硬件思路难以避免一个stall, 有没有其他办法避免Stall?



解决数据冒险的方法

- 能否不用Stall?

以下源程序可生成两种不同的代码，优化的代码可避免Load阻塞

$a = b + c$

$d = e - f$

Slow Code:

```
lw    $2, b
lw    $3, c
add   $1, $2, $3
sw    a, $1
lw    $5, e
lw    $6, f
sub   $4, $5, $6
sw    d, $4
```

调整后

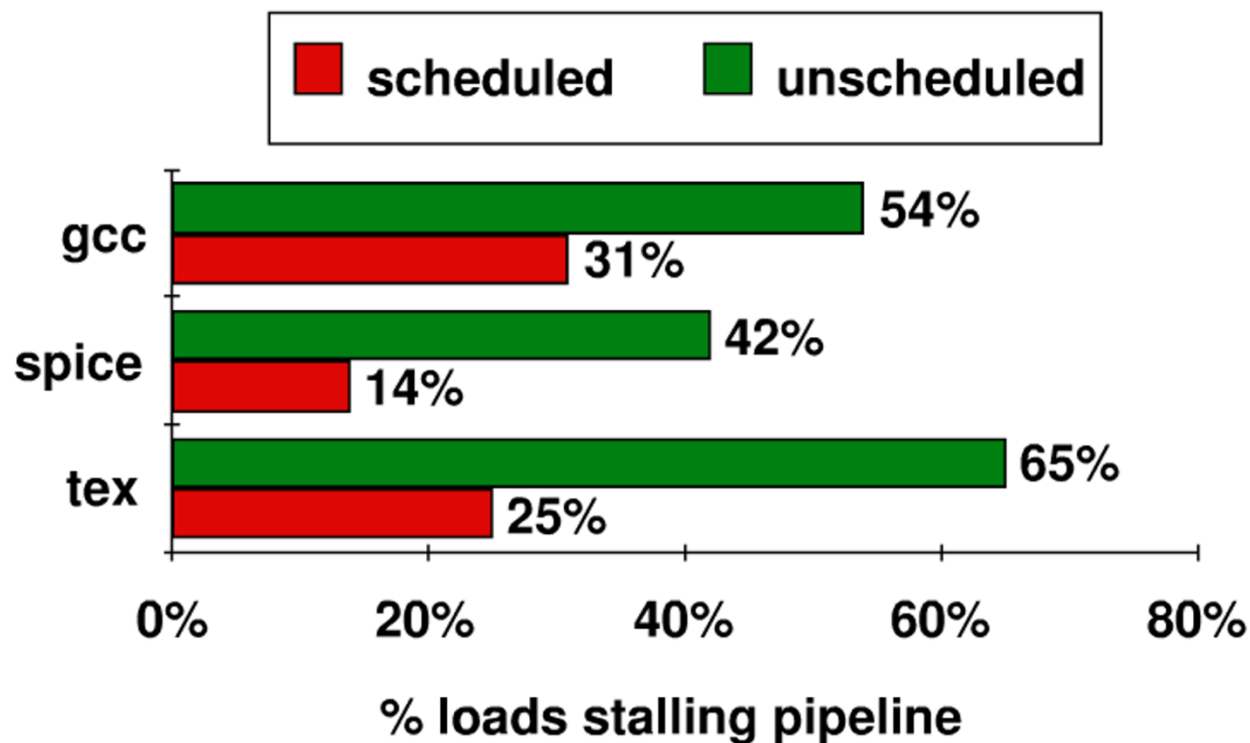
Fast Code:

```
lw    $2, b
lw    $3, c
lw    $5, e
add   $1, $2, $3
lw    $6, f
sw    a, $1
sub   $4, $5, $6
sw    d, $4
```

编译器的优化很重要!

解决数据冒险的方法

- 编译器优化以避免Stall的情况
 - 调度优化后，load阻塞现象大约降低了1/2~1/3



本节课目录

- 复习：流水线基本概念 P4
- 复习：流水线数据通路 P6
- **流水线的冒险**
 - 结构冒险 P10
 - 数据冒险 P15
 - **控制冒险 P38**
- 数据通路的实现（考虑冒险） P57

流水线的冒险

- **结构冒险（Structural Hazard or Resource Conflict）：资源矛盾！**
 - 两条指令在同一周期需要使用同样的硬件
 - 解决方法：增加硬件资源；调整指令占用资源的时间
- **数据关联冒险（Data Hazard）：操作数是否已经更新？**
 - 指令依赖于流水线中运行的其他指令结果
 - 解决方法：Stall；Forwarding；编译器优化
- **控制冒险（Control Hazard）：下一条指令的PC是多少？**
 - 取指令依赖于流水线中指令的结果

流水线：控制冒险

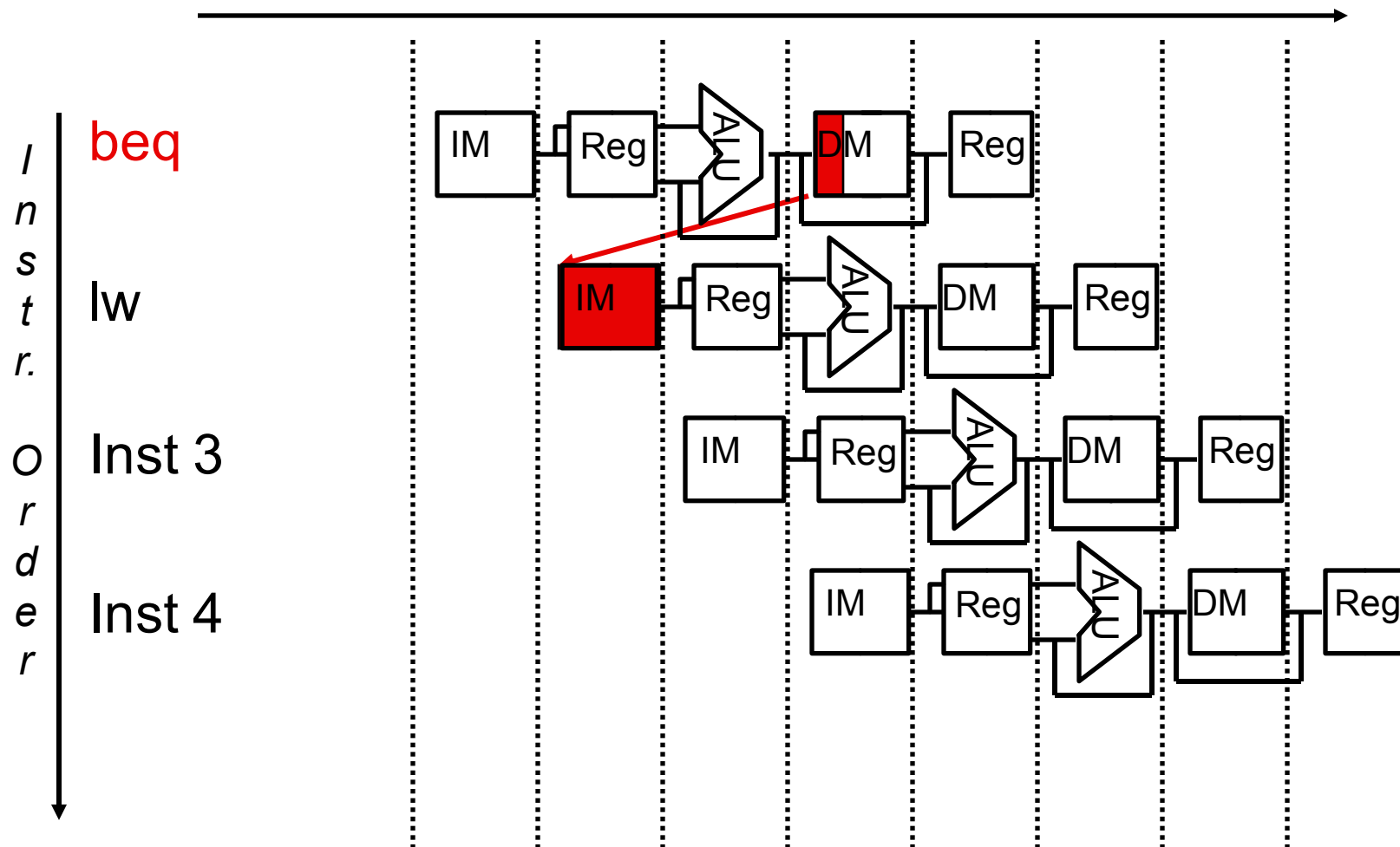
• PC存在冲突

- 分支指令在MEM阶段更新PC；J型指令呢（课后思考）？
- 本表中的其他指令在IF阶段更新PC

流水线	R类型	读存储器	写存储器	分支
IF	IR <= MemInst[PC]; PC <= PC + 4			
ID	op <= IR[31:26]; A<=Reg[IR[25:21]]; B<=Reg[IR[20:16]];			
EX	ALUOut <= A op B	ALUOut <= A + signext(IR[15:0])		ALUOut <= A - B PCAdd <= PC+signext(IR[15:0]) << 2
MEM		MDR <= MemData[ALUOut];	MemData[ALUOut] <= B	if (Zero) PC <= PCAdd
WB	Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		39

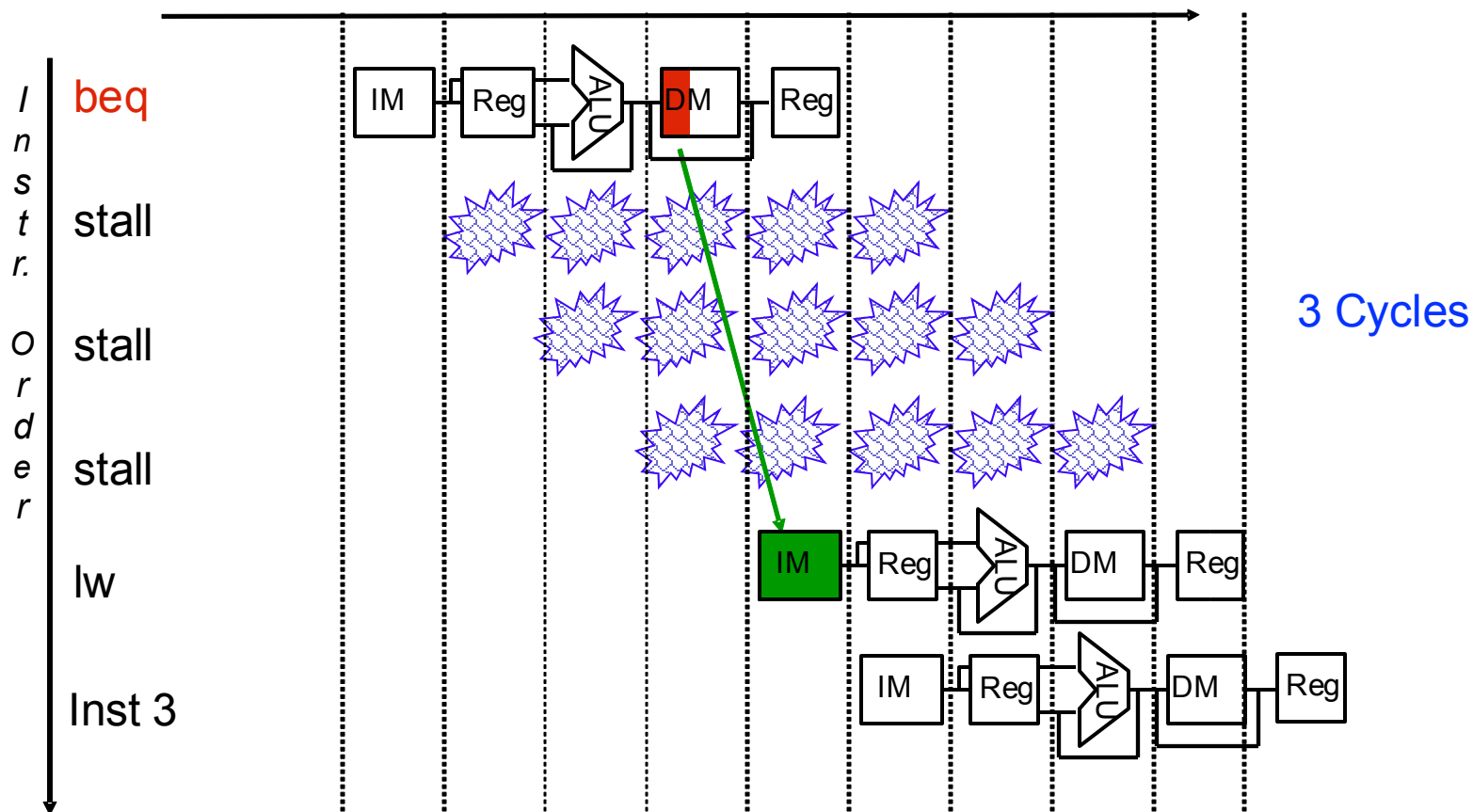
流水线冒险：控制冒险

- 分支指令引起的控制冒险



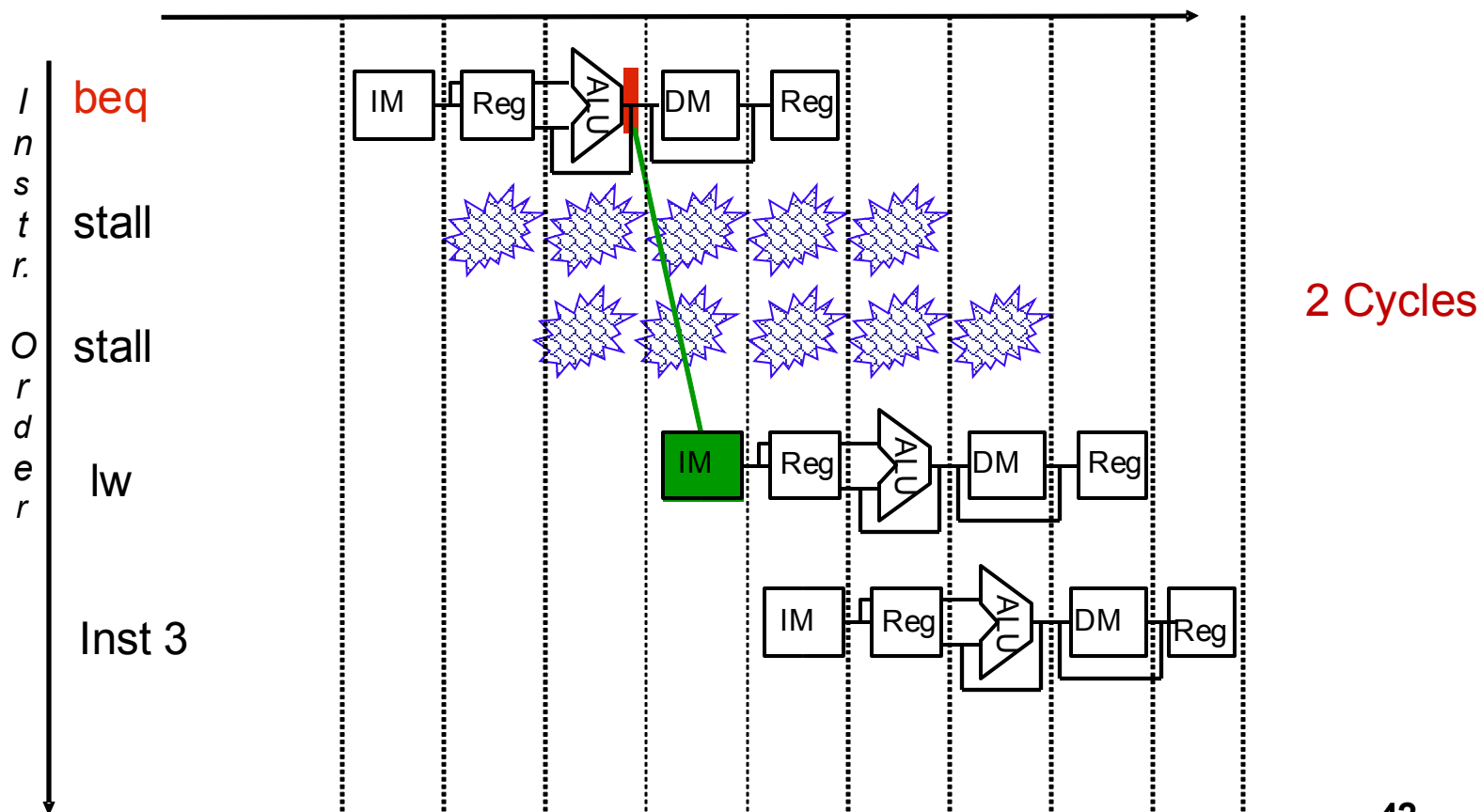
控制冒险解决方法：Stall

- 可采用Stall的方式，插入等待周期
– 影响吞吐率



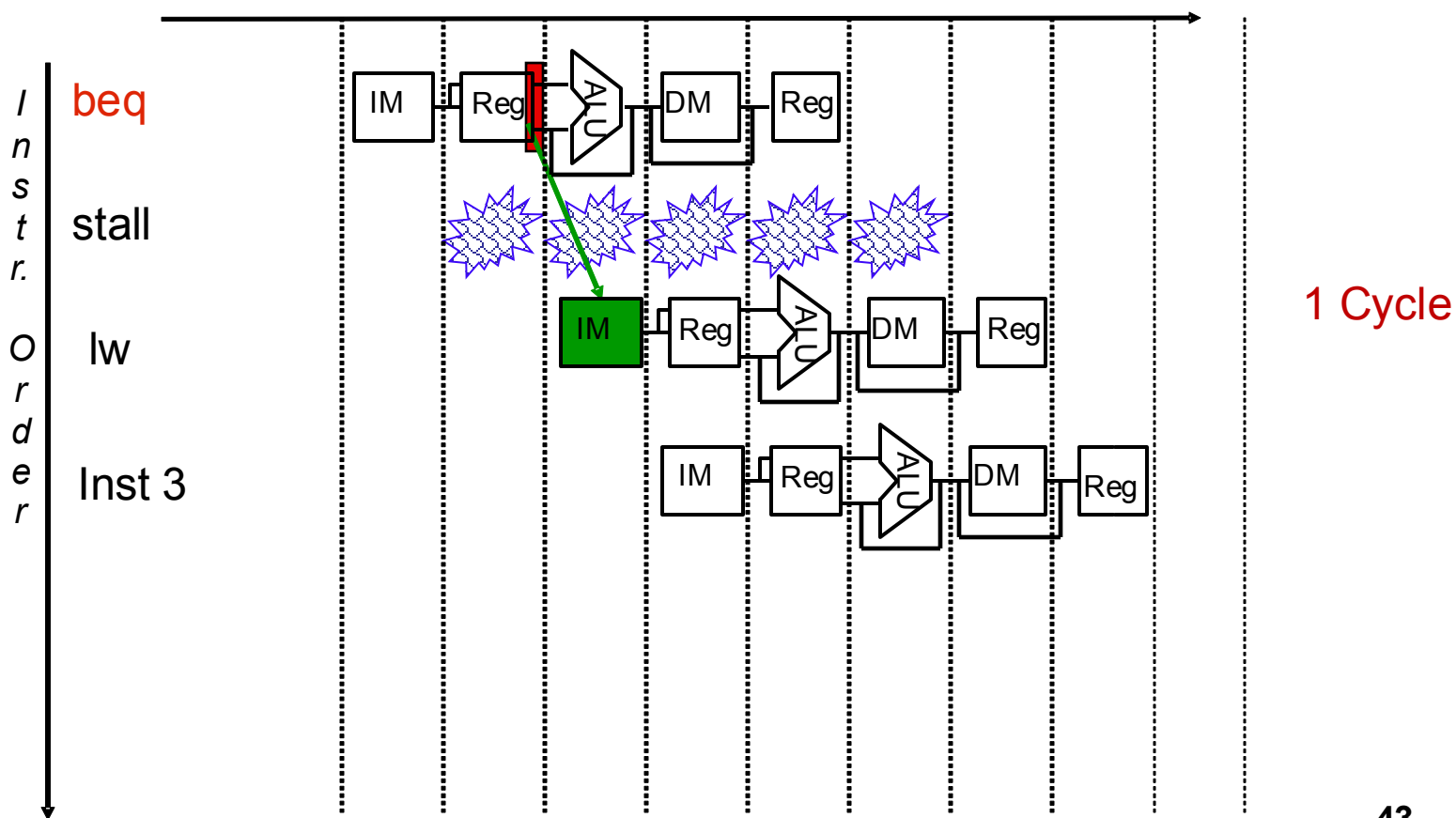
控制冒险解决方法：Forwarding

- 通过Forwarding方式提前给出分支判断 (EX→IF)



控制冒险解决方法：Forwarding

- 通过Forwarding方式提前给出分支判断（ID→IF）
 - 在ID阶段加入分支判断？只阻塞一个周期！



控制冒险解决方法：Forwarding

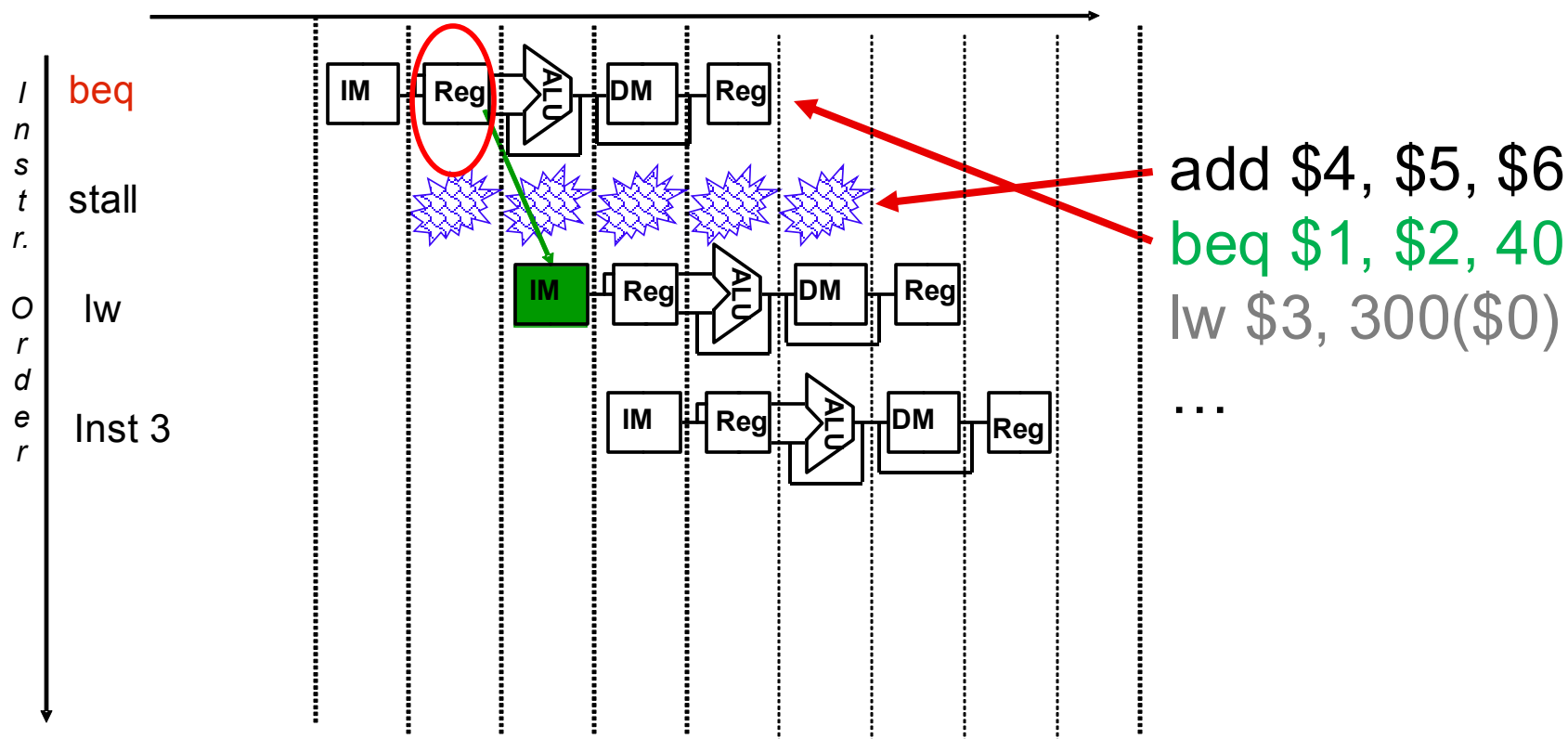
- 在ID阶段加入分支判断：只阻塞一个周期！
 - 可有问题？

流水线	R类型	读存储器	写存储器	分支
IF	IR <= MemInst[PC]; PC <= PC + 4			
ID	op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]]; 分支: If(A == B) PC <= PC + signext(IR[15:0]) << 2			
EX	ALUOut <= A op B EX/MEM.ALUOut, MEM/WB.ALUOut, MDR	ALUOut <= A + signext(IR[15:0]) EX/MEM.ALUOut, MEM/WB.ALUOut, MDR		
MEM		MDR <= MemData[ALUOut];	MemData[ALUOut] <= B	
WB	Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		

控制冒险解决方法：延迟槽技术

- 阻塞的一次Stall可以利用起来吗？

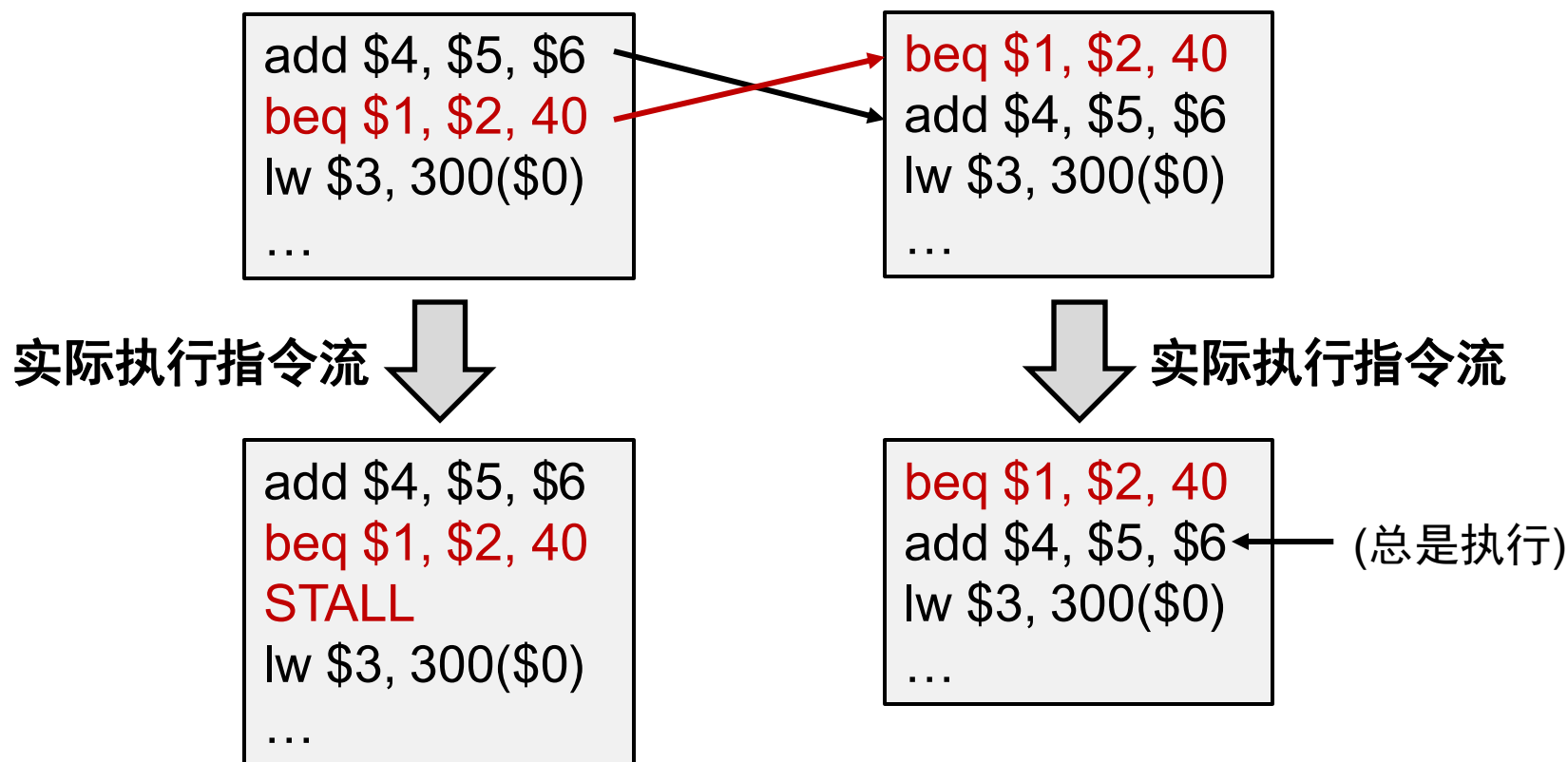
- Beq指令优化后也造成一次Stall，即总等待一个周期才知道是否跳转
- 这一个等待周期只能空着吗？可否用来执行一些必定要执行的指令？



控制冒险解决方法：延迟槽技术

- 延迟槽技术出发点

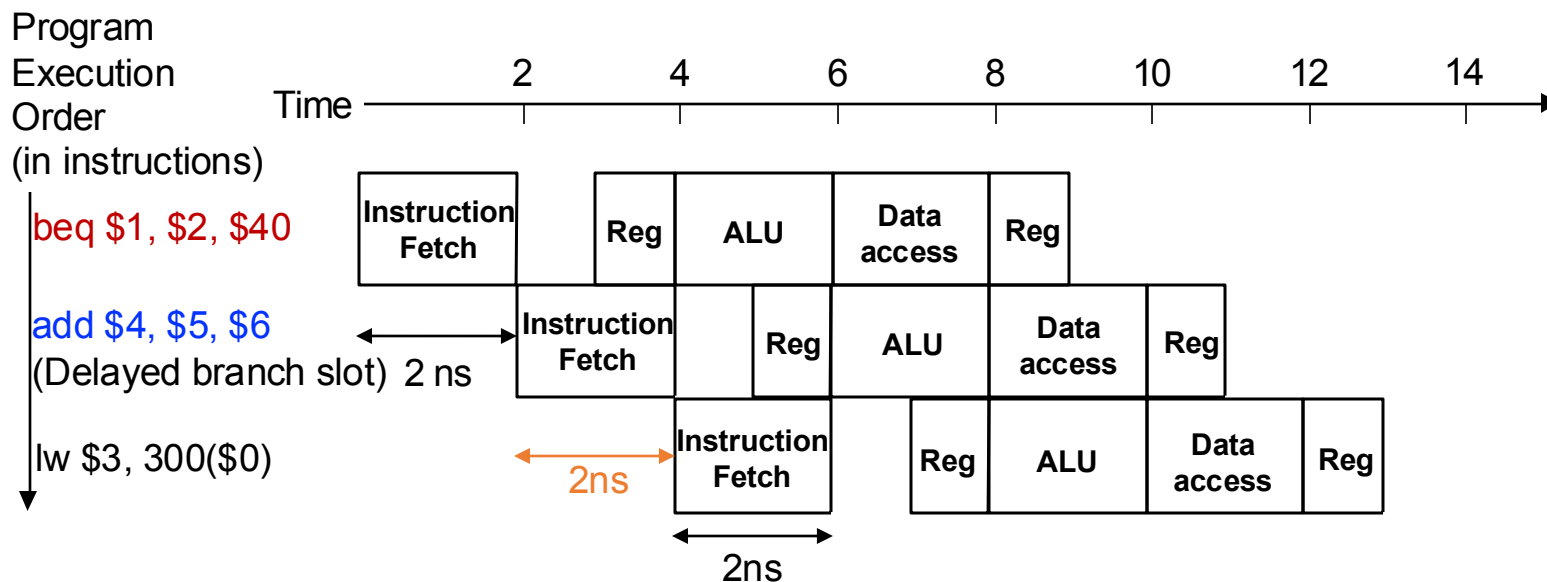
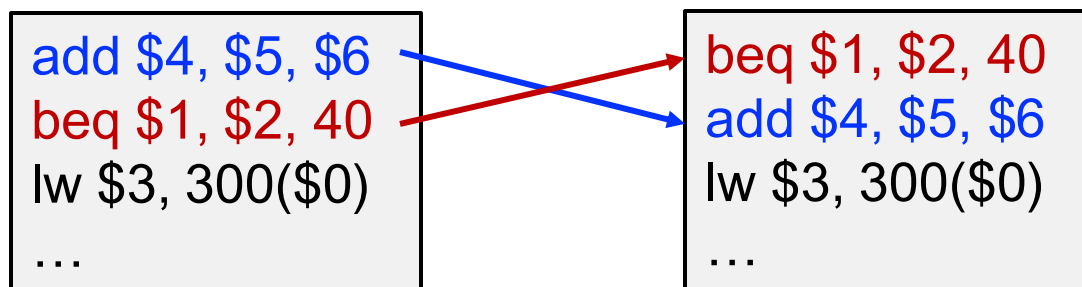
- beq后的一次延迟等待周期可以用来执行一些必定要执行的指令



控制冒险解决方法：延迟槽技术

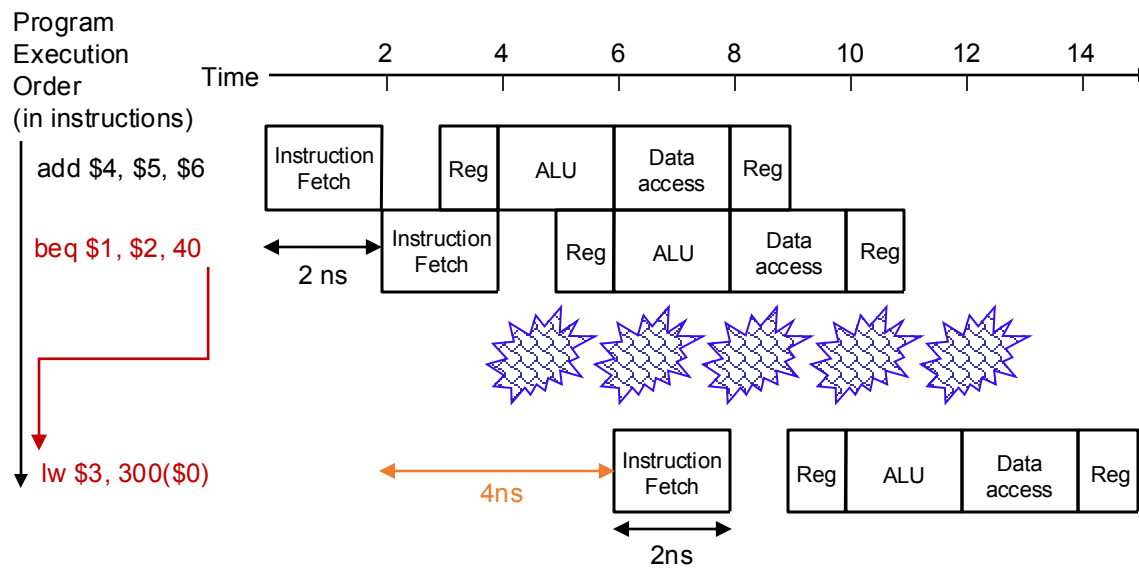
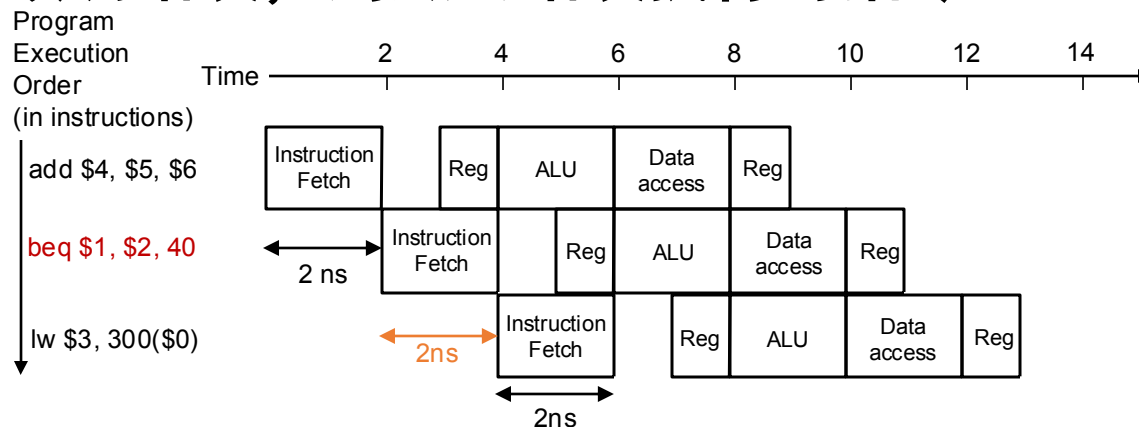
- 延迟槽技术出发点

- beq后的一次延迟等待周期可以用来执行一些必定要执行的指令



控制冒险解决方法：预测

- 方式一：静态预测（总是预测分支不执行或执行）
 - 如果预测错误，则撤销错误执行的指令



控制冒险解决方法：预测

- 每次预测失误会导致不必要的流水线重置
 - 必须保证错误执行的指令不会改变最终的结果，并且在正确的分支地址处重启流水线
 - 如何减小预测失误的概率？
 - 静态 → 动态
- 方式二：动态分支预测（Dynamic branch prediction）
 - 利用历史信息进行动态预测
 - 保存并利用近期记录的行为预测未来的行为

动态分支预测

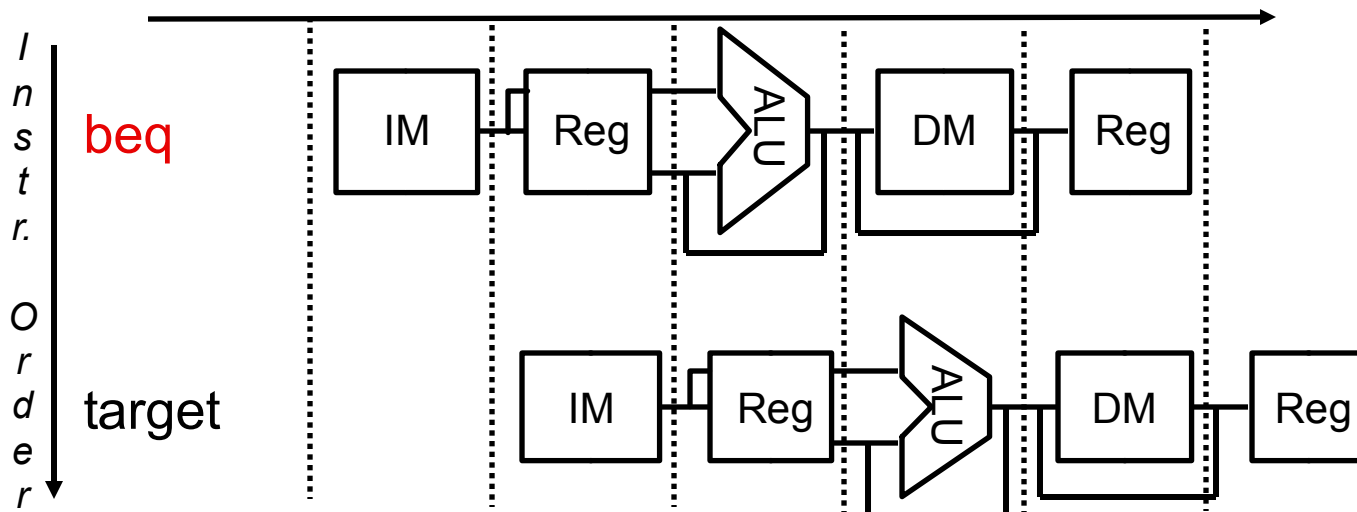
- 动态分支预测的实现

- 增加额外硬件进行动态预测

- 在哪个阶段预测？

- 在IF阶段进行分支预测缓存

- Branch History Table (BHT) → Branch Target Buffer (BTB)
 - 可以用PC（或者PC的低位地址）为索引，记录过去是否跳转

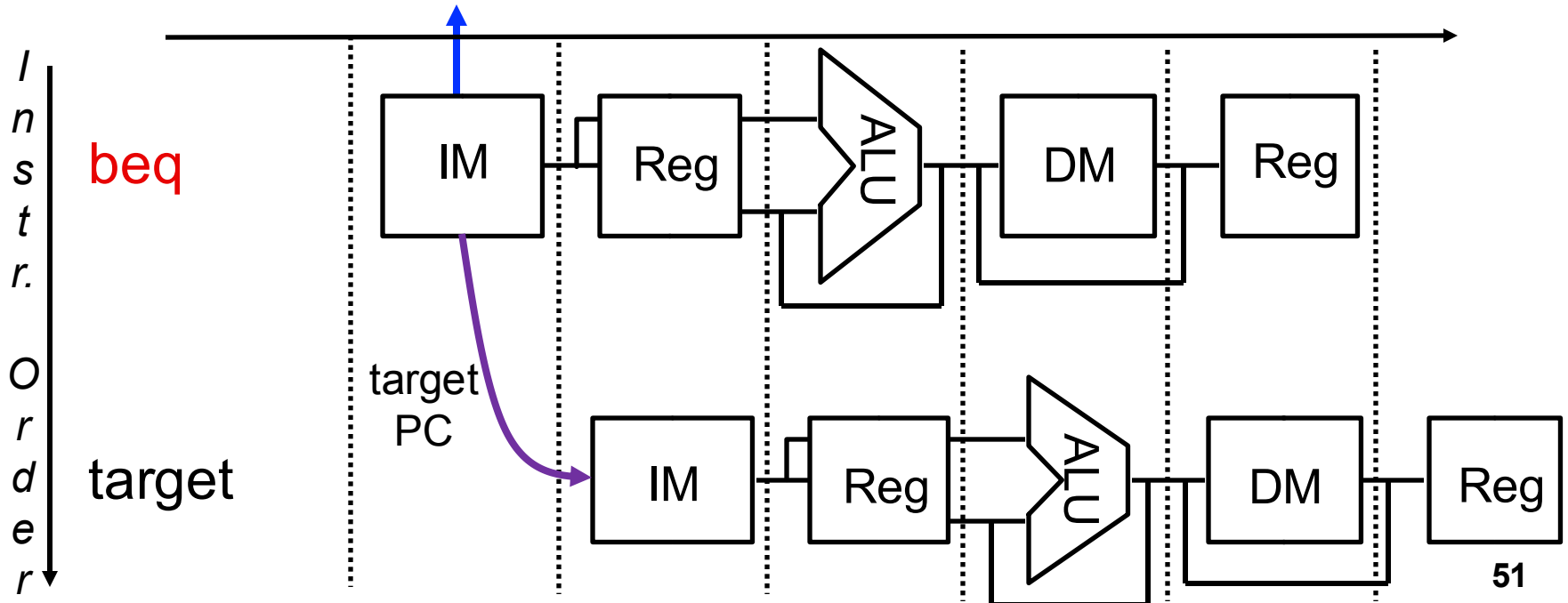


动态分支预测

• 动态分支预测流程：Clock 1

– beq指令的IF阶段

- 查询指令地址是否在BHT和BTB中
- 若存在，**根据历史记录判断是否跳转**：若跳转，取出目标地址作为下一条指令的IF地址
- 若不存在，建立新的条目

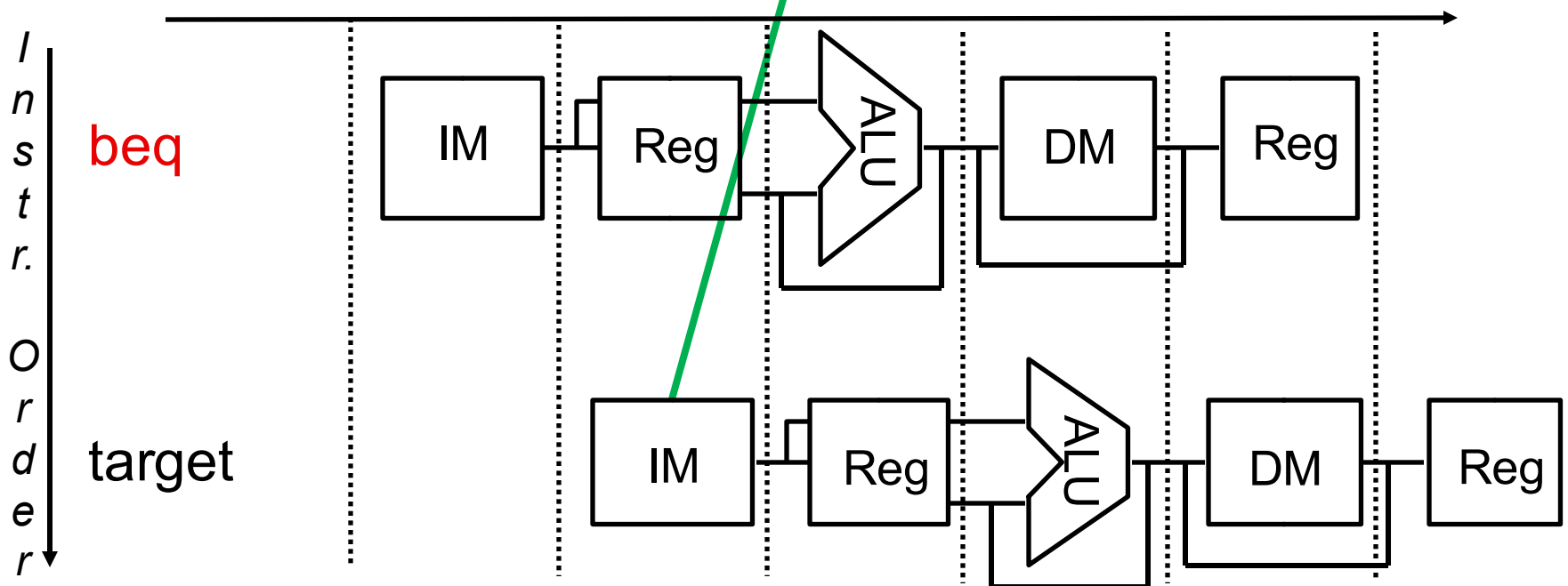


动态分支预测

- 动态分支预测流程：Clock 2

- beq指令的ID阶段

- 下一条指令的IF段根据预测目标地址取出指令



动态分支预测

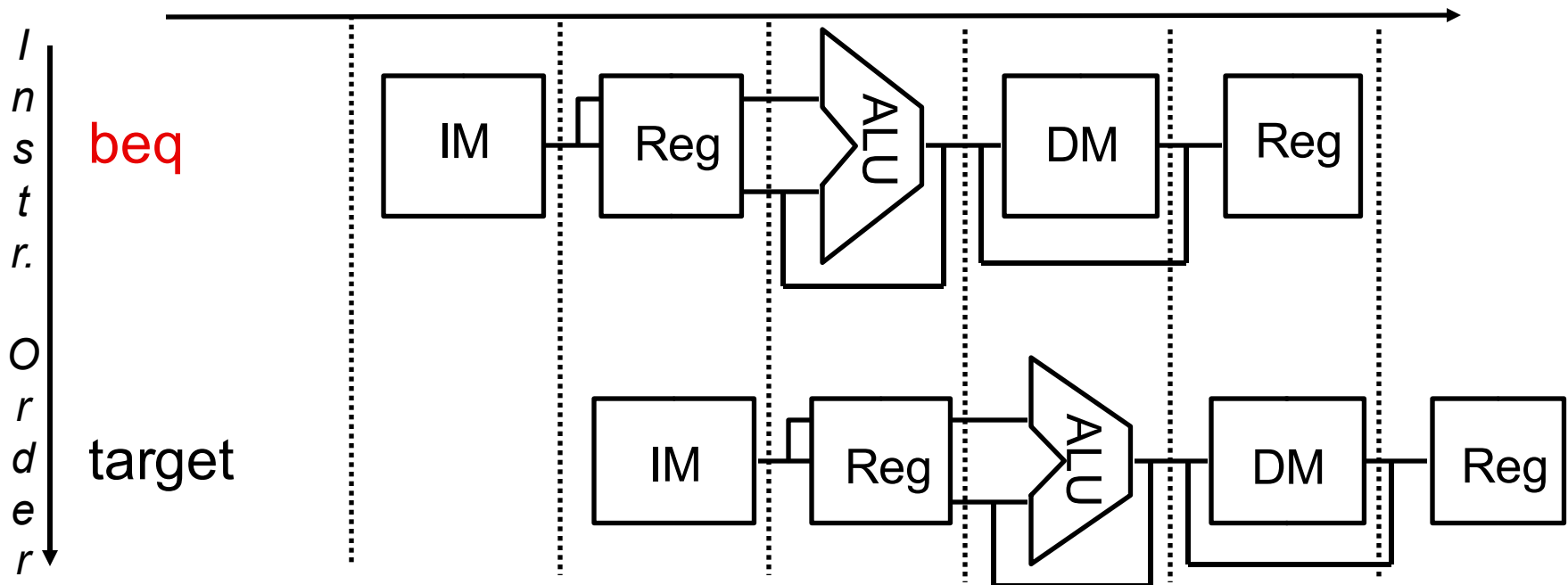
- 动态分支预测的实现

BHT: Branch history table
BTB: Branch Target Buffer

- 由第一个分支指令建立BHT和BTB:

- ID译码发现是beq, 存下

- 这条指令的地址、最终是否跳转（1/0）、如跳转的话目标地址



本节课目录

- 复习：流水线基本概念 P4
- 复习：流水线数据通路 P6
- 流水线的冒险 P10
- **数据通路的实现（考虑冒险）**
 - 结构冒险 P57
 - 数据冒险 P86
 - 控制冒险

流水线实现与优化

- 最大化时钟速度：提高理论最大吞吐率
- 提高有效执行指令比例：提高实际吞吐率
- 最小化所需要的硬件资源：成本最小化

三方面互相影响

流水线实现与优化

- 问：k级流水线一定可以使吞吐率提高k倍吗？
 - 假定每级组合逻辑电路的传播延时都是相同的
 - 不考虑寄存器的建立保持时间要求
- 问题的关键：指令间依赖
 - 流水线是否被阻塞？
 - 流水线是否执行了不必要的指令？

一个例子

After compiling C to assembly MIPS, I have:

...

lw \$2,8(\$fp)

nop

sw \$2,16(\$fp)

li \$2,5 # 0x5

lw \$3,16(\$fp)

nop

beq \$3,\$2,\$L4

nop

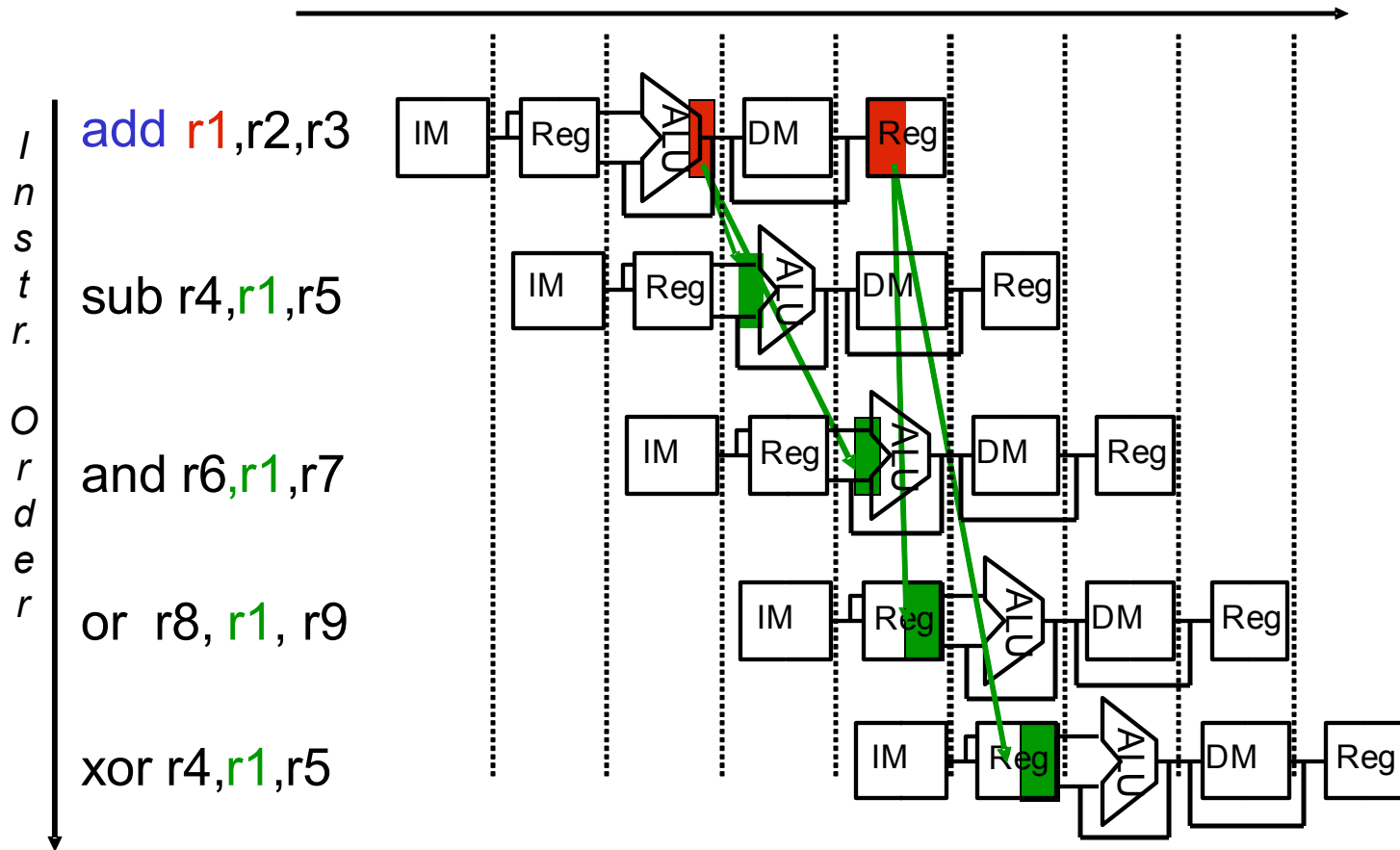
li \$2,10 # 0xa

...

- Why are there nop code lines?
- What is the meaning of nop?
- Does it help anything in pipeline hazards?
- What if I remove all the nops?

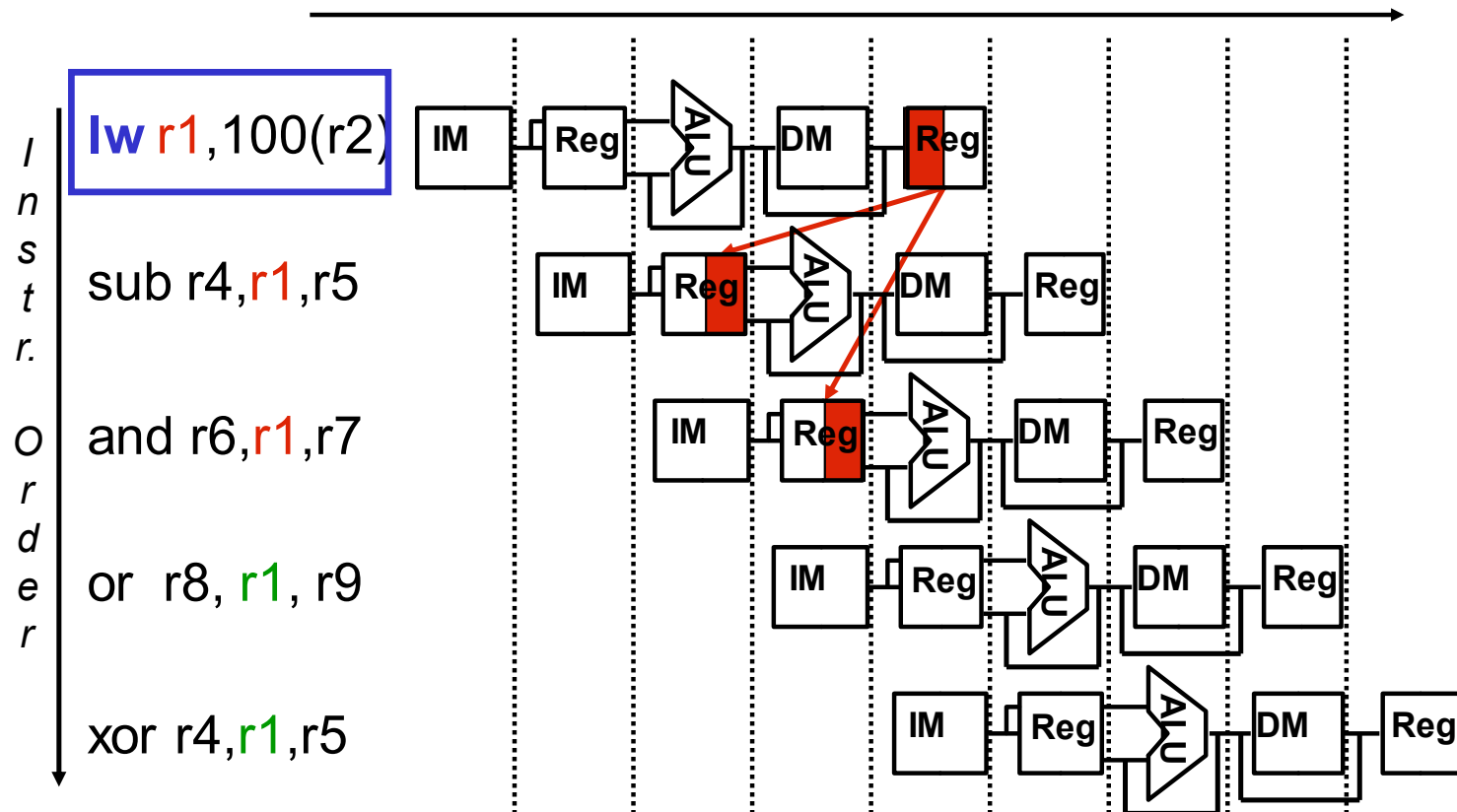
回顾：数据冒险A

- Read after write (RAW)



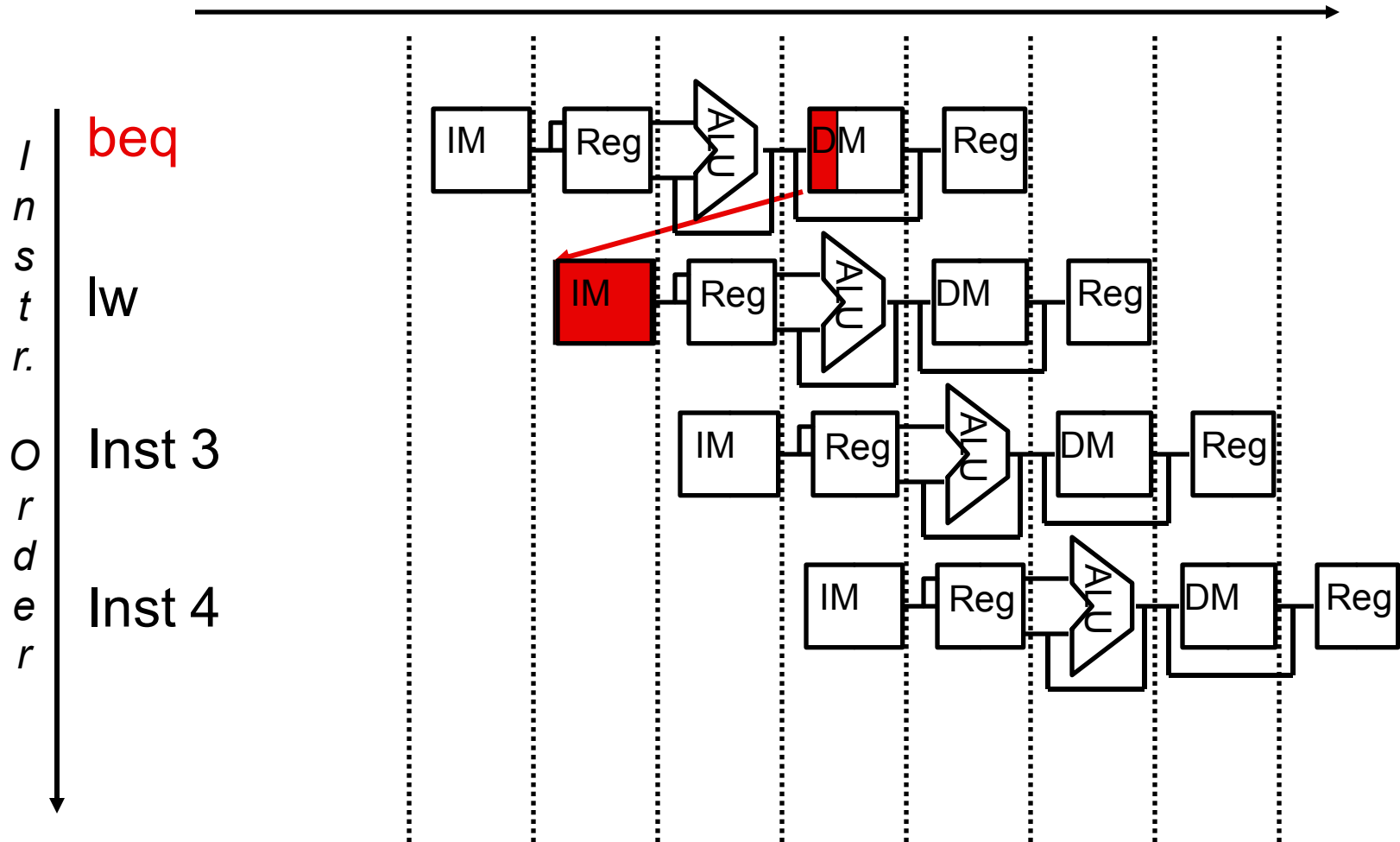
回顾：数据冒险B

- Use before load



回顾：控制冒险

- 分支指令导致的冒险

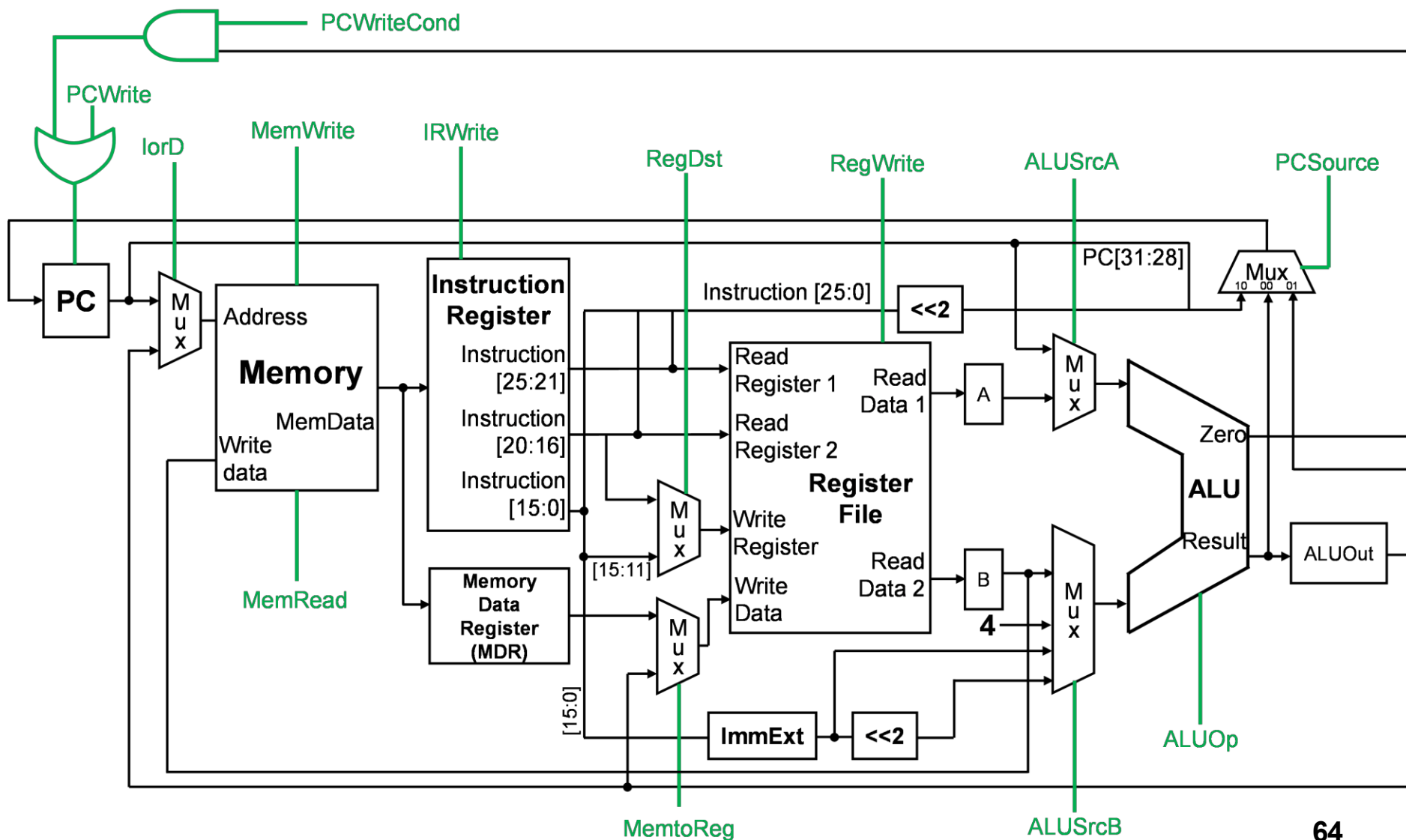


回顾：流水线RTL描述

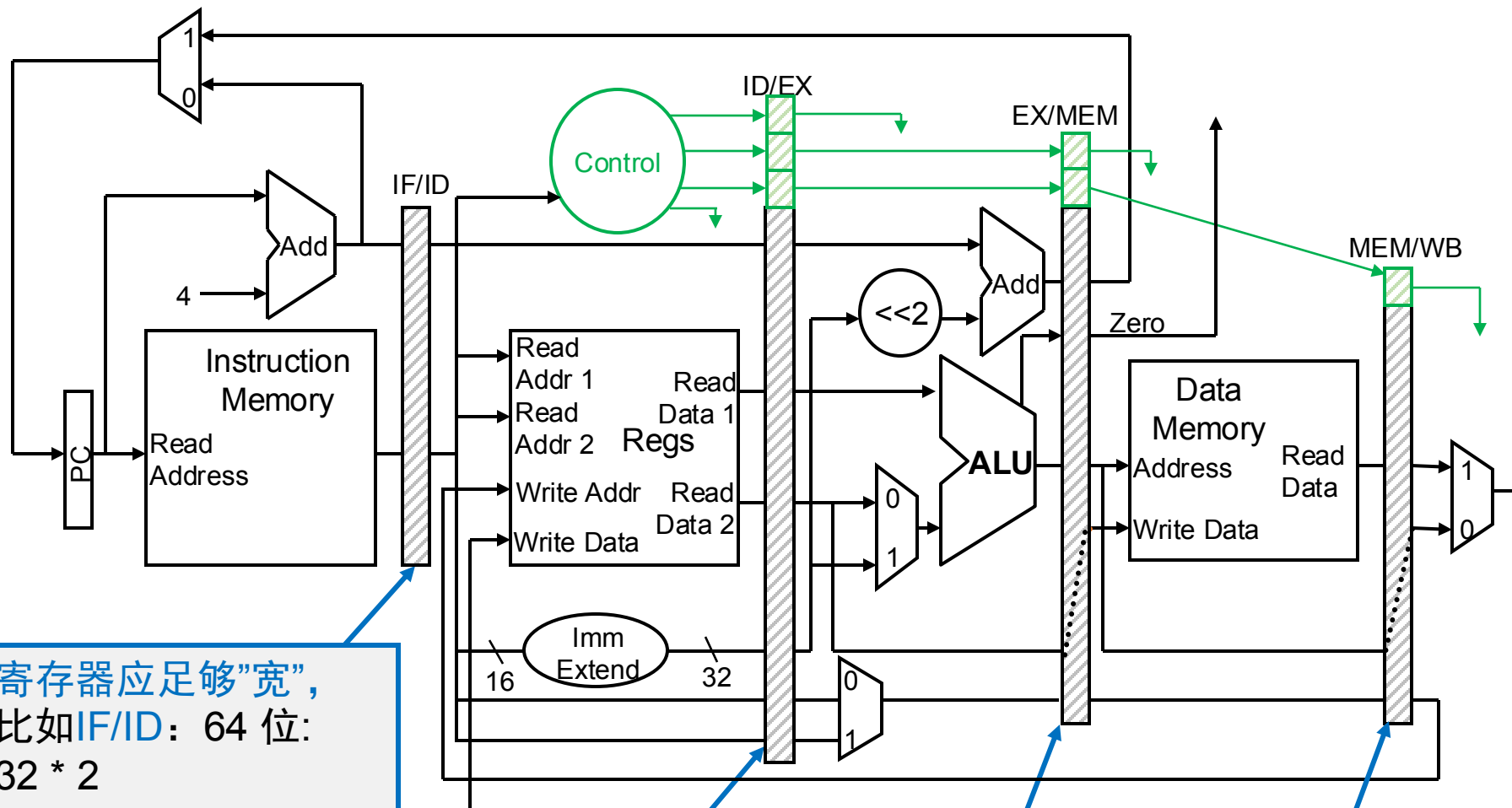
- Forwarding
- 提前分支判断

流水线	R类型	读存储器	写存储器	分支
IF	IR <= MemInst[PC]; PC <= PC + 4			
ID	op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]]; 分支: If(A == B) PC <= PC + signext(IR[15:0]) << 2			Where are A and B from?
EX	ALUOut <= A op B EX/MEM.ALUOut MEM/WB.ALUOut MDR	ALUOut <= A + signext(IR[15:0]) EX/MEM.ALUOut MEM/WB.ALUOut MDR		
MEM		MDR <= MemData[ALUOut];	MemData[ALUOut] <= B	
WB	Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		

参照：多周期的控制信号



简化的流水线数据通路



寄存器应足够“宽”，
比如IF/ID：64 位：
 $32 * 2$

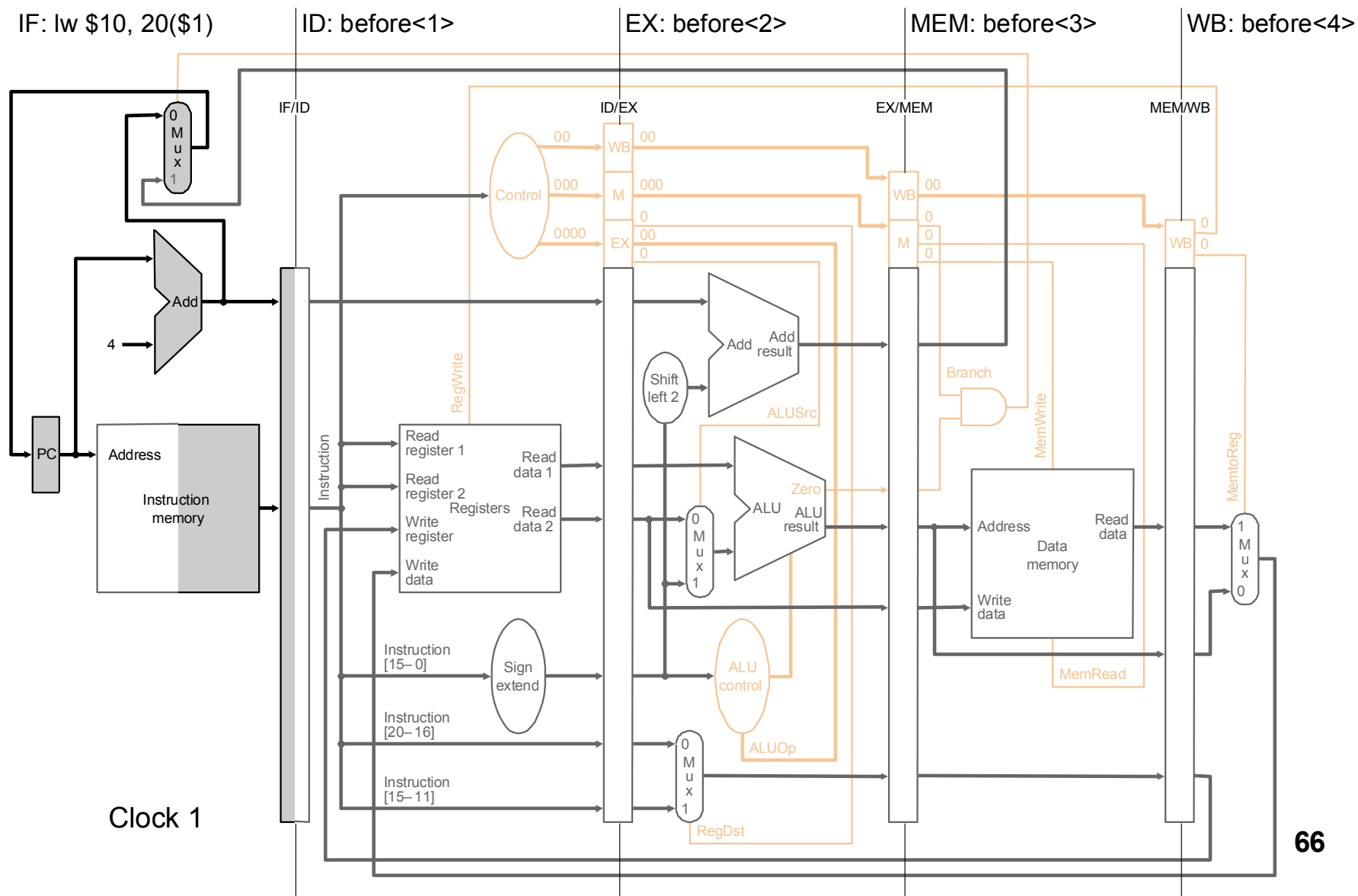
“+”表示：
后续讲解中
会逐渐增加

ID/EX：138 位+：
 $32 * 4 + 5 * 2$

EX/MEM：102 位+：
 $32 * 3 + \text{“zero”} + 5$

MEM/WB：69 位+：
 $32 * 2 + 5$ 65

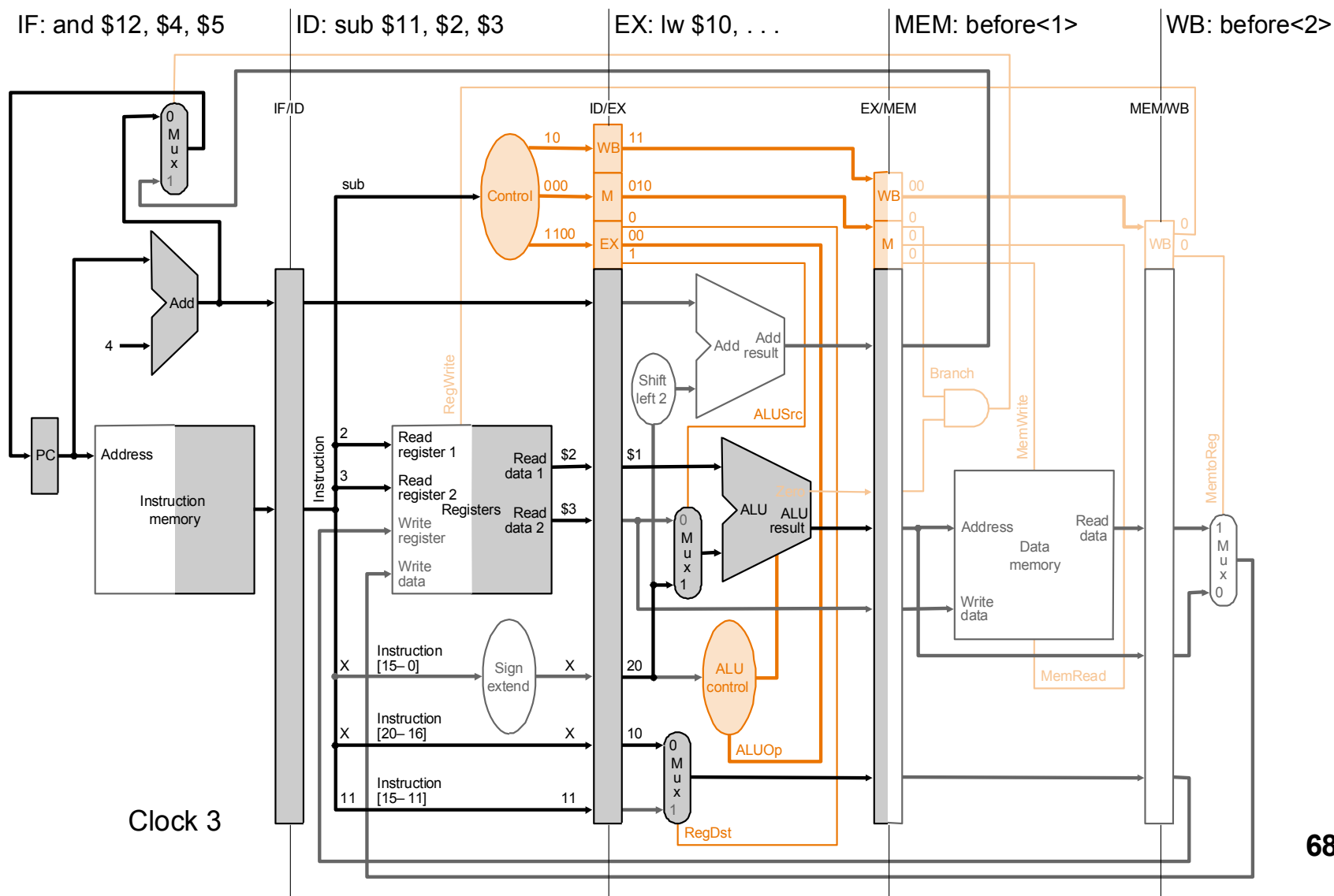
流水线运行示例



Government	Percentage
Current government	85%
Previous government	15%



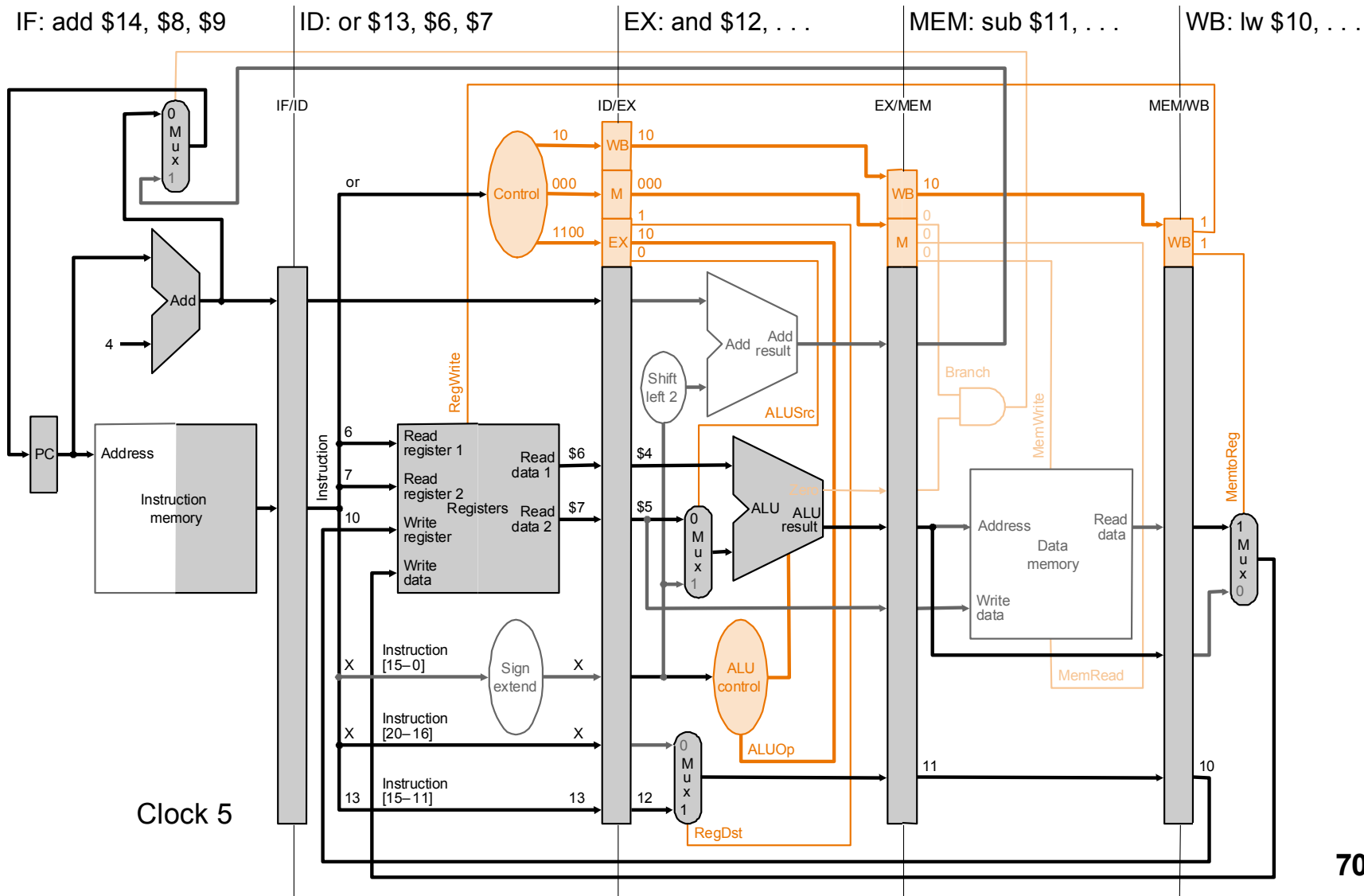
流水线运行示例



Response	Percentage
Yes	45%
No	55%



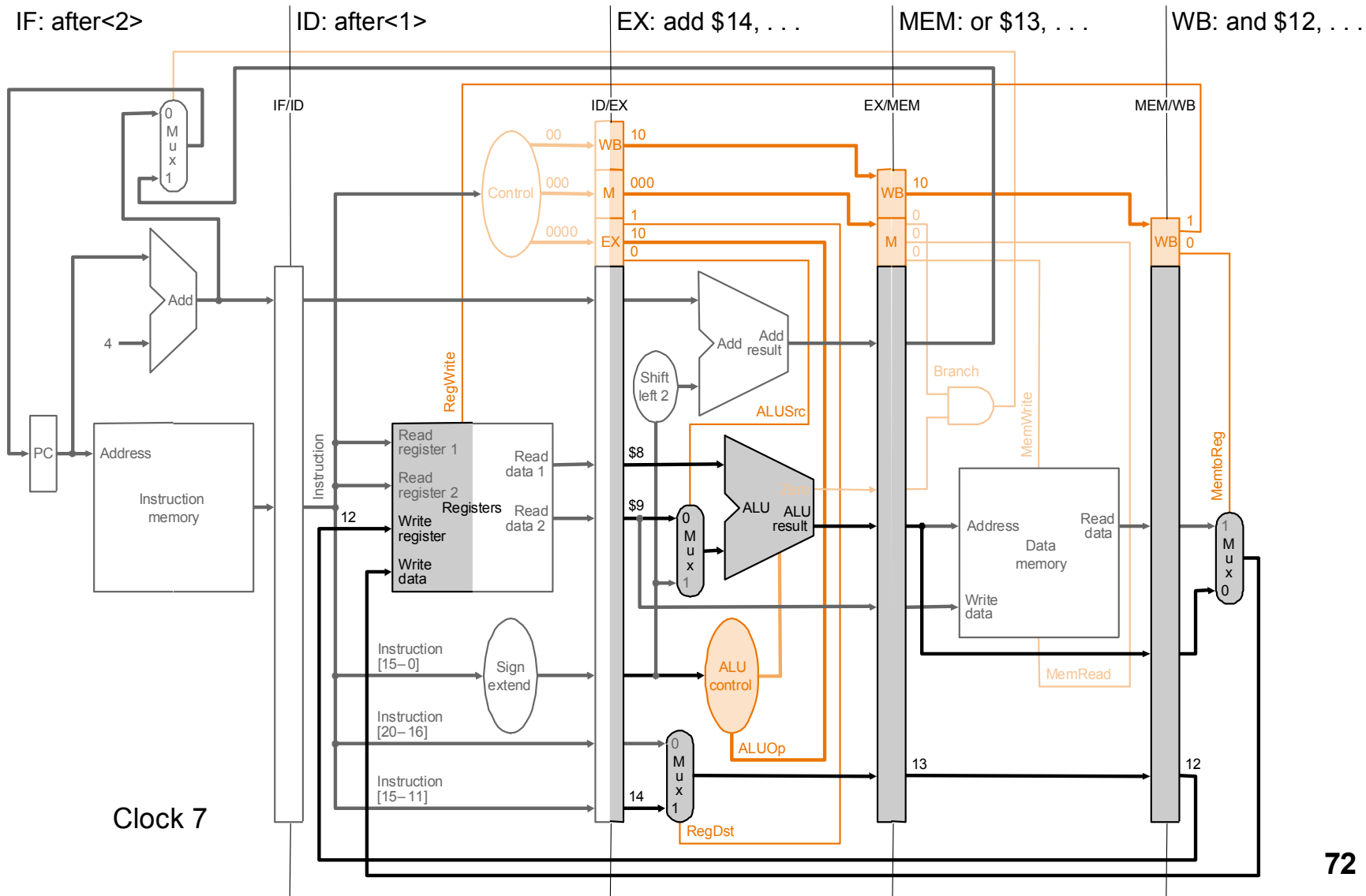
流水线运行示例



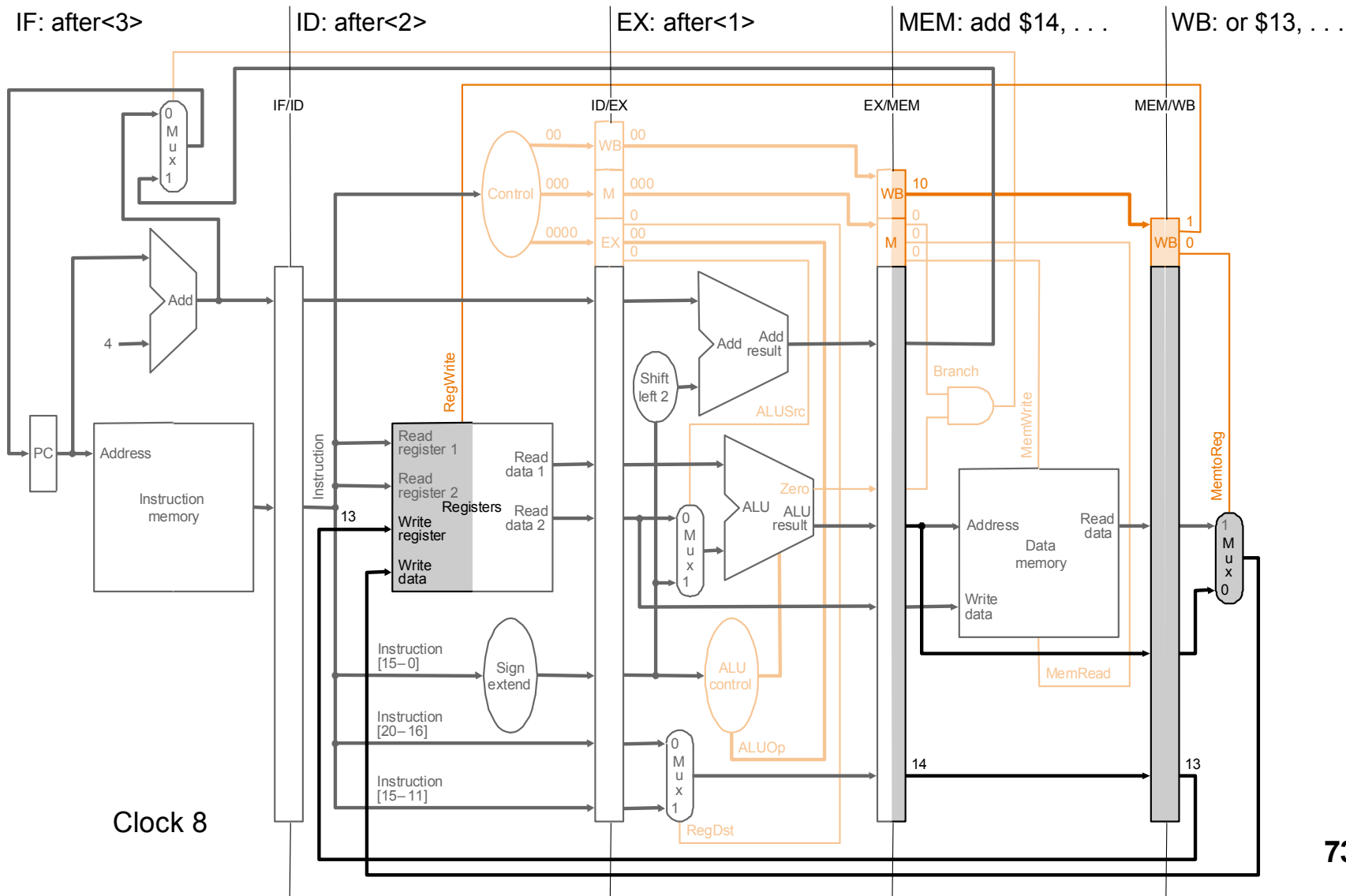
Response	Percentage
Yes, the current government is responsible	75%
No, the crisis is not the government's fault	25%



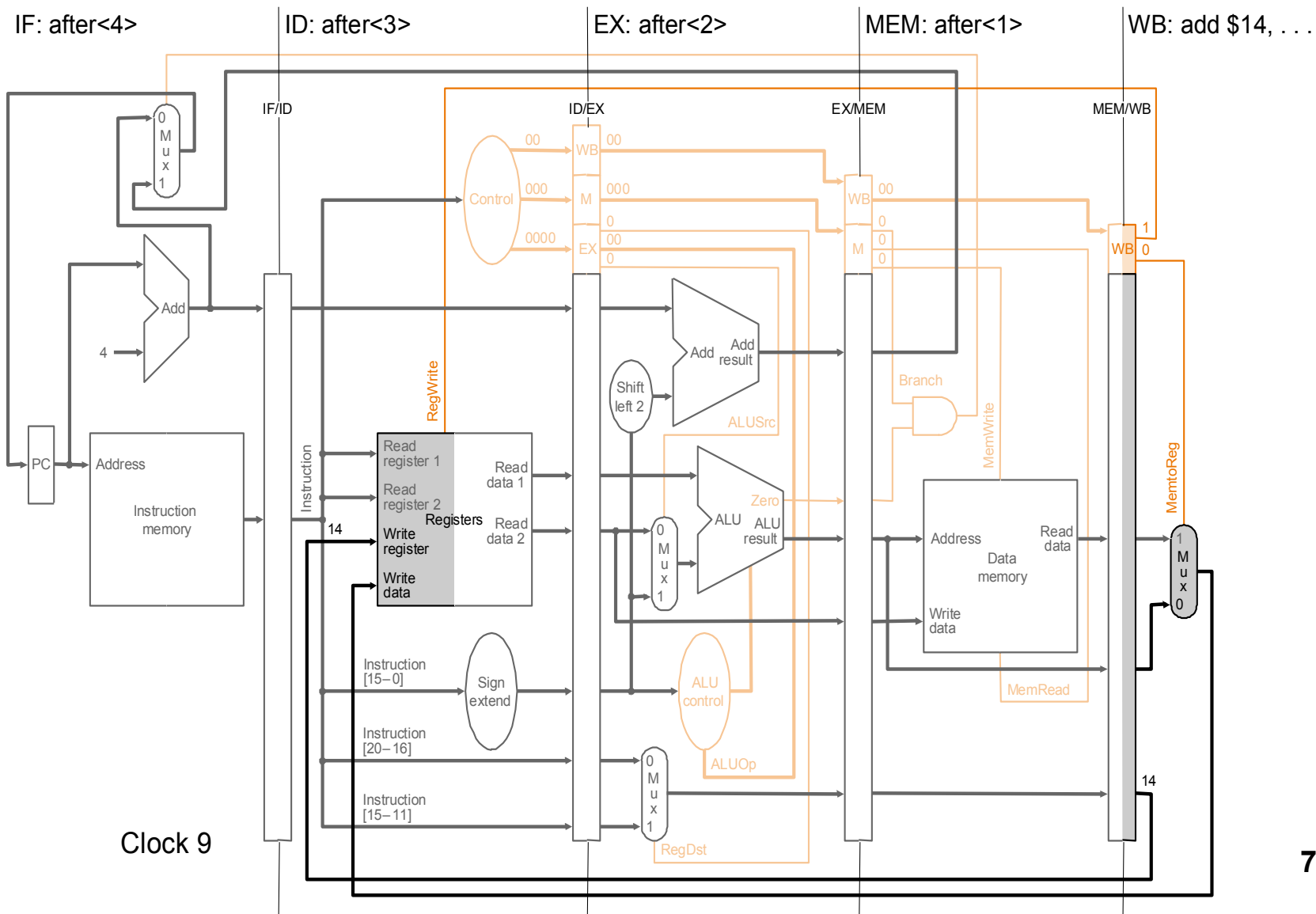
流水线运行示例



流水线运行示例

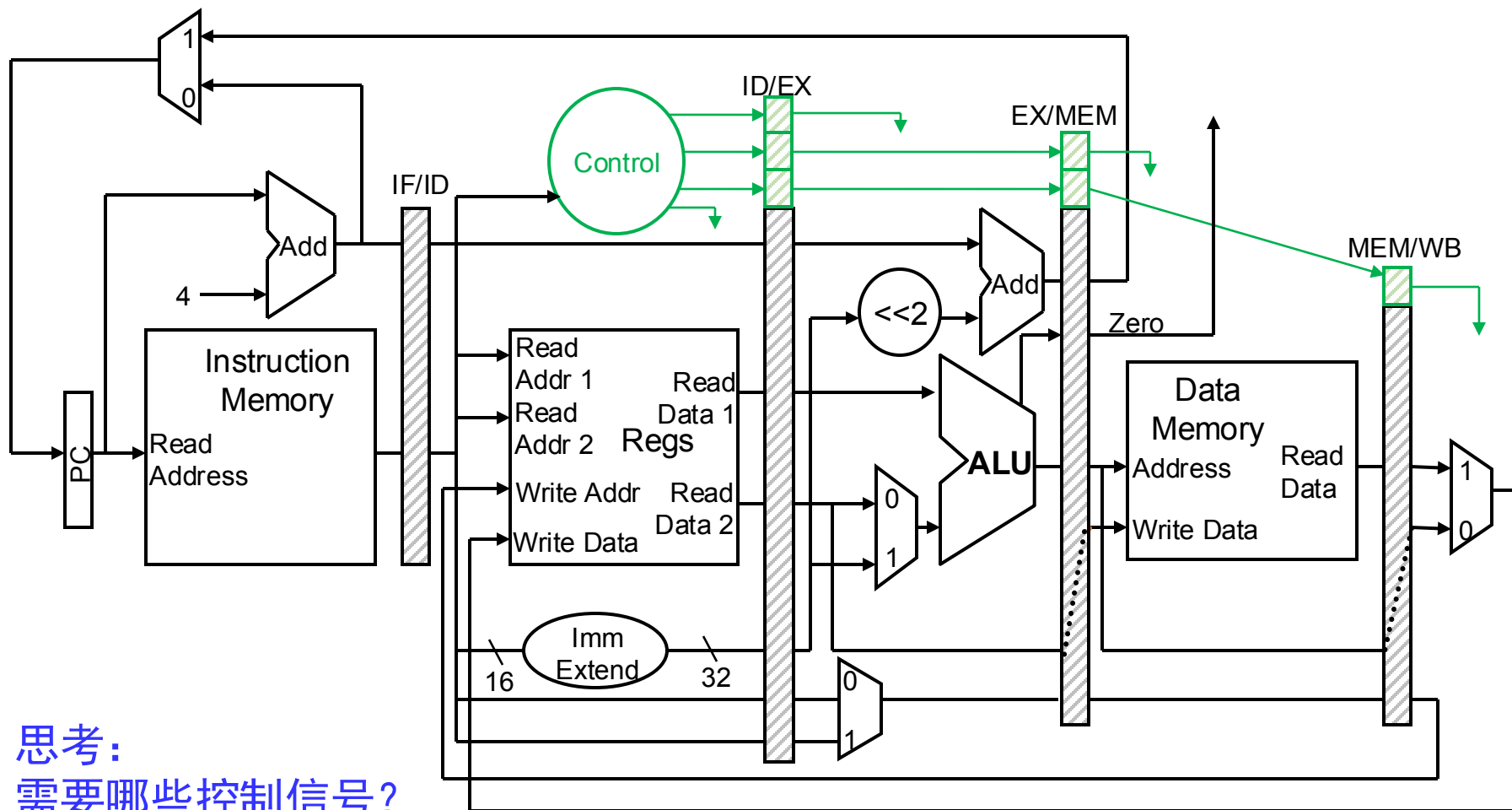


流水线运行示例



流水线的控制信号（不考虑冒险）

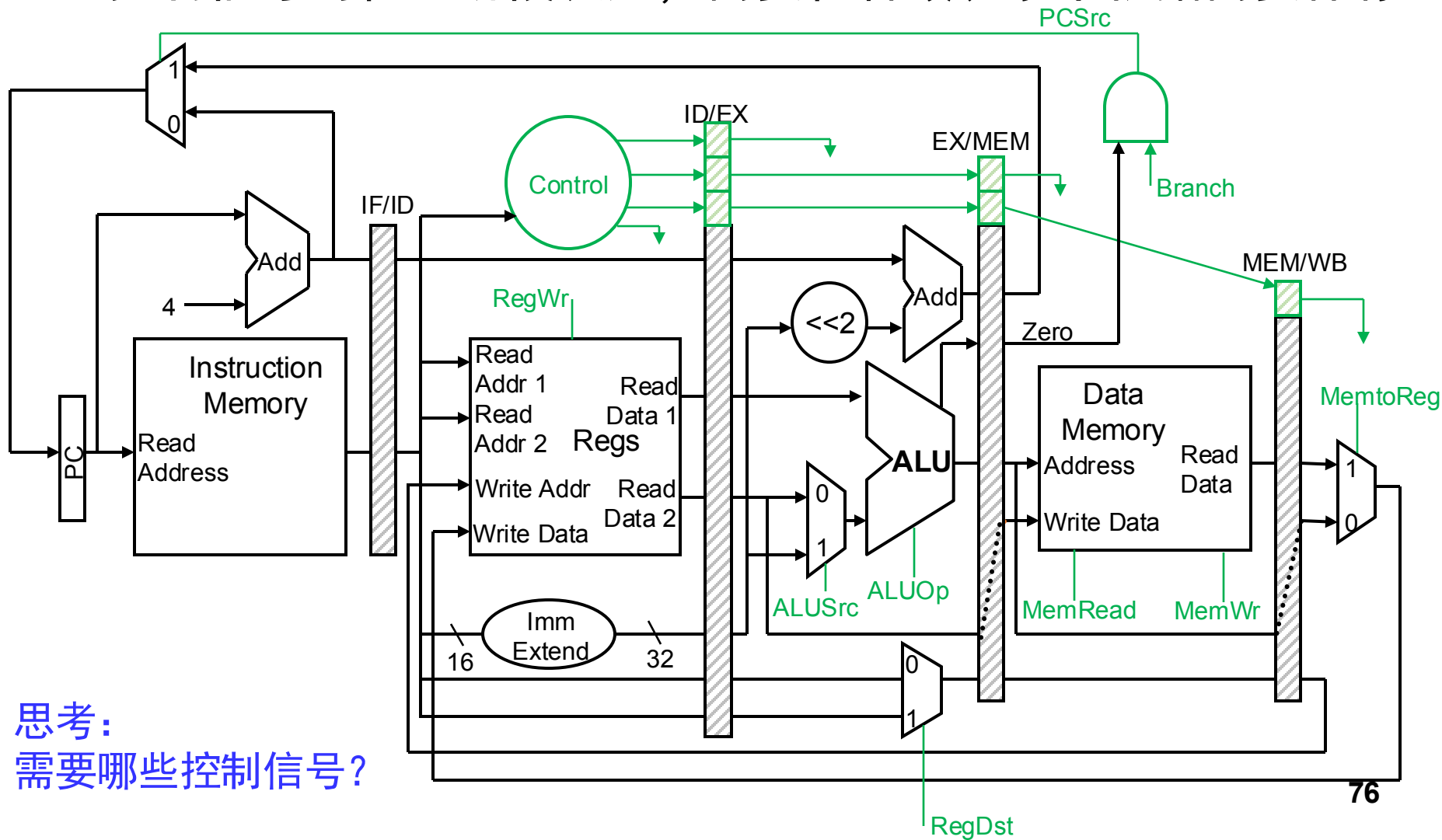
- 控制信号均可在ID阶段决定，并在各级流水中传递使用



思考：
需要哪些控制信号？

流水线的控制信号

- 控制信号均在ID阶段决定，需要在各级流水中根据需要保存



思考：
需要哪些控制信号？

流水线的控制信号：指令间对比

	EX Stage			MEM Stage			WB Stage	
	RegDst	ALUOp[1:0]	ALUSrc	Branch	MemRead	MemWr	RegWr	MemtoReg
R	1	10	0	0	0/X	0	1	0
lw	0	00	1	0	1	0	1	1
sw	X	00	1	0	0	1	0	X
beq	X	01	0	1	0	0	0	X

Q: 为什么不考虑流水线状态寄存器的写入使能信号?

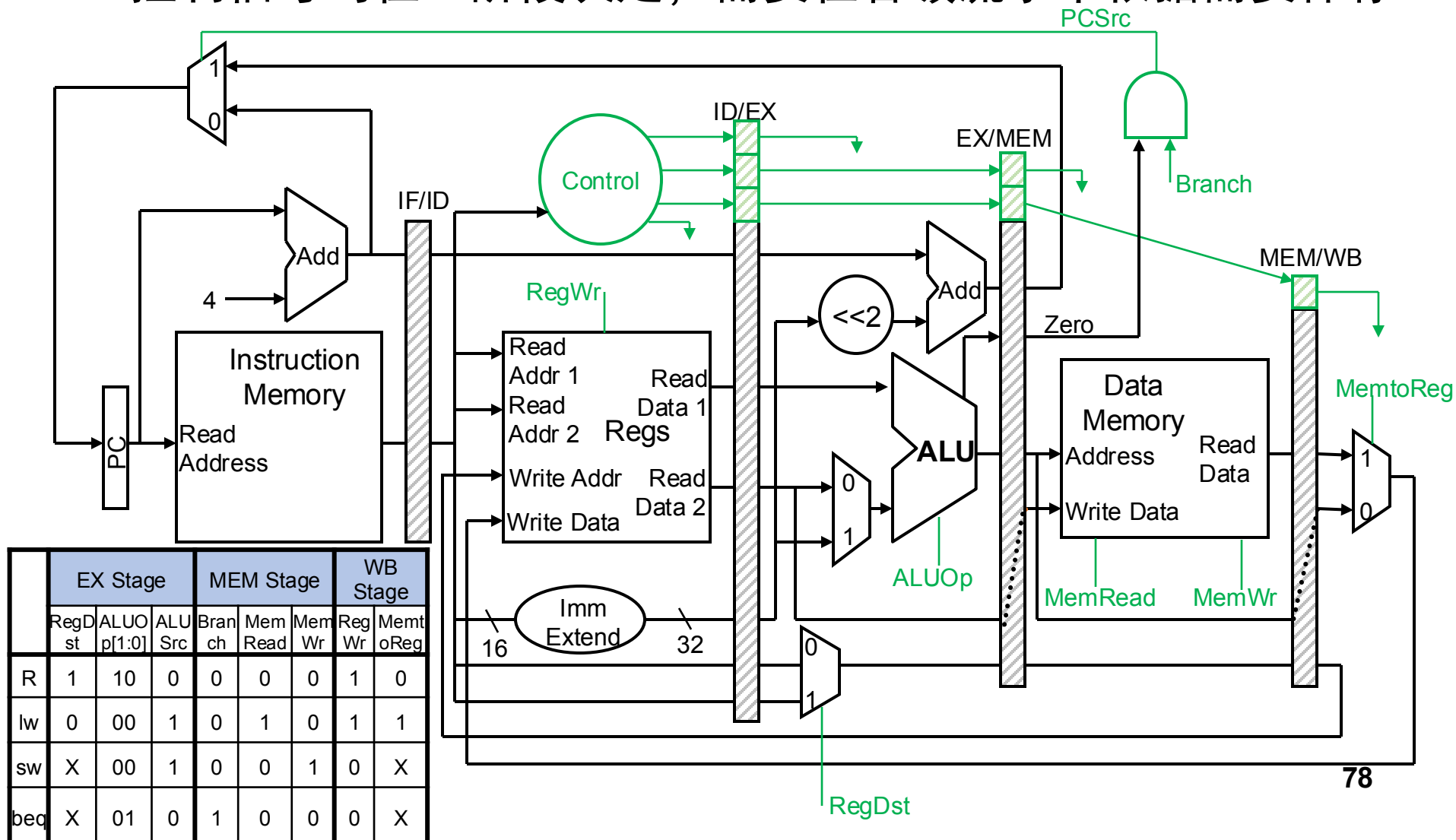
A: (目前认为) 总是写入的, 可认为是恒定有效

Q: 为什么没考虑IF和ID阶段的寄存器的控制信号?

A: 对于所有指令都是相同的

流水线的控制信号

- 控制信号均在ID阶段决定，需要在各级流水中根据需要保存



流水线数据通路的实现（考虑冒险）

- 需要能够检测和处理冒险问题
 - **结构冒险**—硬件资源使用冲突
 - **数据冒险**—数据存储器或者寄存器内容因指令关联而不可获得RAW（写后读），load-use
 - **控制冒险**—因跳转分支指令使得PC无法及时确定

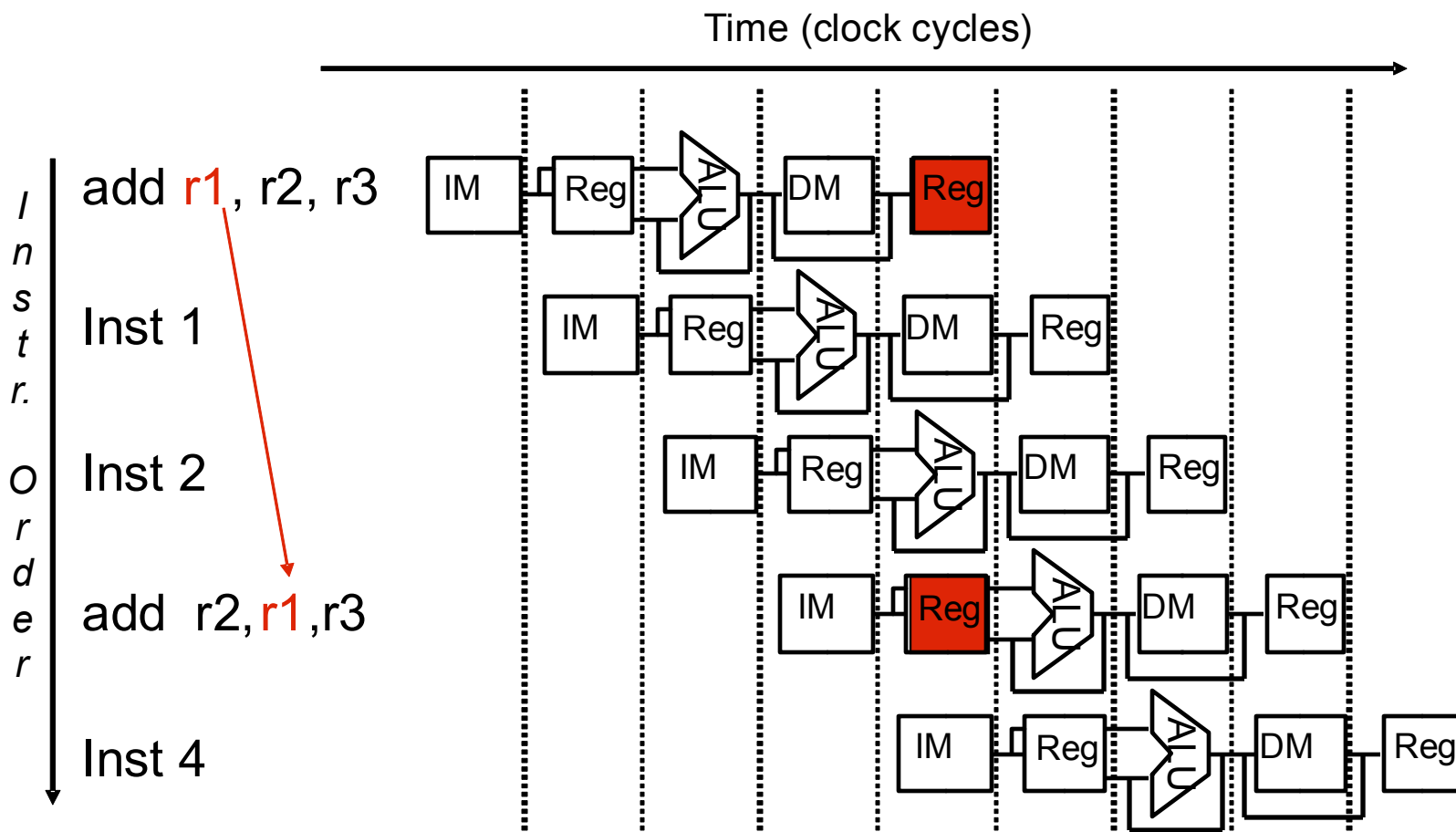
流水线数据通路的实现（考虑冒险）

关键：数据通路和控制信号的设计

- 控制信号如何随流水线运行？
 - 在各个流水级之间增加控制信号的寄存器
 - 对应级用对应控制信号
 - 不再使用的控制信号可不再传递
- 将各个资源和状态(流水级)联系起来
 - 从设计上要避免结构冒险的出现: 保证资源是够用的
- 找到所有的数据冒险和控制冒险问题
 - 如果是寄存器关联引起的=> data hazard: forward/stall
 - 如果是和PC有关的竞争=> control hazard

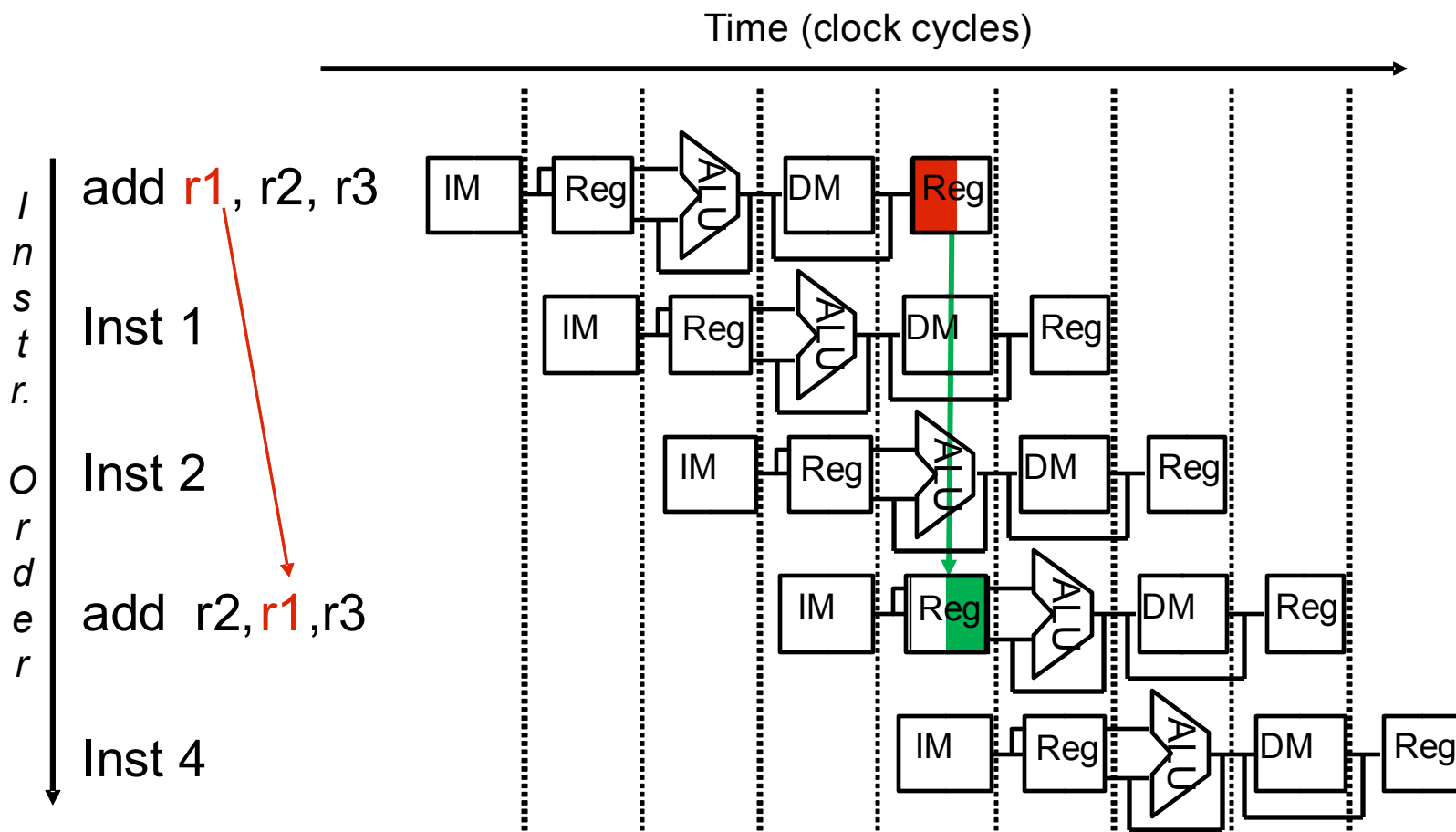
寄存器堆的访问

- 同一周期读取和写入寄存器堆：依靠硬件解决冲突



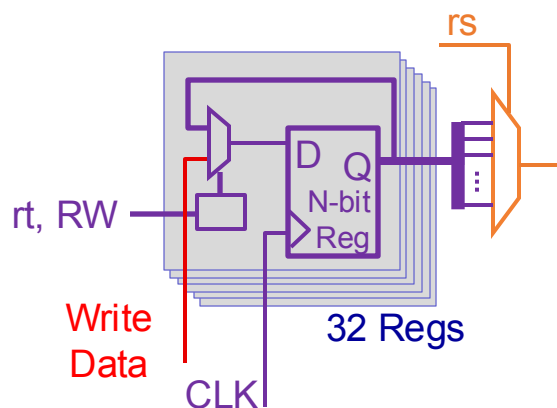
寄存器堆的访问

- 同一周期读取和写入寄存器堆：依靠硬件解决冲突

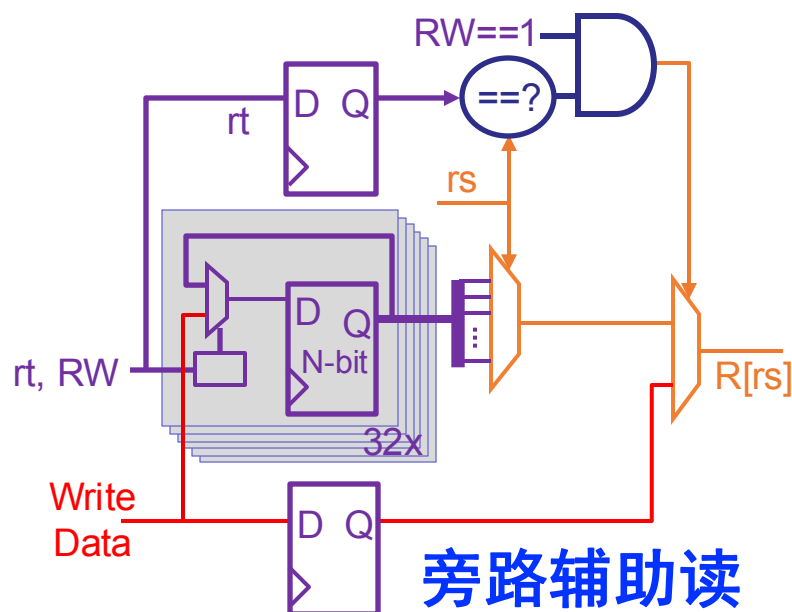


数据转发检测和控制-0

- 时钟触发边沿写入，且支持“**先写后读**”
 - 写入到RF的数据在同周期内可以直接被访问
 - RF/Memory阵列大，“写+读”较慢
- 可使用**辅助旁路**，减少“先更新再读出”的延时
 - 如果为刚写入的数据，从旁路读取



先写后读



旁路辅助读

本节课目录

- 复习：流水线基本概念 P4
- 复习：流水线数据通路 P6
- 流水线的冒险 P10
- **数据通路的实现（考虑冒险）**
 - 结构冒险 P57
 - **数据冒险 P86**
 - 控制冒险

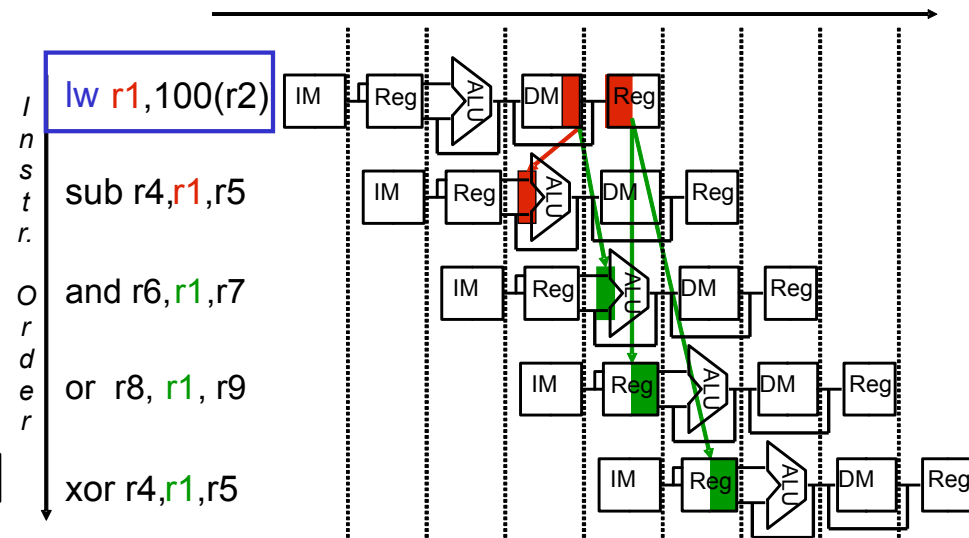
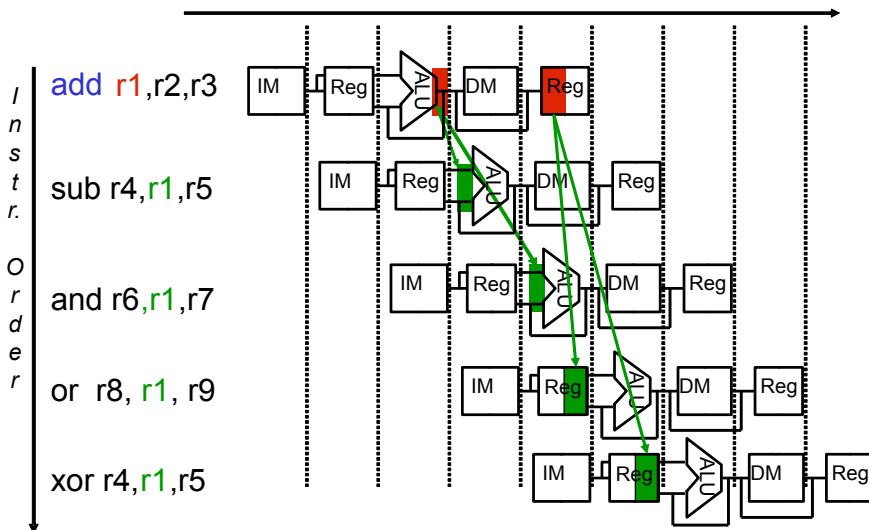
流水线数据通路的实现（考虑冒险）

关键：数据通路和控制信号的设计

- 控制信号如何随流水线运行？
 - 在各个流水级之间增加控制信号的寄存器
 - 对应级用对应控制信号
 - 不再使用的控制信号可不再传递
- 将各个资源和状态(流水级)联系起来
 - 从设计上要避免结构冒险的出现：保证资源是够用的
- 找到所有的数据冒险和控制冒险问题
 - 如果是寄存器关联引起的 → data hazard: forward/stall
 - 如果是和PC有关的竞争 → control hazard

数据转发: Datapath

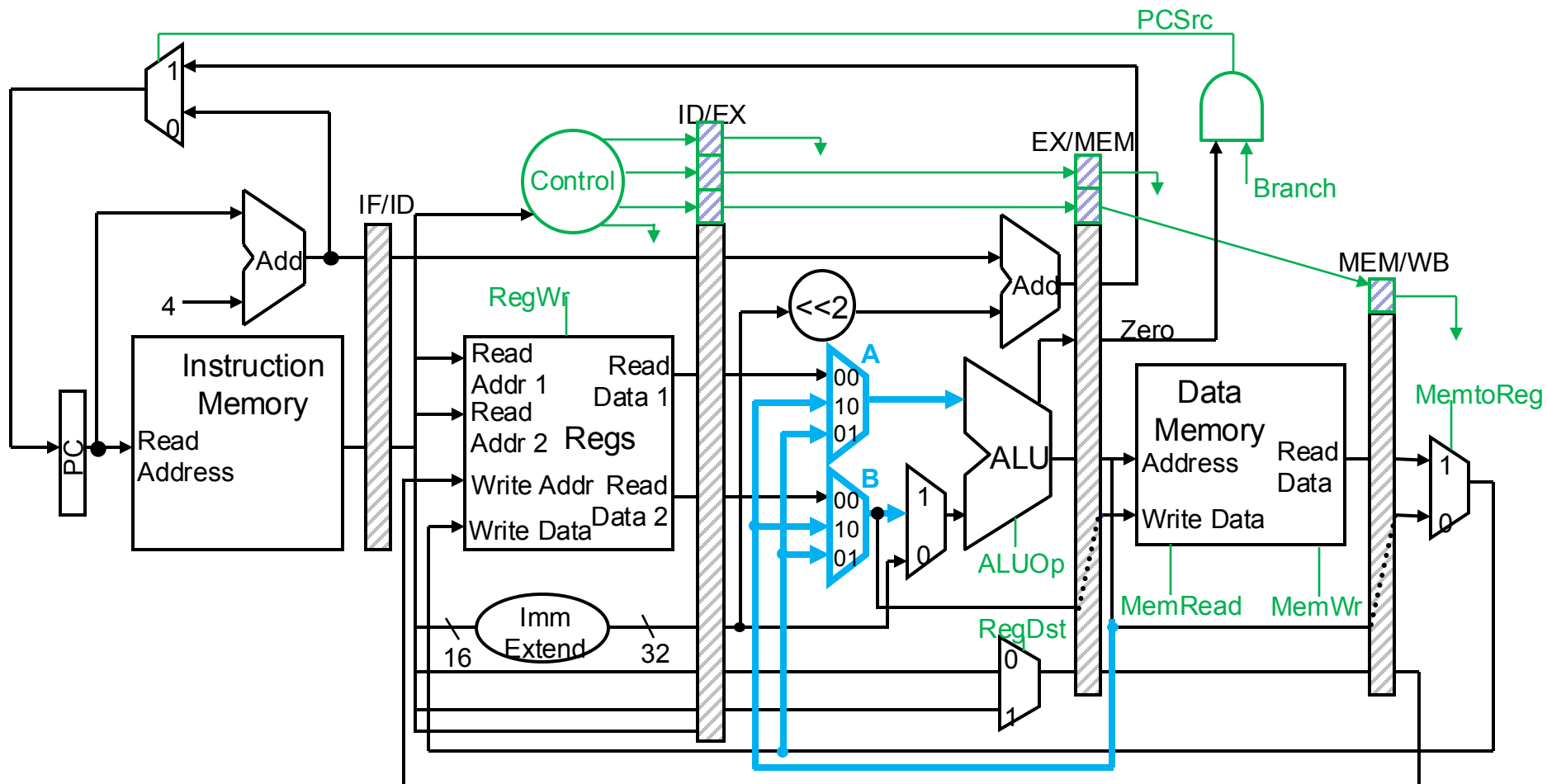
- 数据关联时提前提供数据
 - EX阶段结束前, ALU 运算结果已经就绪
 - MEM阶段结束前, LW指令读取的数据已经就绪



数据转发: Datapath

- 数据关联时提前提供数据
 - EX阶段结束前, 可给出ALU 运行结果或者地址访问时的有效地址
 - MEM阶段结束前, 可提前给出LW指令的结果数据
- 实现: ALU的输入可以直接从后面一级或者两级电路获得, 通过在ALU输入端加多路选择器方式
 - 在前向 A/B 的控制下, ALU 输入端有 MUX, ALU 输出端可作为 EX 阶段后续指令的 ALU 输入端 (Rs、Rt) 进行前向处理
 - 00: 正常数据流 (ID/EX pipeline registers)
 - 10: 从(EX/MEM pipeline registers)前一条指令forward
 - 01: 从(MEM/WB pipeline registers)前两条指令forward
- 上面的forward设计免除大多数数据关联的影响

包含转发的流水线通路



Forwarding数据通路

数据转发检测和控制-1

1. EX/MEM hazard: 要写入的寄存器号(Rd or Rt)

if (**EX/MEM**.RegWrAddr == ID/EX.RegisterRs))

ForwardA = 10

Rs寄存器号

将结果从**前一条指令**回馈到ALU输入端

if (**EX/MEM**.RegWrAddr == ID/EX.RegisterRt))

ForwardB = 10

Rt寄存器号

控制forward多路选择器MUX A/B (见上页)

2. MEM/WB hazard:

if (**MEM/WB**.RegWrAddr == ID/EX.RegisterRs))

ForwardA = 01

将结果从**前前条指令**回馈到ALU输入端

if (**MEM/WB**.RegWrAddr == ID/EX.RegisterRt))

ForwardB = 01

思考：上述条件完整吗？

- 指令是否真的需要更新寄存器：检查xxx.RegWrite
- 有些寄存器不会被写入，例如\$0：检查xxx.RegWrAddr

数据转发检测和控制-2

1. EX/MEM hazard:

```
if (EX/MEM.RegWrite  
and (EX/MEM.RegWrAddr == ID/EX.RegisterRs))  
    ForwardA = 10  
if (EX/MEM.RegWrite  
and (EX/MEM.RegWrAddr == ID/EX.RegisterRt))  
    ForwardB = 10
```

将结果从前一条更新寄存器的指令回馈到ALU输入端

2. MEM/WB hazard:

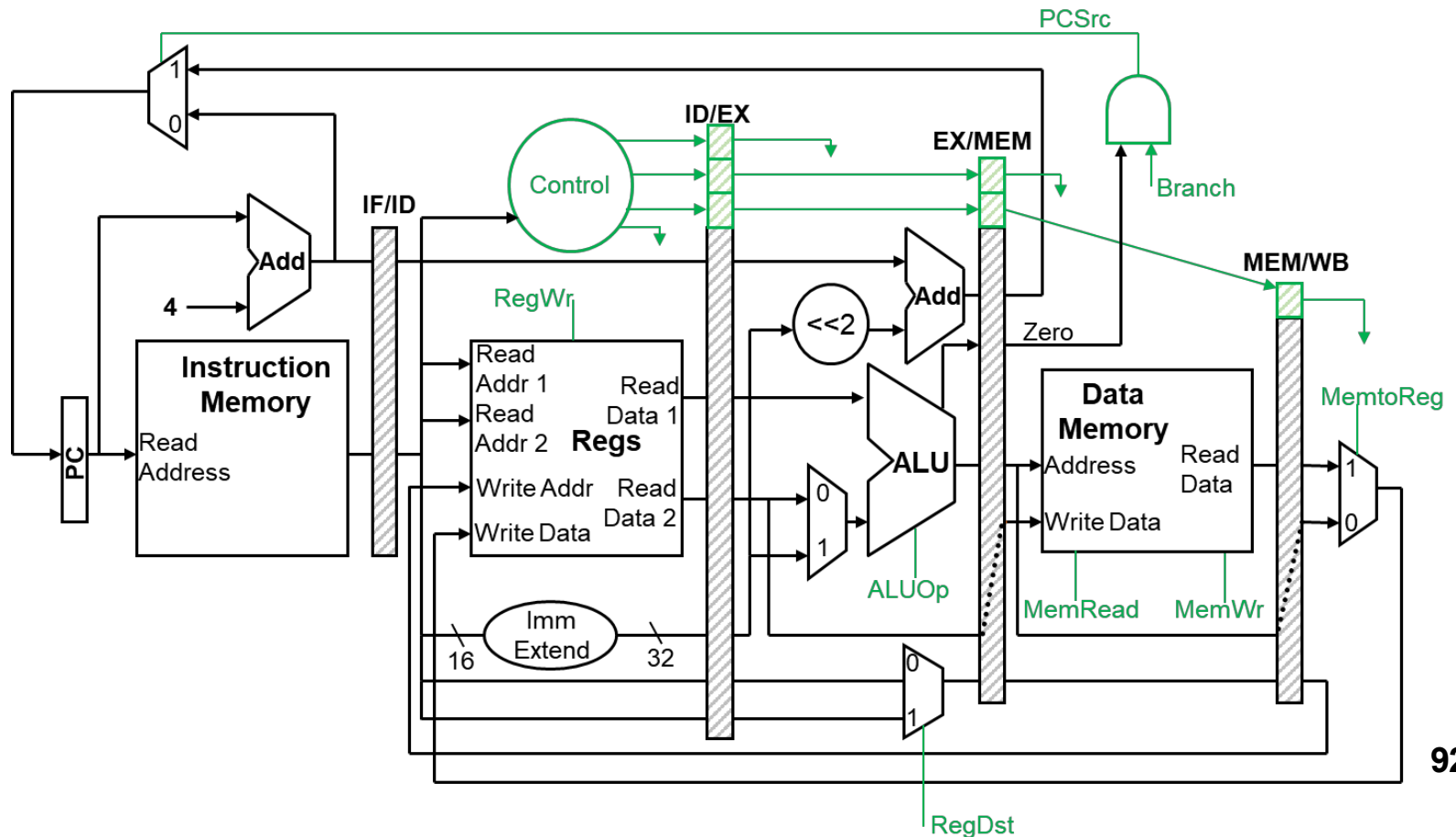
```
if (MEM/WB.RegWrite  
and (MEM/WB.RegWrAddr == ID/EX.RegisterRs))  
    ForwardA = 01  
if (MEM/WB.RegWrite  
and (MEM/WB.RegWrAddr == ID/EX.RegisterRt))  
    ForwardB = 01
```

将结果从前前条更新寄存器的指令回馈到ALU输入端

数据转发检测和控制-2

- \$0寄存器为常量，一直为0: `sllv $0, $1, $2`

6'b0	rs	rt	rd=0	5'b0	000100
------	----	----	------	------	--------



数据转发检测和控制-3

1. EX/MEM hazard:

```
if (EX/MEM.RegWrite
and (EX/MEM.RegWrAddr != 0)
and (EX/MEM.RegWrAddr == ID/EX.RegisterRs))
    ForwardA = 10
if (EX/MEM.RegWrite
and (EX/MEM.RegWrAddr != 0)
and (EX/MEM.RegWrAddr == ID/EX.RegisterRt))
    ForwardB = 10
```

将结果从前一条更新寄存器（且不是\$0）的指令回馈到ALU输入端

2. MEM/WB hazard:

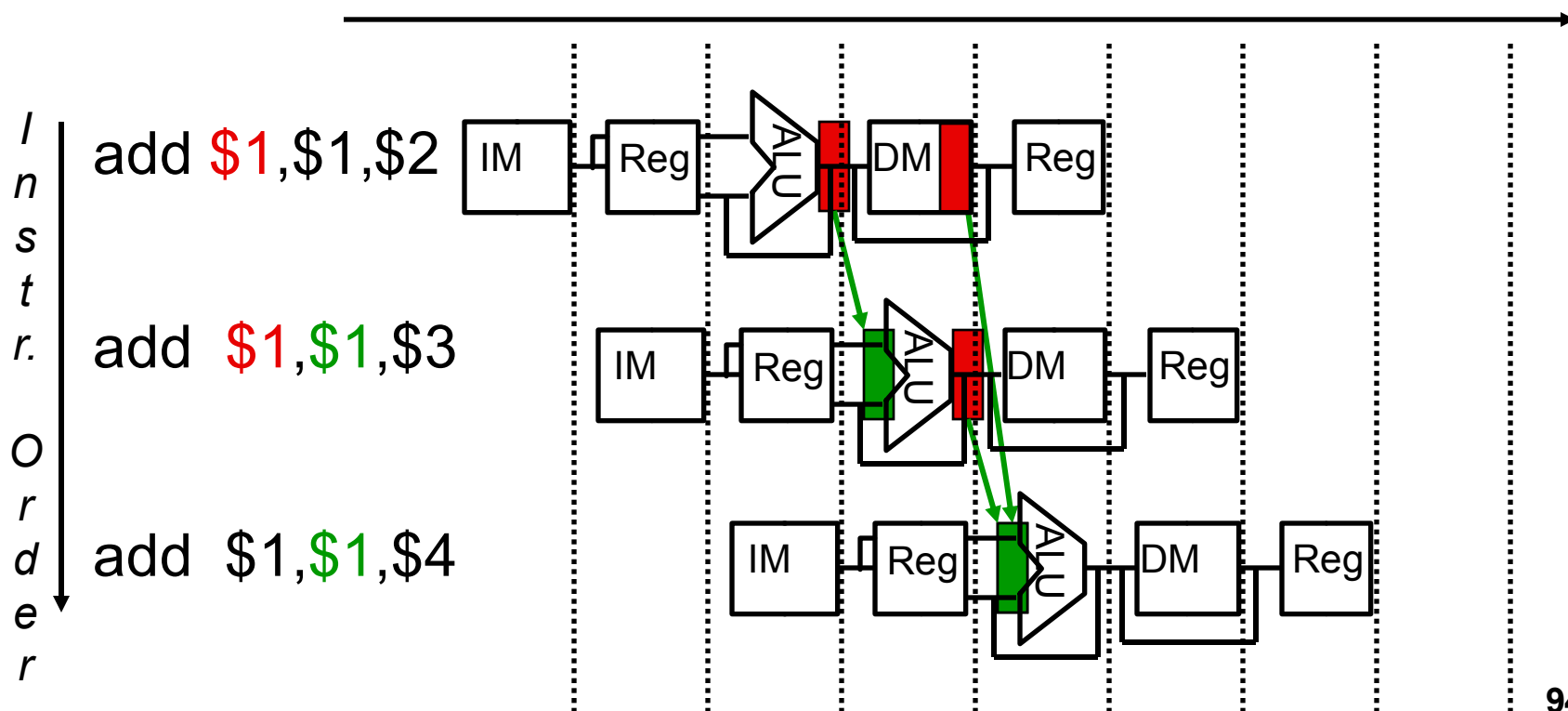
```
if (MEM/WB.RegWrite
and (MEM/WB.RegWrAddr != 0)
and (MEM/WB.RegWrAddr == ID/EX.RegisterRs))
    ForwardA = 01
if (MEM/WB.RegWrite
and (MEM/WB.RegWrAddr != 0)
and (MEM/WB.RegWrAddr == ID/EX.RegisterRt))
    ForwardB = 01
```

将结果从前前条更新寄存器（且不是\$0）的指令回馈到ALU输入端

是否存在其他问题？

数据转发检测和控制-3

- 如果存在两条转发路径？
 - 连续三条指令都有关联：前两条指令都是以关联寄存器为目的寄存器Rd的时候
 - 转发最新的数据：如果转发EXE/MEM.ALUOut数据，那就是最新的



数据转发检测和控制-3

问题：MEM/WB.ALUOut数据的转发是否成立？

2. MEM/WB hazard:

if (MEM/WB.RegWrite

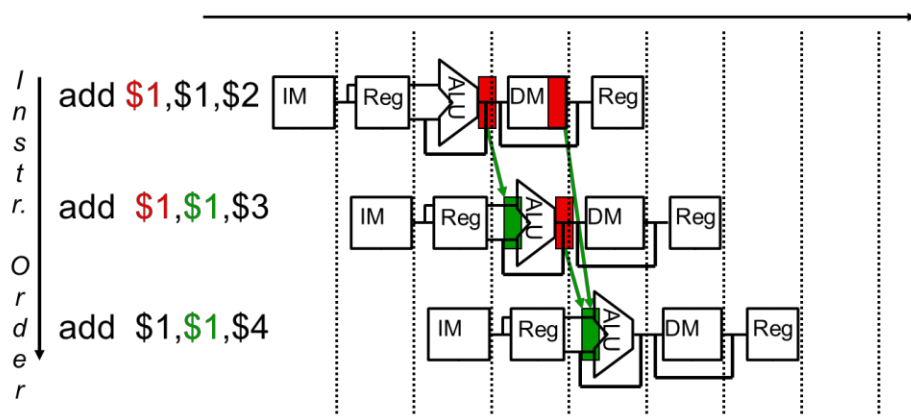
and (MEM/WB.RegWrAddr != 0)

and (MEM/WB.RegWrAddr == ID/EX.RegisterRs)

and (EX/MEM.RegWrAddr != ID/EX.RegisterRs || ~ EX/MEM.RegWrite)
)

ForwardA/B = 01: 从(MEM/WB pipeline registers)前前条指令forward

ForwardA = 01



前前条指令Forward (from MEM/WB)成立的条件

- ✓ 这条指令要写入寄存器
- ✓ 写入的不是Register \$0
- ✓ 目标寄存器号 == 源寄存器号
- ✓ 不从前一条指令EX/MEM转发

数据转发检测和控制-3

问题：MEM/WB.ALUOut数据的转发是否成立？

2. MEM/WB hazard:

if (MEM/WB.RegWrite

and (MEM/WB.RegWrAddr != 0)

and (MEM/WB.RegWrAddr == ID/EX.RegisterRs)

and (EX/MEM.RegWrAddr != ID/EX.RegisterRs || ~ EX/MEM.RegWrite)
)

ForwardA = 01

if (MEM/WB.RegWrite

and (MEM/WB.RegWrAddr != 0)

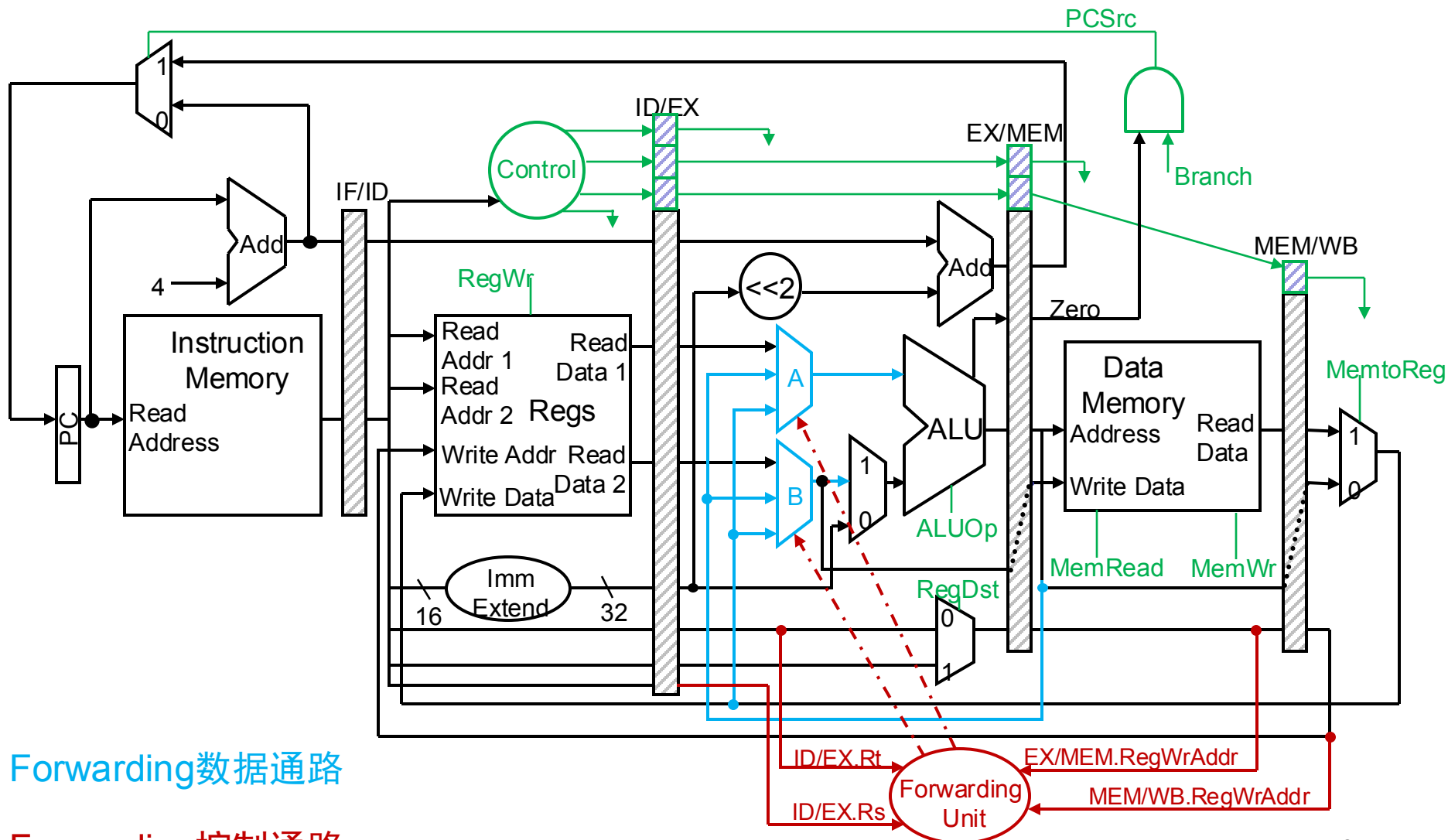
and (MEM/WB.RegWrAddr == ID/EX.RegisterRt)

and (EX/MEM.RegWrAddr != ID/EX.RegisterRt || ~ EX/MEM.RegWrite)
)

ForwardB = 01

FowardA/B = 01: 从(MEM/WB
pipeline registers)前前条指令forward

包含转发的流水线通路

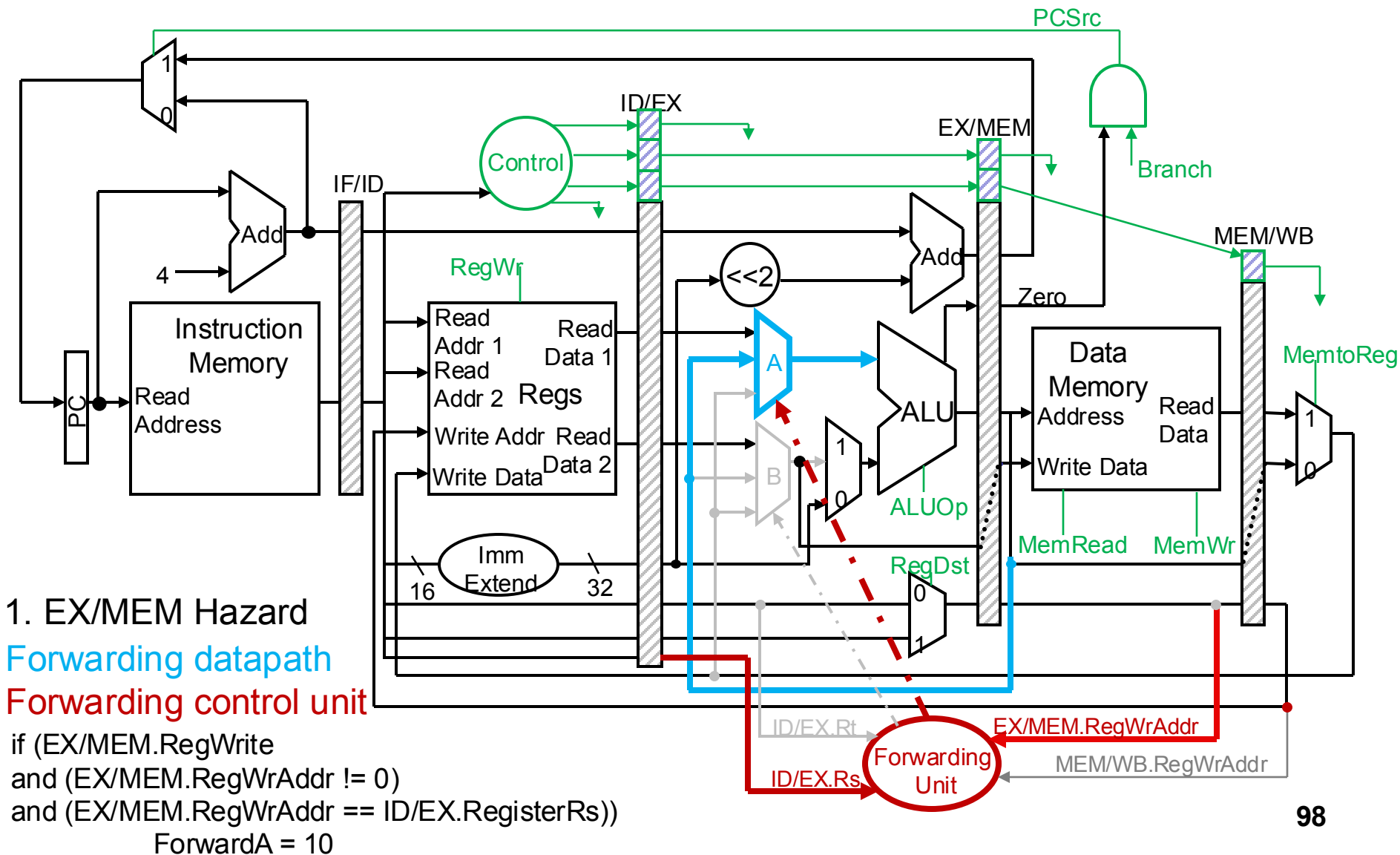


Forwarding数据通路

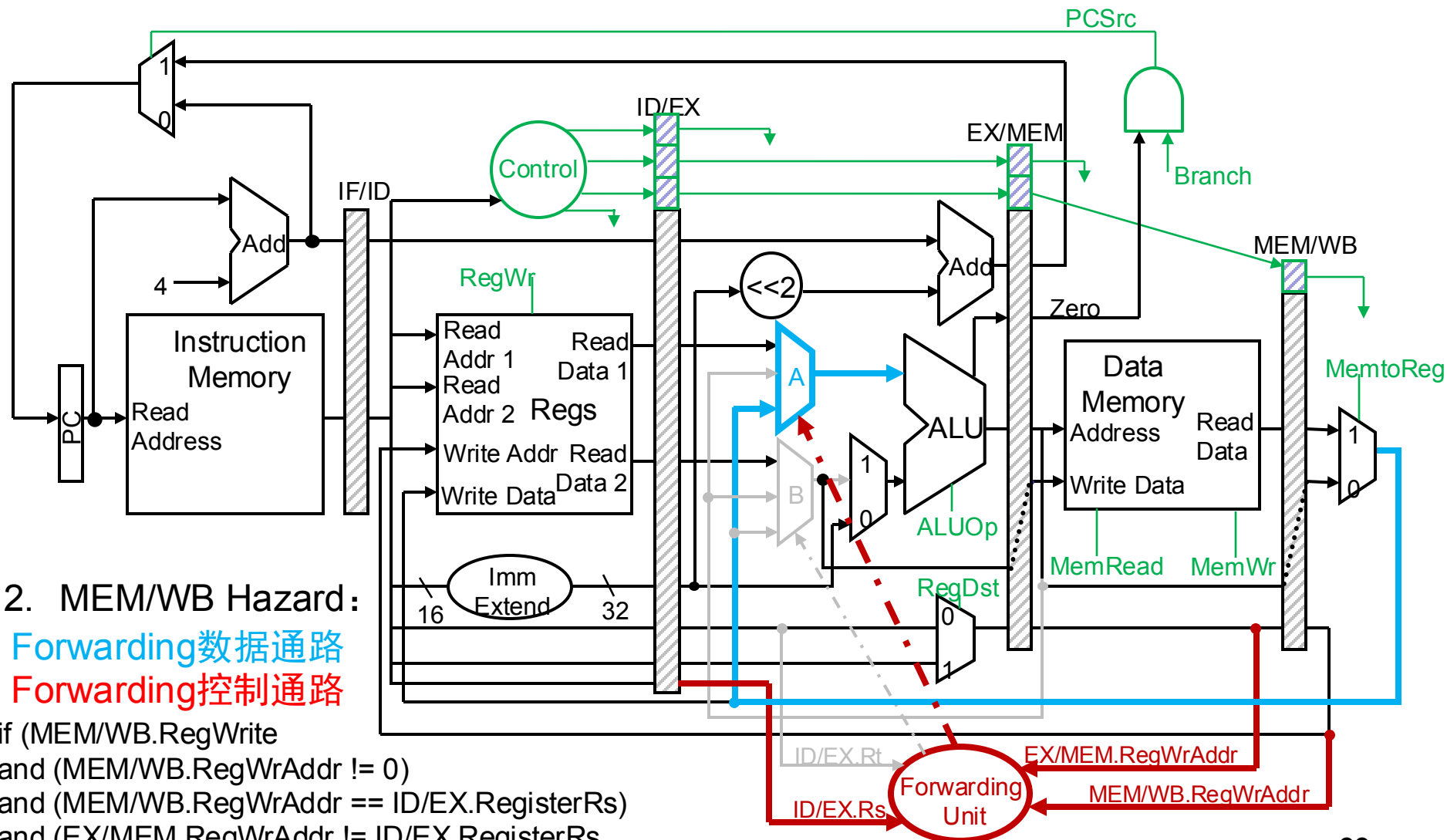
Forwarding控制通路:

输入: ID/EX.Rt, ID/EX.Rs, EX/MEM.RegWrAddr, MEM/WB.RegWrAddr

包含转发的流水线通路：示例



包含转发的流水线通路：示例

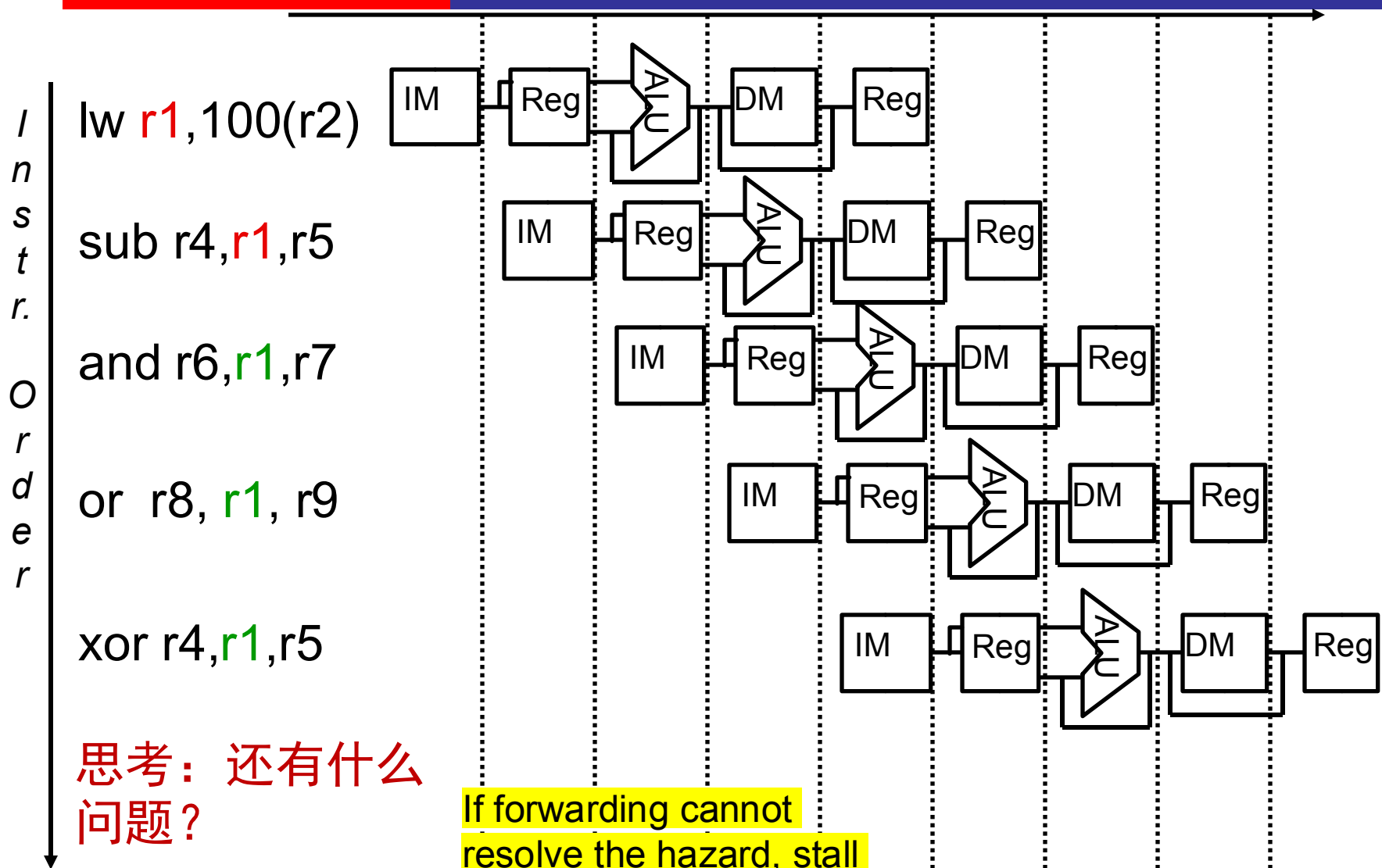


2. MEM/WB Hazard: Forwarding数据通路 Forwarding控制通路

if (MEM/WB.RegWrite
and (MEM/WB.RegWrAddr != 0)
and (MEM/WB.RegWrAddr == ID/EX.RegisterRs)
and (EX/MEM.RegWrAddr != ID/EX.RegisterRs
|| ~ EX/MEM.RegWrite))

ForwardA = 01

数据转发: Load-use Hazards

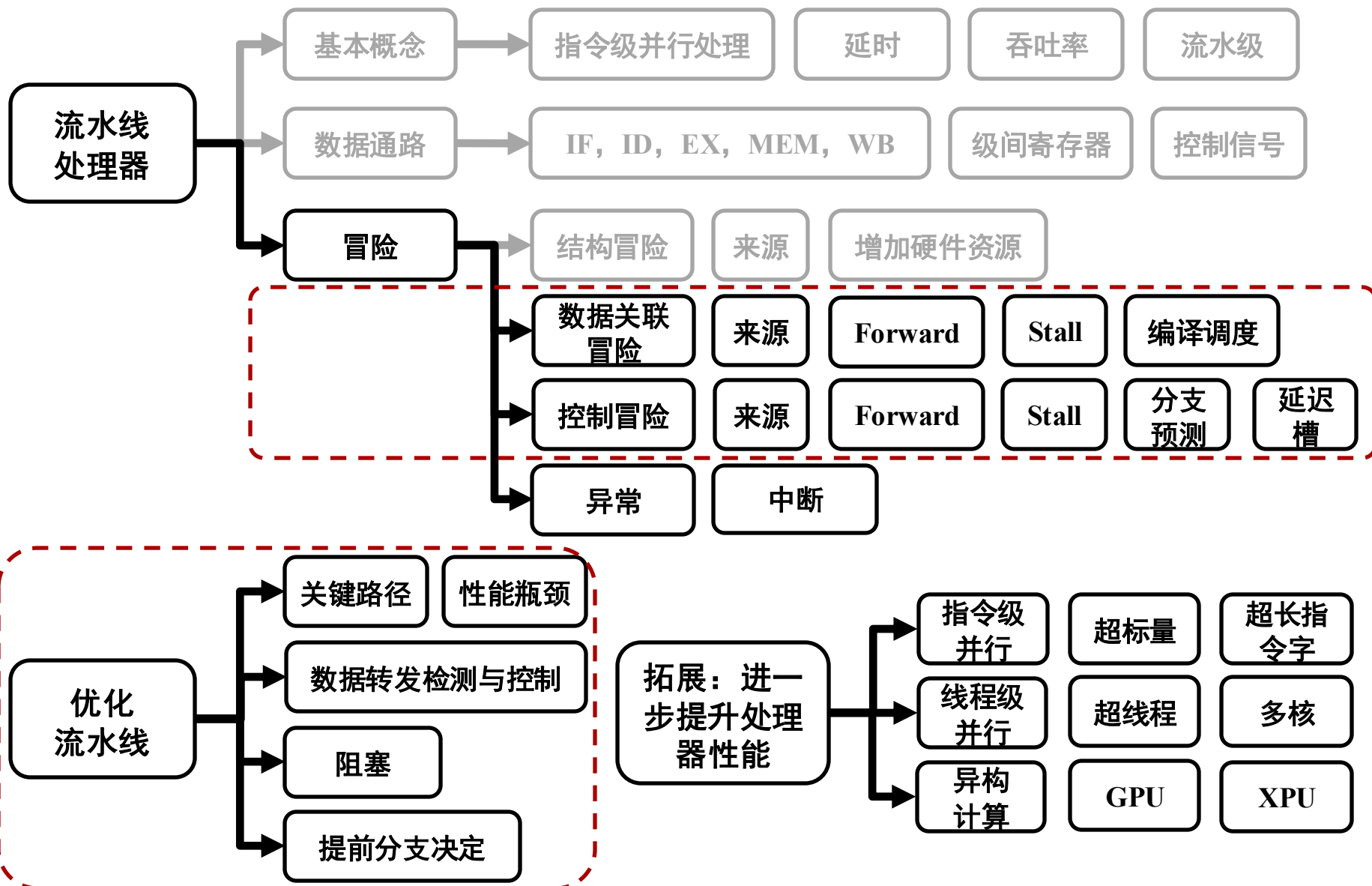


If forwarding cannot resolve the hazard, stall or other action is needed

小结

- **结构冒险**—硬件资源使用冲突
- **数据冒险**—数据存储器或者寄存器内容因指令关联而不可获得RAW（写后读），load-use
- **控制冒险**—因跳转分支指令使得PC无法及时确定

知识点框图



小结

- 流水线的RTL描述： 尝试自行完成整个表格？

流水线	R类型	读存储器	写存储器	分支
IF	IR <= MemInst[PC]; PC <= PC + 4			
ID	op <= IR[31:26]; A <= Reg[IR[25:21]]; B <= Reg[IR[20:16]]; 分支: If(A == B) PC <= PC + signext(IR[15:0]) << 2 Possible forwarding sources?			
EX	ALUOut <= A op B EX/MEM.ALUOut MEM/WB.ALUOut MDR	ALUOut <= A + signext(IR[15:0]) EX/MEM.ALUOut MEM/WB.ALUOut MDR		
MEM		MDR <= MemData[ALUOut];	MemData[ALUOut] <= B Possible forwarding sources?	
WB	Reg[IR[15:11]] <= ALUOut	Reg[IR[20:16]] <= MDR		

流水线结构与冒险小结

- **流水线结构**
 - 5级经典方式：IF ID EXE MEM WB
 - 流水线作用（理论与实际的吞吐率，提升潜力：级数）
 - 代价（单条指令延迟）
 - 时钟频率瓶颈（最慢的一级）
- **冒险及应对策略**
 - 结构冒险：寄存器读取&写入、使用哈佛架构MEM、ALU分离
 - 数据冒险：Forwarding、Stall、编译调度
 - 控制冒险：Stall、Forwarding、延迟槽、分支预测
- **要求**
 - 冒险来源：检测与区分冒险
 - 冒险的解决方法：数据与结构的关联性
 - 数据通路设计：修改指令执行流

本节课学习目标

- **知识点：概念、原理和指标**
 - 单周期、多周期、流水线处理器的区别
 - 流水线的基本概念、数据通路（执行过程）
 - 流水线中的冒险起因与分类
- **能力**
 - 掌握设计和分析流水线处理器的方法
 - 掌握分析和解决流水线中冒险的方法
- **思想**
 - 时空复用
 - 并行加速

作业

- 要求
 - 上传电子版至网络学堂
 - 不要抄袭或迟交
- 作业
 - 教材第7章习题5, 6, 11, 14(1), 15(1)
 - 题目可能与教材有微小差别, 以网络学堂作业附件为准